

Observatório Nacional

DISSERTAÇÃO DE MESTRADO

# PIPELINE PARA DETECÇÃO AUTOMATIZADA DE OCULTAÇÕES ESTELARES EM CURVAS DE LUZ COM TÉCNICAS DE MACHINE LEARNING

THIAGO LAIDLER VIDAL CUNHA

RIO DE JANEIRO
2026

# Agradecimentos

Gostaria de expressar minha sincera gratidão ao meu orientador, Dr. Julio Camargo, por acreditar neste projeto desde o início. Sua confiança em uma proposta ambiciosa, mesmo diante das limitações impostas pela minha rotina profissional, foi fundamental para que este trabalho se tornasse realidade.

Agradeço também aos integrantes do Grupo do Rio pela colaboração e disposição em ajudar ao longo desta pesquisa. Um agradecimento especial aos meus ex-orientadores, Dr. Marcelo Assafin e Dr. Sérgio Santos-Filho, cuja orientação durante a Iniciação Científica construiu a base sólida que tornou esta etapa muito mais leve.

Registro aqui minha gratidão ao Observatório Nacional por acolher meu projeto, mesmo em condições atípicas. Ser aceito como mestrando, ainda que sem vínculo de bolsa e conciliando atividades profissionais, foi uma oportunidade de ouro.

Aos meus amigos e à minha família, obrigado por estarem presentes durante toda essa jornada. Em especial, aos meus pais, Cesar Vidal Cunha e Adriana Vieira Laidler, e ao meu irmão, Matheus Laidler Vidal Cunha, por todo o amor, incentivo e apoio nos momentos mais difíceis. Agradeço também ao apoio da minha namorada, Daniela Ferreira de Monte Lima, que acompanhou meus devaneios sobre trabalhar com astrofísica desde a adolescência e esteve ao meu lado quando terminei esta dissertação.

Aos meus tios, Cláudio Vidal e Cláudia Lassance, agradeço pelo apoio constante e por terem me proporcionado, desde cedo, o aprendizado da língua inglesa, ferramenta indispensável na minha formação. Às minhas tias Alessandra Vieira Laidler e Christiane Vieira Laidler, pelo carinho e incentivo. Ao meu avô, capitão Oswaldo Laidler, agradeço pelo incentivo e inspiração ao longo de todos esses anos. Aos meus primos Yan Lassance e Raphael Lassance, agradeço muito por todo o apoio ao longo do caminho.

Este trabalho é dedicado a todos vocês.

*"O primeiro princípio é que você não deve enganar a si mesmo — e você é a pessoa mais fácil de enganar."*

— Richard Feynman

Thiago Laidler Vidal Cunha

PIPELINE PARA DETECÇÃO AUTOMATIZADA DE OCULTAÇÕES ESTELARES EM CURVAS DE LUZ COM TÉCNICAS DE MACHINE LEARNING

## RESUMO

Entre as técnicas disponíveis para a caracterização física de asteroides, Centauros e objetos transnetunianos, as ocultações estelares destacam-se pela elevada precisão. Nesses eventos, um corpo do Sistema Solar bloqueia temporariamente a luz de uma estrela distante para um dado observador. A análise da curva de luz resultante permite inferir tamanho, forma projetada e, em casos favoráveis, atmosfera e anéis do corpo ocultante, com resolução espacial da ordem de quilômetros. O ~~crescimento do volume de dados produzido~~ número crescente de curvas obtidas por redes de telescópios ~~e catálogos como o Gaia,~~ — impulsionado pela astrometria do catálogo Gaia, que permite prever esses eventos com mais precisão — tornou, no entanto, ~~tornou~~ a triagem manual de todas as curvas um problema relevante e motivou o desenvolvimento de métodos automatizados de detecção. Esta dissertação apresenta uma *pipeline* completa para detecção automatizada de ocultações estelares em curvas de luz, baseada em técnicas de aprendizado de máquina. O problema é formulado como classificação binária: cada curva é descrita por um vetor de características numéricas (*features*) e classificada como positiva (ocultação presente) ou negativa (ausência de evento). Quatro classificadores foram treinados e comparados: Regressão Logística, Random Forest, XGBoost e CatBoost. O conjunto de dados reúne 1693 curvas (802 positivas e 891 negativas). Seis experimentos avaliaram o impacto da seleção de *features* e a generalização para dados exclusivamente reais. Os resultados demonstram desempenho elevado e consistente: F1-score entre 0,978 e 0,994 e AUC-ROC superior a 0,995 no conjunto de teste. ~~A *pipeline* permite ainda o ajuste do limiar de decisão em pós-processamento, operando em regime de alta sensibilidade sem necessidade de retreinamento. Nesse contexto~~Como, na triagem, o custo de perder um evento genuíno (falso negativo) supera o de revisar um alarme falso (falso positivo), a *pipeline* permite ajustar o limiar de decisão em pós-processamento e operar em regime de alta sensibilidade, sem necessidade de retreinamento. Como validação em dados reais, utilizou-se a ocultação pelo objeto transnetuniano (50000) Quaoar, não incluída no conjunto de treinamento. A aplicação da ferramenta permitiu avaliar a identificação de ocultações em uma curva de luz complexa. Embora os sinais produzidos pelos anéis finos apresentem probabilidades absolutas baixas, estas são cerca de duas ordens de grandeza superiores às atribuídas a trechos de ruído semelhantes. Como consequência, um ajuste simples do limiar é suficiente para recuperar esses eventos sutis sem introduzir falsos alarmes. ~~Os resultados demonstram~~ Esses resultados indicam o potencial da ferramenta para revisitar

curvas de luz complexas em busca de eventos mais sutis ainda não identificados numa primeira análise. Assim, este trabalho contribui com uma ferramenta reproduzível para a triagem e priorização de curvas de luz em estudos de ocultações estelares.

Thiago Laidler Vidal Cunha

AUTOMATED PIPELINE FOR DETECTING STELLAR OCCULTATIONS IN LIGHT CURVES USING MACHINE LEARNING

## ABSTRACT

Among the techniques available for the physical characterization of asteroids, Centaurs and trans-Neptunian objects, stellar occultations stand out for their high precision. In such events, a Solar System body temporarily blocks the light of a distant star as seen by a given observer. Analysis of the resulting light curve allows one to infer the size, projected shape and, in favorable cases, the atmosphere and rings of the occulting body, with spatial resolution on the order of kilometers. The growing ~~data volume produced~~ number of light curves obtained by telescope networks ~~and catalogs such as Gaia~~ — driven by Gaia astrometry, which allows these events to be predicted more precisely — has, however, made the manual screening of all light curves a relevant problem and motivated the development of automated detection methods. This dissertation presents a complete pipeline for the automated detection of stellar occultations in light curves, based on machine learning techniques. The problem is cast as binary classification: each curve is described by a vector of numerical features and classified as positive (occultation present) or negative (no event). Four classifiers were trained and compared: Logistic Regression, Random Forest, XGBoost and CatBoost. The dataset comprises 1693 light curves (802 positive and 891 negative). Six experiments assessed the impact of feature selection and the generalization to exclusively real data. The results demonstrate high and consistent performance: F1-scores between 0.978 and 0.994 and AUC-ROC above 0.995 on the test set. ~~The pipeline also supports decision-threshold tuning as a post-processing step, operating in a high-recall regime without the need for retraining. In this context~~Since, in screening, the cost of missing a genuine event (false negative) outweighs that of reviewing a false alarm (false positive), the pipeline allows the decision threshold to be tuned as a post-processing step, operating in a high-recall regime without the need for retraining. As a validation on real data, the occultation by the trans-Neptunian object (50000) Quaoar, not included in the training set, was used. Applying the tool made it possible to assess the identification of occultations in a complex light curve. Although the signals produced by the thin rings have low absolute probabilities, these are about two orders of magnitude higher than those assigned to similar noise segments. As a consequence, a simple threshold adjustment is enough to recover these subtle events without introducing false alarms. ~~The results demonstrate~~ These results indicate the potential of the tool to revisit complex light curves in search of subtler events that a first analysis leaves unidentified. This work thus contributes a reproducible tool for the screening and prioritization of light curves in

stellar-occultation studies.

# Lista de Figuras

2.1 Representação artística de uma ocultação estelar pelo Centauro (10199) Chariklo e seu sistema de anéis. A curva de luz esquemática (abaixo) mostra as quedas de fluxo correspondentes à passagem dos anéis e do corpo principal diante da estrela. Crédito: Lucie Maquet, 2014. . . . . 8

2.2 Exemplo completo do fluxo observacional: ocultação estelar por Umbriel (satélite de Urano), observada em 21 de setembro de 2020 (Assafin et al. , 2023). (a) Mapa de previsão da sombra sobre a superfície terrestre. (b) Detalhe do posicionamento dos observadores na costa leste dos EUA (Google Earth). (c) ~~Exemplo de quadros~~ Quadros CCD ~~obtidos por~~ de uma das estações, processados pelo PRAIA: ~~são imagens como estas que alimentam os pacotes de fotometria~~ imagem original (~~e.g. PRAIA~~esquerda)~~para a construção das curvas de luz. Neste caso~~, ~~o brilho intenso~~ modelo do halo de Urano ~~exigiu~~ (centro) e imagem após a ~~aplicação de~~ subtração por coronagrafia digital ~~para viabilizar~~ (direita), que viabiliza a fotometria ~~precisa~~ de Umbriel e da estrela. (d) Curva de luz de ocultação resultante~~da redução via PRAIA~~: razão entre o fluxo ~~normalizado~~ da estrela somado ao de Umbriel e o fluxo de calibração (azul), e ~~ruído de~~ fundo de céu (verde)~~, com a queda de ocultação claramente visível~~. . . . . 10

2.3 Componentes do modelo de curva de luz sintética utilizado pelo SORA (Gomes-Júnior et al. , 2022). Verde: modelo geométrico, o ponto de partida — corpo opaco de borda nítida ocultando uma estrela pontual, sem difração e com amostragem instantânea. Preto: o mesmo modelo incluindo a difração de Fresnel~~(azul), tamanho~~ . Azul: incluindo também o diâmetro angular da estrela. Traços vermelhos: resposta instrumental, ~~ciclo de integração do detector~~ e isto é, ~~resultante da combinação dessas componentes,~~ o ~~modelo geométrico~~ valor que cada exposição registraria ao integrar a curva azul durante o tempo de exposição (~~verde~~a largura de cada traço). ~~O ajuste simultâneo aos~~ A curva sintética ajustada a dados ~~observados~~ reais combina todos esses efeitos (~~pontos vermelhos) permite determinar com precisão os instantes de imersão e emersão, necessários para~~ Figura 2.4a~~reconstrução da forma do corpo~~). . . . . 13

2.4 (a) Curva de luz de ocultação observada (preto) e ~~modelo ajustado~~ curva sintética ajustada (vermelho) para uma das cordas da ocultação por Umbriel em 2020; os instantes de imersão e emersão ~~, marcados por circunferências, definem a geometria~~ encontram-se muito próximos dos pontos circulados nas bordas da ~~corda~~ queda (Assafin et al. , 2023). (b) Perfil do limbo de Umbriel ~~; no plano do céu:~~ cada linha azul ~~(incerteza vermelho) corresponde a é~~ uma corda positiva projetada~~no plano do céu~~ , com as incertezas dos extremos em vermelho (Assafin et al. , 2023). . . . . 15

3.1 Exemplo ilustrativo de regressão linear simples~~(~~, com dados genéricos, não referentes a curvas de luz de ocultação~~) (Levada , 2026)~~. Fonte: Levada (2026) . . . . . 20

3.2 ~~Aplicação da regressão linear ao problema de classificação~~ Classificação binária ~~: a reta produz saídas fora~~ (sim = 1, não = 0) em função do ~~intervalo [0, 1] (Levada , 2026). Nosso conjunto~~ tamanho da queda de ~~dados mostra quatro amostras de curvas de luz com grandes amplitudes e outras quatro com baixas amplitudes~~fluxo (marcadores). ~~Nesse exemplo simples,~~ A linha tracejada indica o ~~valor~~ limiar de ~~θ~~decisão; a linha contínua, ~~5 para~~ o ~~recorte~~ resultado da ~~reta divide perfeitamente as duas classificaçõesregressão~~ linear aplicada aos dados (ver discussão no texto). Fonte: adaptado de Levada (2026). . . . . 22

3.3 ~~O mesmo problema anterior~~Como na Figura 3.2, ~~mas~~ com a adição de um ponto discrepante (*outlier*). ~~Uma das curvas possui tamanho~~ Fonte: adaptado de ~~amplitude muito diferente da escala normal, possivelmente resultado da falta de normalização~~Levada (2026).~~Veja como a reta de regressão se desloca e o limiar 0,5 passa a classificar incorretamente (Levada , 2026).~~ 23

3.4 Comparação entre as distribuições Gaussiana e de Laplace, que correspondem aos priors assumidos sobre os parâmetros na interpretação bayesiana das regularizações Ridge e Lasso, respectivamente. . . . . 26

3.5 Função sigmoide $\sigma(z) = 1/(1 + e^{-z})$. . . . . 26

3.6 Esquema simplificado do funcionamento de ~~uma Floresta Aleatória~~um Random Forest. . . . . 29

3.7 Exemplo ilustrativo: 25 árvores com acurácia individual em torno de 65% podem resultar em uma floresta com acurácia de aproximadamente 94%. . 30

3.8 Exemplos de curvas ROC . . . . . 37

3.9 Exemplo de curvas de aprendizado . . . . . 40

3.10 Esquema da validação cruzada com $q = 5$ dobras . . . . . 41

4.1 Esquema da *pipeline*: as curvas de luz são convertidas em vetores de características; o treinamento utiliza essas características, e não as séries temporais brutas. . . . . 44
4.2 Fluxograma da aquisição e organização dos dados externos e da integração na etapa de inserção no banco de dados. . . . . 46
4.3 Fluxograma da *pipeline* responsável pela consolidação do banco de dados. . 48

5.1 Matrizes de confusão dos quatro modelos no Experimento 1 (28 *features*). Eixos: real (vertical) e predito (horizontal). Classes: Negativa (0) e Positiva (1). . . . . 69
5.2 Curvas precisão-sensibilidade dos quatro modelos ~~(~~no Experimento~~5).~~ 5. Cada ~~ponto da~~ posição ao longo de uma curva corresponde a um valor ~~de~~ do limiar $\tau$~~: variando $\tau$ de 1 a 0.~~ No gráfico, ~~percorre-se a curva da esquerda (alta precisão, baixa~~ "revocação" é sinônimo de sensibilidade ~~)~~ ~~para a direita~~ (~~alta sensibilidade, menor precisão~~*recall*).~~O ponto ideal é o canto superior direito (precisão e sensibilidade próximas de 1); quanto mais a curva se aproxima dele, melhor o modelo. Como todos os modelos operam em valores muito altos, a região de interesse concentra-se acima de 0,9 em ambos os eixos.~~ . . . . . 75
5.3 Precisão, sensibilidade, F1-score e $F_2$-score em função do limiar de decisão $\tau$ (Experimento 5). (a) Random Forest e XGBoost; (b) CatBoost e Regressão Logística. . . . . 76
5.4 Curva de luz da ocultação estelar por Quaoar utilizada no estudo de caso, com o tempo referenciado ao instante 2022-08-09 06:34:49,26 UTC. O *dip* central (anotado como "Quaoar"), de duração $\approx$ 48 s e profundidade $\approx$ 100%, corresponde à ocultação pelo corpo principal. As estruturas secundárias rasas, isoladas nos quatro quadros ampliados (*insets*), são as assinaturas dos dois anéis~~denominados~~ , Q1R (Morgado et al. , 2023) e Q2R ~~em~~ (Pereira et al. , 2023), sendo Q1R observado em duas porções (parte contínua e parte densa) e Q2R em duas travessias antes e depois do corpo. A linha pontilhada indica o *baseline* (~~$f_{\text{norm}} = 1$~~$F_{\text{norm}} = 1$). . . . . 78
5.5 Recortes da curva Gemini-'Alopeke canal *Red*- filtro $z$ (visível-vermelho) de Quaoar, com o veredito do XGBoost em cada janela colorida. A legenda indica, por recorte, a probabilidade $\hat{p}$ e a classificação no limiar padrão ($\tau = 0{,}5$) e no limiar ajustado ($\tau = 0{,}03$). Os anéis finos Q2R, negativos em $\tau = 0{,}5$, são recuperados em $\tau = 0{,}03$, enquanto o controle e o trecho de ruído permanecem corretamente negativos. Tempo relativo a 2022-08-09 06:34:49,26 UTC. . . . . 80
5.6 Curvas ROC — Experimento 1 (28 *features*). . . . . 83

5.7 Importância de variáveis por modelo — Experimento 2 (configuração enxuta, 11 *features*). (a) Regressão Logística, (b) Random Forest, (c) XGBoost, (d) CatBoost. . . . . 84
5.8 Distribuição da *feature* `kmeans_centroid_dist` por classe (1–99 percentil). As curvas negativas concentram-se em valores baixos (regime único) e as positivas em valores altos (dois patamares), evidenciando o poder discriminativo da variável. . . . . 86
5.9 Curvas de aprendizado de (a) XGBoost e (b) CatBoost na configuração do Experimento 2: *logloss* de treino (azul) e de teste (vermelho) em função das iterações de *boosting*. . . . . 88

A.1 Matrizes de confusão dos quatro modelos no Experimento 4 (14 *features*). . 104
A.2 Matrizes de confusão dos quatro modelos no Experimento 5 (13 *features*, sem `Feature_Savgol_Min`). . . . . 106
A.3 Matrizes de confusão dos quatro modelos no Experimento 6 (12 *features*, sem `kmeans_centroid_dist`). . . . . 107
A.4 Matrizes de confusão dos quatro modelos no Experimento 2 (11 *features*). . 109
A.5 Matrizes de confusão dos quatro modelos na variante 65/35 do Experimento 3 (teste somente em curvas reais). . . . . 110
A.6 Curvas ROC — variante 65/35 do Experimento 3 (teste somente em curvas reais). . . . . 111
A.7 Curvas ROC — Experimento 4 (14 *features*). . . . . 111
A.8 Curvas ROC — Experimento 5 (13 *features*). . . . . 112
A.9 Curvas ROC — Experimento 6 (12 *features*, sem `kmeans`). . . . . 112
A.10 Curvas ROC — Experimento 2 (11 *features*). . . . . 113
A.11 Importância de variáveis por modelo — Experimento 4 (14 *features*). (a) Regressão Logística, (b) Random Forest, (c) XGBoost, (d) CatBoost. . . . 114
A.12 Importância de variáveis por modelo — Experimento 5 (13 *features*, sem `Feature_Savgol_Min`). (a) Regressão Logística, (b) Random Forest, (c) XGBoost, (d) CatBoost. . . . . 115

# Lista de Tabelas


3.1 Dados do exemplo numérico da Regressão Logística (valores ilustrativos). . 24
3.2 Algoritmos de otimização e seu papel na etapa de treinamento em projetos de ML. . . . . 34

5.1 Conjunto completo de 28 *features* extraídas pela *pipeline*. A coluna "Família" indica o grupo funcional; a coluna "Definição" resume o cálculo. . . . . 64
5.2 Conjunto de 13 *features* após a análise de redundância. O conjunto de referência operacional (Experimento 2) remove ainda `Feature_Savgol_Min` e `kmeans_centroid_dist`, restando 11. . . . . 65
5.3 Comparação dos métodos de cálculo de importância de *features* utilizados por cada modelo. . . . . 67
5.4 Distribuição das amostras por fonte no *dataset* utilizado nos experimentos. 68
5.5 Métricas de desempenho no conjunto de teste — Experimento 1. Conjunto completo de 28 *features*, divisão 80%/20%. . . . . 68
5.6 Métricas de desempenho no conjunto de teste — Experimento 2. 11 *features* (sem `Feature_Savgol_Min` e sem `kmeans_centroid_dist`), divisão 80%/20%. . . . . 70
5.7 Métricas de desempenho no conjunto de teste — Experimento 3. Teste somente em curvas reais (11 *features*, divisão 80%/20%; teste com 198 curvas: 160 positivas e 38 negativas). . . . . 71
5.8 Pontos operacionais selecionados na análise de limiar (Experimento 5, 13 *features*). Para cada modelo: limiar padrão ($\tau = 0{,}5$), limiar sensível ou ótimo, e métricas associadas (FP = falsos positivos, FN = falsos negativos). 74
5.9 Probabilidade de ocultação $\hat{p}$ atribuída por CatBoost e XGBoost (Exp. 2) à curva inteira (*baseline*) e a cada recorte da curva Gemini-'Alopeke *Red-z*. Janelas em segundos relativos ao instante 2022-08-09 06:34:49,26 UTC. . . 79
5.10 Comparação do F1-score entre os seis experimentos, por modelo. Exp. 1: 28 *features*; Exp. 2: 11 *features* (ablação combinada); Exp. 3: teste somente em curvas reais (198 curvas: 160 positivas e 38 negativas; ~~sintéticas~~ simuladas alocadas ao treino); Exp. 4: 14 *features*; Exp. 5: 13 *features*; Exp. 6: 12 *features* (sem `kmeans`). . . . . 82

5.11 Validação cruzada estratificada (~~$k = 5$~~ $q = 5$): média ± desvio padrão das métricas por modelo (Experimento 5). . . . . 86
5.12 Teste de McNemar entre pares de modelos (Experimento 4). $b$: apenas A acerta; $c$: apenas B acerta. $p < 0{,}05$ indicaria diferença significativa; nenhum par é significativo ($\alpha = 0{,}05$). . . . . 87

A.1 Métricas de desempenho no conjunto de teste — Experimento 4. Subconjunto de 14 *features*, divisão 80%/20%. . . . . 104
A.2 Métricas de desempenho no conjunto de teste — Experimento 5. 13 *features* (sem `Feature_Savgol_Min`), divisão 80%/20%. . . . . 105
A.3 Métricas de desempenho no conjunto de teste — Experimento 6. 12 *features* (sem `kmeans_centroid_dist`), divisão 80%/20%. . . . . 105
A.4 Comparação do F1-score: com `kmeans` (Exp. 5), sem `kmeans` (Exp. 6) e ablação combinada (Exp. 2). Divisão 80%/20%. . . . . 108
A.5 Métricas de desempenho no conjunto de teste — variante do Experimento 3 com divisão 65%/35% (teste somente em curvas reais: 347 curvas, 281 positivas e 66 negativas). . . . . 109

# Sumário

**Agradecimentos** **3**

**Lista de Figuras** **11**

**Lista de Tabelas** **15**

**1 Introdução** **1**
1.1 Contexto e Relevância . . . . . 1
1.2 Problema de Pesquisa . . . . . 3

**2 Ocultações Estelares e Processamento de Dados** **7**
2.1 O que são Ocultações Estelares? . . . . . 7
2.2 Da observação às curvas de luz: coleta e armazenamento . . . . . 11
2.3 ~~Características~~ Caracterização das curvas de luz ~~e detecção positiva/negativa~~de ocultação . . . . . 11
2.4 Considerações finais do capítulo . . . . . 14

**3 Modelos e algoritmos para ~~Machine Learning~~*Machine Learning*** **17**
3.1 Conceitos fundamentais de ~~Machine Learning~~*Machine Learning* . . . . . 17
3.1.1 Definição e papel do aprendizado de máquina . . . . . 17
3.1.2 Aprendizado supervisionado versus não supervisionado . . . . . 19
3.2 O problema da classificação supervisionada . . . . . 20
3.3 Modelos de classificação utilizados na ~~pipeline~~*pipeline* . . . . . 21
3.3.1 Regressão Logística . . . . . 21
3.3.2 Árvores de decisão e critérios de impureza . . . . . 26
3.3.3 Random Forest (Floresta Aleatória) . . . . . 28
3.3.4 XGBoost (~~eXtreme Gradient Boosting~~*eXtreme Gradient Boosting*) . 30
3.3.5 CatBoost . . . . . 31
3.4 Agrupamento não supervisionado: K-means . . . . . 32
3.5 Algoritmos de otimização no treinamento . . . . . 33
3.6 Engenharia de características e validação de modelos . . . . . 34
3.6.1 Engenharia de características . . . . . 34

3.6.2 Métricas de avaliação para classificação binária . . . 35
3.6.3 ~~Overfitting, underfitting~~ *Overfitting*, *underfitting* e o compromisso entre viés e variância . . . 36
3.6.4 Curvas de aprendizado . . . 39
3.6.5 Validação de modelos . . . 39

**4 Metodologia: ~~Pipeline~~ *Pipeline* de Detecção 43**
4.1 Visão geral da ~~pipeline~~*pipeline* . . . 43
4.2 Aquisição e armazenamento dos dados . . . 44
4.2.1 Fontes de dados . . . 44
4.2.2 Estrutura do banco de dados . . . 47
4.3 Acesso aos dados e normalização . . . 47
4.3.1 Consulta e classificação . . . 47
4.3.2 Normalização do fluxo . . . 49
4.3.3 Remoção de *outliers* . . . 49
4.4 Construção do conjunto de dados para aprendizado de máquina . . . 49
4.4.1 Tamanho e composição do conjunto de dados . . . 50
4.4.2 Curvas positivas e negativas do banco . . . 50
4.4.3 Recorte de segmentos negativos a partir de curvas positivas . . . 50
4.4.4 Curvas ~~sintéticas~~ negativas simuladas . . . 51
4.5 Engenharia de características . . . 52
4.5.1 ~~Estatísticas básicas~~ Amplitude e desvio padrão do fluxo . . . 52
4.5.2 Suavização ~~e médias móveis~~(filtro de Savitzky-Golay) . . . 53
4.5.3 ~~Max drawdown~~Queda máxima acumulada (*max drawdown*) . . . 53
4.5.4 K-means no fluxo suavizado . . . 54
4.5.5 Derivadas . . . 54
4.5.6 Comparação entre quartis (testes estatísticos) . . . 55
4.5.7 Métricas de profundidade, duração e ajuste de modelos . . . 55
4.5.8 Resumo das *features* . . . 57
4.6 Treinamento e avaliação dos modelos . . . 57
4.6.1 Preparação dos dados e divisão treino/teste . . . 57
4.6.2 Modelos utilizados . . . 58
4.6.3 Métricas de avaliação . . . 59
4.6.4 Fluxo de treinamento e persistência . . . 59

**5 Resultados 61**
5.1 Visão geral dos experimentos . . . 61
5.2 Descrição das *features* e análise de redundância . . . 63
5.2.1 Conjunto completo de *features* . . . 63
5.2.2 Análise de redundância . . . 63

5.3 Importância das *features*: metodologia . . . . . 65
5.3.1 Random Forest — *Mean Decrease in Impurity* (MDI) . . . . . 66
5.3.2 XGBoost — *Weight* (contagem de *splits*) . . . . . 66
5.3.3 CatBoost — *PredictionValuesChange* . . . . . 66
5.3.4 Regressão Logística — valor absoluto dos coeficientes . . . . . 67
5.3.5 Comparação entre os métodos . . . . . 67
5.4 Configuração experimental . . . . . 68
5.5 Resultados do Experimento 1 (linha de base, 28 *features*) . . . . . 68
5.6 Redução do conjunto de *features* . . . . . 69
5.6.1 Configuração enxuta: Experimento 2 (11 *features*) . . . . . 70
5.7 Teste de generalização: curvas exclusivamente reais . . . . . 71
5.7.1 ~~Motivação~~ . . . . . 71
5.7.1 ~~Resultados do Experimento 3~~ . . . . . 71
5.7.1 ~~Interpretação~~ . . . . . 71
5.8 Minimização de falsos negativos . . . . . 72
5.8.1 Motivação: assimetria de custo entre erros . . . . . 72
5.8.2 Método: variação do limiar de classificação . . . . . 73
5.8.3 Escolha do ponto operacional . . . . . 73
5.8.4 Resultados . . . . . 74
5.8.5 Implicações para uso operacional . . . . . 75
5.9 Estudo de caso: aplicação a uma curva real de Quaoar . . . . . 76
5.9.1 Motivação . . . . . 76
5.9.2 Procedimento . . . . . 77
5.9.3 Aplicação a recortes da curva: sondando os anéis . . . . . 79
5.9.4 Interpretação . . . . . 81
5.10 Comparação entre os experimentos . . . . . 82
5.10.1 Análise de importância das *features* . . . . . 83
5.11 Validação cruzada estratificada . . . . . 85
5.11.1 Teste de McNemar entre modelos . . . . . 87
5.12 Curvas de aprendizado dos modelos de *boosting* . . . . . 88
5.13 Discussão dos resultados . . . . . 89
5.13.1 Desempenho e generalização . . . . . 89
5.13.2 Análise de redundância . . . . . 89
5.13.3 Interpretação da importância das *features* . . . . . 90
5.13.4 Custo assimétrico e ajuste de limiar . . . . . 90

**6 Conclusões** **91**
6.1 Resumo dos achados . . . . . 91
6.2 Resposta à hipótese de pesquisa . . . . . 92

6.3 Contribuições originais . . . . . 92
6.4 Limitações . . . . . 93
6.5 Trabalhos futuros . . . . . 94

**Referências Bibliográficas** **97**

**A Resultados experimentais complementares** **103**
A.1 Experimento 4: subconjunto de 14 *features* . . . . . 103
A.2 Experimento 5: 13 *features* (sem `Feature_Savgol_Min`) . . . . . 105
A.3 Experimento 6: ablação isolada do ~~K-Means~~K-means . . . . . 105
A.4 Variante do Experimento 3 com divisão 65/35: ablação do tamanho do treino 108
A.5 Curvas ROC dos experimentos complementares . . . . . 110
A.6 Importância das *features* nos Experimentos 4 e 5 . . . . . 110

**B Hiperparâmetros dos classificadores** **117**

**C Ambiente de software e versões** **119**

# Capítulo 1

# Introdução

## 1.1 Contexto e Relevância

O Sistema Solar não nasceu em seu estado atual, mas foi esculpido por interações gravitacionais a partir do colapso de um núcleo pré-estelar, formando um disco de gás e poeira regulado por campos magnéticos sob condições extremas de temperatura e pressão (Morbidelli et al. , 2024). Nesse sentido, o estudo da dinâmica de seus constituintes é o primeiro passo em direção a compreender como sistemas planetários se formam e evoluem com o tempo.

Ao longo da história, o mapeamento do Sistema Solar evoluiu de registros rudimentares do movimento dos astros para análises sofisticadas de sua dinâmica. Áreas como Astronomia Fundamental e Mecânica Celeste passaram a fornecer modelos matemáticos e simulações que revelam os caminhos que levaram à configuração planetária atual.

De forma geral, a metodologia científica é fundamentada em processos de decisão baseados em dados: hipóteses são confrontadas com observações, e a interpretação confiável exige métodos estatísticos que quantifiquem incertezas e permitam distinguir sinal de ruído (Wall & Jenkins , 2003). Nesse contexto, a combinação de observações, cada vez mais precisas, e modelos mais sofisticados possibilitou importantes avanços na descrição de Sistemas Planetários.

Nas últimas décadas, técnicas numéricas como a integração direta das equações de movimento permitiram esclarecer processos antes apenas sugeridos teoricamente - entre eles, a migração dos planetas gigantes e a reorganização das regiões de ressonância, elementos do Modelo de Nice (Gomes , 2009). São esses avanços que ajudam a reconstituir um passado dinâmico e caótico que moldou o Sistema Solar tal como o observamos hoje.

~~Dentro desse panorama~~Nesse contexto, o Sistema Solar ~~Exterior surge como~~ exterior é a região menos processada pelo Sol desde a formação do sistema planetário e, portanto, um laboratório natural ~~. Nele, destacam-se~~ para estudar as condições e os mecanismos primordiais, cujos registros ali se preservam (Gomes , 2009; Morbidelli et al. , 2024). Nele

se destacam os **Centauros** ~~, em~~ — pequenos corpos com órbitas heliocêntricas instáveis entre Júpiter e Netuno ~~,~~— e os **Objetos Transnetunianos** (do inglês, *Trans-Neptunian Objects*, TNOs), com órbitas além da ~~órbita~~ de Netuno. A caracterização física desses corpos — tamanho, forma, albedo, densidade e presença de atmosfera, anéis ou satélites — é essencial para compreender a origem e a evolução dinâmica do nosso sistema planetário (Ortiz et al. , 2020). Esses corpos têm tamanhos típicos da ordem de centenas de quilômetros e distâncias de 15 a 100 UA do Sol; a combinação de pequeno porte e grande distância resulta em baixo brilho aparente e dificulta medições precisas. ~~Ainda assim, por preservarem características primitivas, carregam pistas valiosas sobre a formação e a evolução do Sistema Solar.~~ Até recentemente, propriedades físicas fundamentais - tamanho, forma, massa, atmosfera - só podiam ser inferidas por métodos indiretos (fotometria, espectroscopia, telescópios espaciais, etc.). A determinação de diâmetros, em particular, dependia sobretudo de técnicas radiométricas baseadas em telescópios espaciais (Fornasier et al. , 2013). No entanto, um tipo de observação terrestre, de custo relativamente baixo e acessível a telescópios modestos, passou a constituir uma técnica bastante relevante para a determinação ~~de diâmetros e formas~~da forma e do tamanho de pequenos corpos do Sistema Solar: **as ocultações estelares**.

Ocultações estelares ocorrem quando um corpo do Sistema Solar passa diante de uma estrela distante, bloqueando temporariamente sua luz ~~(Knieling et al. , 2024). A técnica tornou-se uma das mais precisas para a determinação de dimensões e formas projetadas desses objetos, sendo superada nesse aspecto apenas por encontros diretos com sondas espaciais~~para um dado observador na Terra (Knieling et al. , 2024; Ortiz et al. , 2020). A análise da **curva de luz de ocultação** -—a variação temporal do fluxo ~~estelar -~~ da estrela registrada durante o evento, que não deve ser confundida com as curvas de luz rotacionais usadas para determinar períodos de rotação — permite estimar a posição do objeto, inferir sua forma projetada e identificar estruturas periféricas~~(satélites, ~~, como satélites e anéis ) ~~(Cazeneuve et al. , 2023); sua versatilidade vai de asteroides próximos a TNOs extremamente distantes (Fraser et al. , 2024). A técnica tornou-se uma das mais precisas para a determinação de dimensões e formas projetadas desses objetos, sendo superada nesse aspecto apenas por encontros diretos com sondas espaciais. Ocultações são ainda empregadas há décadas no estudo de atmosferas planetárias (Zhu et al. , 2022).

~~O crescimento do~~ Esse sucesso tem um preço. O volume de dados gerados por campanhas observacionais ~~, porém, tem tornado~~ cresceu muito, impulsionado (i) pela precisão astrométrica dos catálogos do satélite Gaia (Gaia Collaboration et al. , 2016, 2023), que tornou as previsões de ocultação muito mais confiáveis, e (ii) pelo barateamento de câmeras astronômicas rápidas e sensíveis, que popularizou esse tipo de observação entre astrônomos amadores e redes de ciência cidadã. Com isso, a identificação manual dos eventos tornou-se uma tarefa custosa, ~~impulsionando~~ o que impulsiona a adoção de métodos computacionais, em particular de técnicas de **aprendizado de máquina** (do inglês,

*Machine Learning*, ML). Neste contexto, ML refere-se a algoritmos que, a partir de dados de exemplo, ajustam internamente seus parâmetros para realizar uma tarefa (por exemplo, decidir se uma curva contém ou não uma ocultação), em vez de seguir regras fixas programadas à mão. No contexto das ocultações, o ML tem sido usado para detecção confiável de eventos, mitigação de ruídos e caracterização de sinais (Cazeneuve et al. , 2023): redes neurais convolucionais ~~(como o ODNet)~~ (como a ODNet, Cazeneuve et al. , 2023) analisam séries temporais ou imagens da estrela alvo e de estrelas de referência, distinguindo ocultações reais de flutuações atmosféricas ou artefatos instrumentais; em buscas por TNOs em campos densos, redes residuais fazem triagem dos candidatos gerados por técnicas de empilhamento e deslocamento (*shift-and-stack*), reduzindo detecções espúrias (Fraser et al. , 2024). Além disso, processos gaussianos têm sido explorados para modelar curvas de luz e extrair parâmetros temporais com estimativa nativa de incertezas (Deisenroth et al. , 2019; Knieling et al. , 2024).

## 1.2 Problema de Pesquisa

O avanço instrumental e a multiplicação de redes de observação (incluindo ciência cidadã) expandiram fortemente o volume de dados de ocultações. Em muitas campanhas, cada participante registra milhares de imagens, embora a ocultação ocupe apenas uma pequena fração desse conjunto (Cazeneuve et al. , 2023). O desafio central é identificar rapidamente, nesse volume massivo, quais curvas de luz exibem um evento positivo e, portanto, requerem atenção imediata.

É preciso, porém, distinguir dois níveis de exigência sobre essa triagem. Detectar a queda ~~profunda associada ao **corpo principal** do ocultante~~ de fluxo profunda associada à detecção do **corpo principal** é o requisito mínimo: trata-se de um sinal de alta amplitude que qualquer ferramenta de detecção deve recuperar sem falhas. O objetivo de maior valor científico, contudo, é não deixar escapar os **sinais sutis** — assinaturas rasas de anéis, atmosferas tênues ou estruturas secundárias — que costumam passar despercebidos em uma primeira inspeção. O exemplo emblemático ~~é a descoberta dos~~ são os anéis do objeto transnetuniano (50000) Quaoar~~, identificados apenas após~~ . O primeiro anel (Q1R), denso e situado muito além do limite de Roche, foi identificado a partir de uma **revisita** ~~cuidadosa dos dados de ocultação pelo próprio autor da análise (Pereira et al. , 2023)~~a dados de ocultações anteriores, motivada por uma observação nova (Morgado et al. , 2023); o segundo anel (Q2R), mais tênue, é descrito por Pereira et al. (2023). Casos como esse evidenciam que uma ferramenta capaz de chamar a atenção do analista para os trechos potencialmente relevantes de uma curva — mesmo quando o sinal é fraco — pode ser decisiva para que uma descoberta inédita não seja perdida. Esta dissertação demonstra, entre seus resultados, justamente essa capacidade aplicada a uma curva real de Quaoar (Seção 5.9).

A tarefa é complexa~~: ruídos -~~. Ruídos observacionais e variabilidades atmosféricas (nuvens, *seeing* — o borramento da imagem estelar causado pela turbulência atmosférica —, etc.) podem produzir quedas ou picos espúrios que ~~mimetizam~~ imitam uma ocultação (Cazeneuve et al. , 2023; Knieling et al. , 2024)~~;~~. Além disso, as observações frequentemente operam no limite instrumental, ~~exigindo~~ o que exige definições cuidadosas de detecção e critérios estatísticos para não confundir excursões de ruído com sinal; em buscas por TNOs em regiões densas, sinais podem ainda ser artefatos da subtração de estrelas fixas (Fraser et al. , 2024).

A detecção rápida, precisa e escalável tornou-se, assim, um gargalo no fluxo de trabalho científico. Métodos que dispensam ML, ~~por exemplo,~~ como janelas de suavização, podem ser limitados em determinados regimes: a suavização tende a mascarar eventos muito rápidos (com tempos de ingresso e egresso menores que a janela escolhida), e o desempenho em estrelas ~~de baixo brilho aparente para o telescópio em uso~~ com baixa relação sinal-ruído (do inglês, *Signal-to-Noise Ratio*, SNR) depende fortemente da experiência do analista. Nesses casos, o sinal pode ser facilmente confundido com flutuações de ruído. Técnicas de ML ~~emergem~~ surgem como alternativa: redes neurais e modelos de regressão ~~moderna (como Processos Gaussianos~~modernos (como os processos gaussianos) já são usados para filtrar candidatos, reconstruir dados e apoiar levantamentos (Deisenroth et al. , 2019; Fraser et al. , 2024; Carleo et al. , 2019), e ~~se apresentam~~ apresentam-se como caminho natural para os desafios de escalabilidade e precisão na era do grande volume de dados (*big data*) em astronomia (Cazeneuve et al. , 2023).

O objetivo principal desta dissertação é desenvolver e validar uma abordagem automatizada, baseada em técnicas fundamentais de ML, para detectar de forma robusta e eficiente eventos de ocultação estelar em dados fotométricos. Os objetivos específicos são: (a) implementar um **fluxo de trabalho em etapas** (*pipeline*): aquisição dos dados, pré-processamento e extração de **características numéricas** (*features*) a partir de cada curva de luz - por exemplo, amplitude do fluxo, mínimo da série suavizada, distância entre centróides de um agrupamento em dois níveis, etc.; (b) **treinar** e comparar **classificadores** de ML (Regressão Logística, Random Forest, XGBoost e CatBoost), isto é, algoritmos que recebem um vetor de características e respondem "positivo" ou "negativo" (ou uma probabilidade de evento), ajustando seus parâmetros a partir de curvas cuja classificação verdadeira é conhecida; (c) avaliar o desempenho com métricas de classificação binária (acurácia, precisão, sensibilidade, F1-score, área sob a curva ROC) em dados reais e ~~sintéticos~~simulados, permitindo interpretar a capacidade de generalização e a importância de cada variável.

Neste trabalho optou-se por classificadores baseados em árvores de decisão e em vetores de características extraídos de forma explícita a partir das curvas, em vez de redes convolucionais que operam diretamente sobre as séries temporais brutas (como o ODNet). As razões são três: a **interpretabilidade** ~~: —~~ é possível verificar quais grandezas

(~~features~~*features*) mais influenciam a decisão do modelo, o que é valioso para a análise científica ~~; a reprodutibilidade~~, a **reprodutibilidade** com volumes de dados moderados ~~; e a facilidade~~ e a **facilidade** de integração ~~em fluxos~~ a *pipelines* de análise já existentes. A metodologia explora a capacidade desses modelos em identificar padrões típicos de ocultação e distingui-los de ruídos instrumentais, atmosféricos ou intrínsecos, com perspectiva de incorporar também redes neurais em trabalhos futuros.

A *pipeline* desenvolvida atua como ferramenta de organização, priorização e triagem em grandes bases observacionais: (1) indica quais curvas de luz merecem atenção prioritária e (2) estima a probabilidade de evento em curvas ruidosas, visando superar limitações atuais de precisão e velocidade e, no futuro, permitir implementações em tempo real a bordo de telescópios.

A hipótese de pesquisa que norteia o trabalho é a de que é possível obter desempenho superior ou comparável ao de abordagens manuais ou baseadas em redes convolucionais usando ~~um fluxo baseado~~ uma *pipeline* baseada em características clássicas (estatísticas do fluxo, derivadas, testes entre quartis, distância entre centróides do K-means, etc.) e em ~~ensembles~~***ensembles*** de árvores (combinações de muitos modelos de árvore de decisão), aproveitando as vantagens de interpretabilidade, velocidade de processamento, organização dos dados e controle sobre cada etapa.

A dissertação está organizada da seguinte forma. O **Capítulo 2** apresenta a fenomenologia das ocultações estelares e o processamento dos dados astronômicos que originam as curvas de luz. O **Capítulo 3** expõe os fundamentos dos modelos de ML utilizados, em linguagem acessível a leitores com formação em física ou astronomia: classificação **supervisionada** (aprendizado a partir de exemplos com rótulo conhecido), agrupamento (K-means) e os conceitos de validação e generalização. O **Capítulo 4** descreve a metodologia completa da *pipeline*: aquisição e armazenamento dos dados, pré-processamento, extração de características e treinamento e avaliação dos classificadores. O **Capítulo 5** apresenta os resultados dos experimentos e a discussão, retomando os objetivos específicos e as métricas de avaliação. O **Capítulo 6** apresenta as conclusões do trabalho, com contribuições e limitações de forma explícita.

# Capítulo 2

# Ocultações Estelares e Processamento de Dados

## 2.1 O que são Ocultações Estelares?

Uma **ocultação estelar** ocorre quando um objeto celeste (planeta, satélite natural, asteroide ou objeto transnetuniano) passa na frente de uma estrela distante, bloqueando temporariamente parte ou toda a sua luz. Esse evento se manifesta como uma queda característica no fluxo luminoso registrado por um observador na Terra. Como a estrela ocultada está a distâncias muito maiores que a do corpo do Sistema Solar à Terra (a estrela ~~em a~~ anos-luz~~, o corpo em ~~; o corpo a minutos ou horas-luz), os raios luminosos da estrela que chegam ao corpo são essencialmente paralelos ~~: a sombra projetada na~~ entre si. Por isso, no plano perpendicular à direção da estrela — o plano do céu —, a sombra reproduz o perfil do corpo em escala 1:1. Essa correspondência deixa de ser fiel quando o tamanho do corpo se torna comparável à escala de Fresnel ou ao diâmetro aparente da estrela projetado à distância do corpo (Seção 2.3); sobre o solo, a sombra aparece ainda alongada pela inclinação da superfície ~~terrestre preserva o limbo do corpo , com escala unitária~~terrestre em relação aos raios. Essa projeção, combinada à alta resolução temporal das observações, permite determinar tamanho, forma projetada no plano do céu e, em casos favoráveis, atmosfera, anéis ou satélites do corpo ocultante, com precisão comparável à de sondas espaciais (Knieling et al. , 2024; Sicardy et al. , 2011). A Figura 2.1 ilustra esquematicamente o fenômeno para o Centauro (10199) Chariklo e seu sistema de anéis: a curva de luz de ocultação (painel inferior) registra as quedas de fluxo estelar produzidas pela passagem dos anéis e do corpo principal diante da estrela~~—~~, tal como medidas por um observador na superfície da Terra. Chariklo serve aqui apenas como ilustração do fenômeno. Nas demais figuras deste capítulo adota-se como exemplo a ocultação por Umbriel, satélite de Urano, de 21 de setembro de 2020 (Assafin et al. , 2023): além de ser um caso bem documentado, as curvas de luz dessa campanha fazem parte do banco

de dados utilizado nesta dissertação (Capítulo 4).

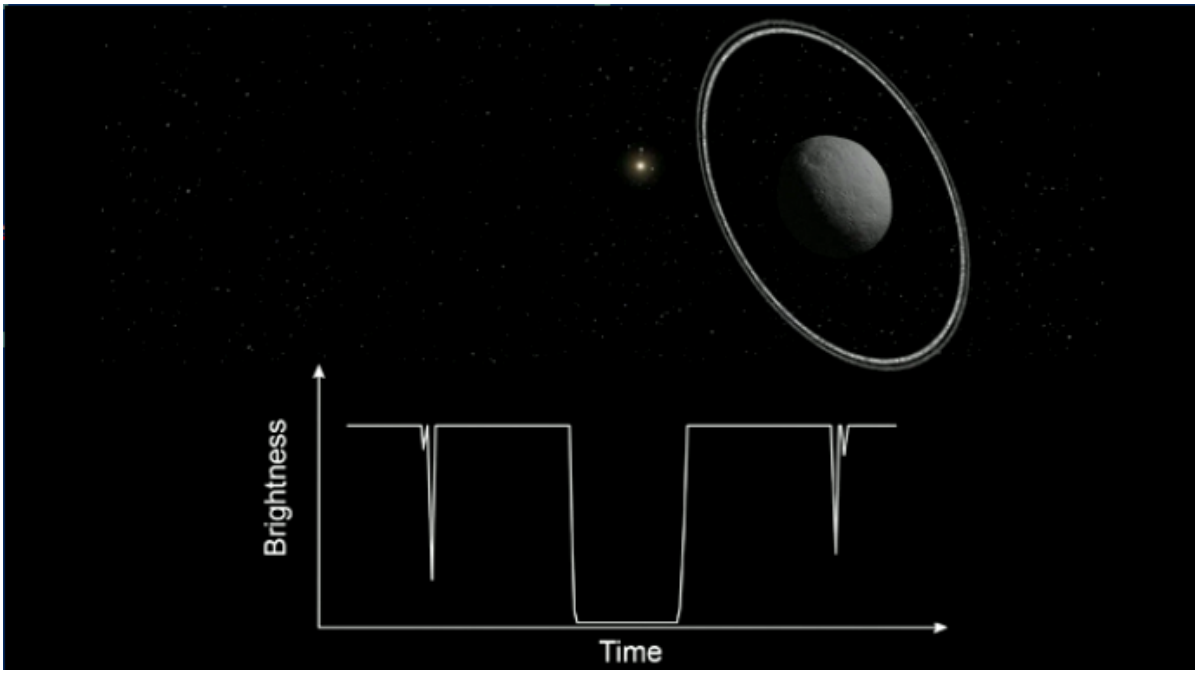
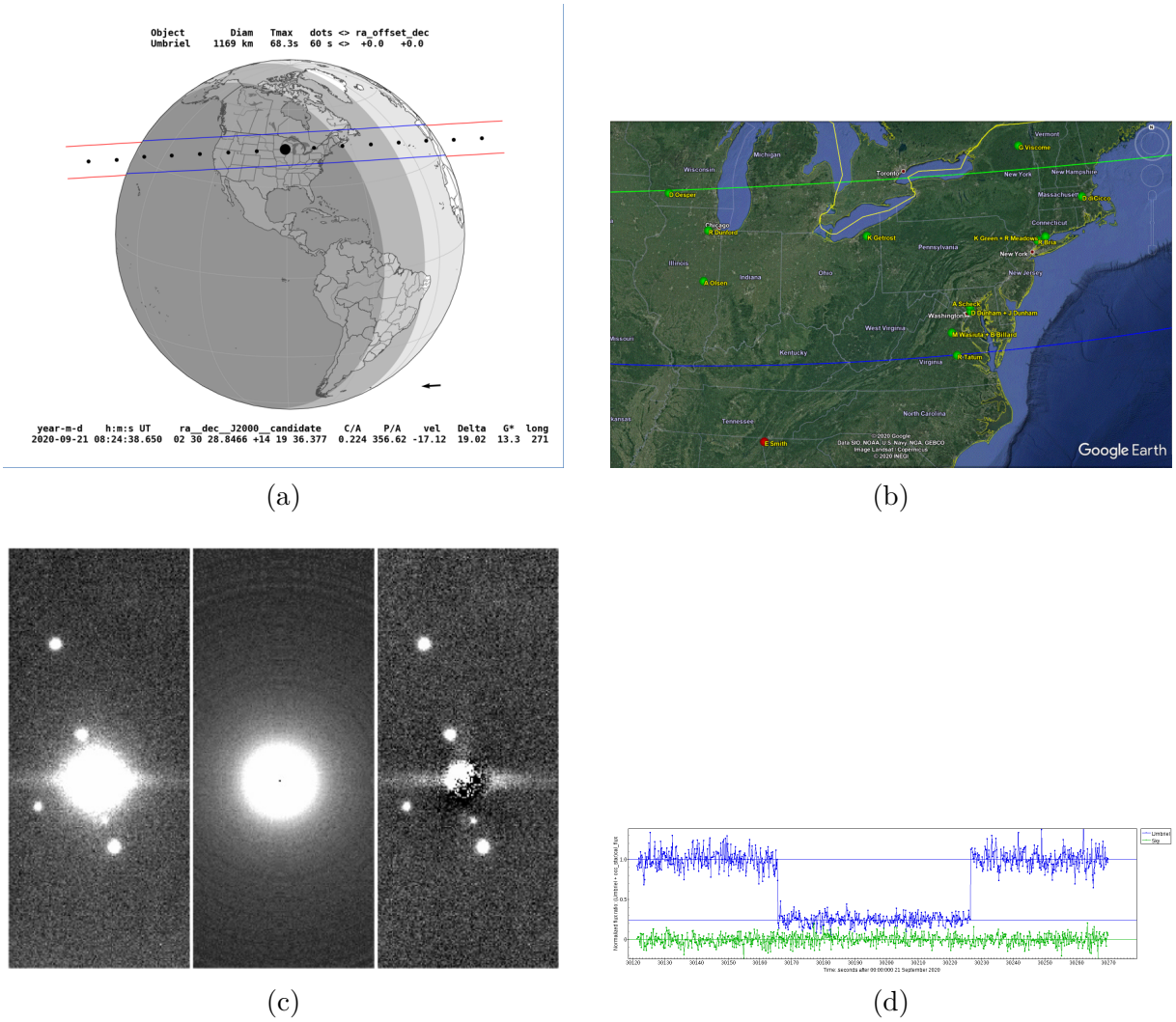



Figura 2.1: Representação artística de uma ocultação estelar pelo Centauro (10199) Chariklo e seu sistema de anéis. A curva de luz esquemática (abaixo) mostra as quedas de fluxo correspondentes à passagem dos anéis e do corpo principal diante da estrela. Crédito: Lucie Maquet, 2014.

A sombra do corpo ocultante projeta-se na superfície terrestre ~~. Cada observador, em sua posição geográfica, registra um~~ e move-se sobre ela com velocidade $V_S$ — de alguns a algumas dezenas de $\mathrm{km\,s^{-1}}$ —, que é a velocidade do corpo *relativa ao observador*, projetada no plano do céu. Ela combina o movimento orbital do corpo, o movimento orbital da Terra (termo dominante para Centauros e TNOs, que se deslocam a poucos $\mathrm{km\,s^{-1}}$) e, em menor grau, a rotação da Terra ($\lesssim 0{,}46\,\mathrm{km\,s^{-1}}$). Na ocultação por Umbriel de 2020, por exemplo, $V_S \approx 17\,\mathrm{km\,s^{-1}}$ (Figura 2.2a). Cada observador dentro do caminho da sombra registra o intervalo de tempo durante o qual a estrela ~~é ocultada~~ - permanece oculta — uma **corda** positiva —; os instantes de imersão e emersão, projetados no plano do céu~~—~~, fornecem dois pontos do limbo do objeto. Observadores fora do caminho registram cordas negativas, que limitam até onde o limbo pode se estender. A duração e o instante central da corda dependem da posição do observador em relação ao trajeto da sombra. Vários observadores em locais diferentes produzem várias cordas; a combinação dessas cordas permite reconstruir a forma e o tamanho do corpo no plano do céu com ~~alta~~ precisão ~~(Gomes-Júnior et al. , 2022). A~~ quilométrica (Assafin et al. , 2023; Ortiz et al. , 2020). No entanto, a previsão de ocultações depende criticamente da precisão das posições estelares e das efemérides do corpo; a melhoria contínua desses ingredientes tem ampliado o número de eventos previstos e observados com sucesso, inserindo a técnica na era do grande volume de dados (*big data*) em astronomia

(Knieling et al. , 2024; Gomes-Júnior et al. , 2022). Graças à precisão astrométrica do catálogo Gaia (Gaia Collaboration et al. , 2016, 2023), o gargalo atual para a previsão de ocultações está na qualidade das efemérides dos pequenos corpos do Sistema Solar. A Figura 2.2 ~~reúne um exemplo completo desse fluxo observacional~~ percorre, para essa ocultação por Umbriel, o fluxo observacional completo — do mapa de previsão da sombra e do posicionamento dos observadores aos quadros CCD e à curva de luz ~~reduzida — para a~~ de ocultação ~~por Umbriel (satélite de Urano) em 2020.~~ reduzida.

A observação é tipicamente feita por telescópios amadores e profissionais, com detectores CCD ou câmeras de vídeo. A análise das imagens serve-se de **fotometria diferencial de abertura**~~: mede-se~~, em que se mede o fluxo da ~~estrela alvo~~ estrela-alvo em relação ~~a~~ ao de estrelas de referência no mesmo campo~~, de modo a cancelar~~. Como a luz de todas atravessa praticamente a mesma coluna atmosférica no mesmo instante, cancelam-se efeitos sistemáticos ~~, tendo em conta a curta duração do evento (~~comuns, como extinção atmosférica, variações de transparência ~~). O resultado da fotometria , ao longo do tempo,~~ etc. Essa fotometria é ~~uma **curva de luz** - a série temporal do fluxo (ou~~ usualmente feita com pacotes dedicados, como o PRAIA (*Platform for Reduction of Astronomical Images Automatically*) (Assafin et al. , 2011), que também implementa a **coronagrafia digital**: a subtração, imagem a imagem, de um modelo do halo de um objeto muito brilhante próximo ao alvo — como Urano, no caso de Umbriel (Assafin , 2023). A **curva de luz de ocultação** é a razão ~~de fluxos)~~ entre o fluxo medido na abertura da estrela-alvo e o fluxo das estrelas de referência da mesma imagem, em função do tempo. Em geral, o corpo ocultante não é resolvido da estrela: a abertura contém a soma estrela + corpo, e a luz do corpo continua presente durante a ocultação (Seção 2.3). Em uma detecção **positiva**, a curva exibe um trecho de fluxo aproximadamente constante (estrela ainda não ocultada), uma queda relativamente abrupta (~~ingresso da estrela atrás do corpo~~a imersão: o corpo passa diante da estrela e o observador entra na sombra), um patamar ~~baixo (ocultação total ou parcial)~~ de baixo fluxo e o retorno ao ~~fluxo total da estrela (egresso~~nível inicial (a emersão). Em uma detecção **negativa**, não há queda significativa; a curva permanece estável (ou com flutuações de ruído), e essa informação é igualmente útil~~para dar um limite superior ao tamanho da sombra e refinar modelos. A análise detalhada da curva (ajuste de modelos que incluem~~: uma corda negativa delimita até onde o limbo do corpo pode se estender naquela direção. O ajuste de uma curva sintética — que considera o diâmetro aparente da estrela ~~,~~ projetado à distância do corpo, a difração de Fresnel e o tempo de exposição ~~)~~ usado na observação — permite obter os instantes de imersão e emersão com ~~alta~~ precisão ~~e~~da ordem de frações de segundo e, projetando esses instantes no plano do céu, ~~a partir deles,~~ as dimensões e a forma ~~do corpo (Gomes-Júnior et al. , 2022)~~bidimensional do objeto (Gomes-Júnior et al. , 2022; Ortiz et al. , 2020).




(a) (b)

(c) (d)

Figura 2.2: Exemplo completo do fluxo observacional: ocultação estelar por Umbriel (satélite de Urano), observada em 21 de setembro de 2020 (Assafin et al. , 2023). (a) Mapa de previsão da sombra sobre a superfície terrestre. (b) Detalhe do posicionamento dos observadores na costa leste dos EUA (Google Earth). (c) ~~Exemplo de quadros~~ Quadros CCD ~~obtidos por~~ de uma das estações, processados pelo PRAIA: ~~são imagens como estas que alimentam os pacotes de fotometria~~ imagem original (~~e.g. PRAIA~~esquerda)~~para a construção das curvas de luz. Neste caso~~, ~~o brilho intenso~~ modelo do halo de Urano ~~exigiu~~ (centro) e imagem após a ~~aplicação de~~ subtração por coronagrafia digital ~~para viabilizar~~ (direita), que viabiliza a fotometria ~~precisa~~ de Umbriel e da estrela. (d) Curva de luz de ocultação resultante~~da redução via PRAIA~~: razão entre o fluxo ~~normalizado~~ da estrela somado ao de Umbriel e o fluxo de calibração (azul), e ~~ruído de~~ fundo de céu (verde)~~, com a queda de ocultação claramente visível~~.

## 2.2 Da observação às curvas de luz: coleta e armazenamento

As curvas de luz utilizadas em campanhas de ocultações provêm de redes de telescópios (profissionais e amadores) que registram o fluxo da estrela alvo em cadência rápida (exposições de fração de segundo a poucos segundos, com o mínimo possível de intervalo entre imagens sucessivas — o chamado tempo morto) para maximizar a resolução temporal e, portanto, espacial ao longo da corda. O volume de dados gerado é grande: cada observação pode produzir milhares de quadros e da ordem de gigabytes por evento (Cazeneuve et al. , 2023). A redução fotométrica (extração do fluxo em cada quadro) é feita com pacotes dedicados, como o PRAIA (*Package for the Reduction of Astronomical Images Automatically*)o PRAIA, que automatiza a fotometria de abertura e a construção da curva de luz a partir das imagens (Assafin , 2023). O SORA (*Stellar Occultation Reduction and Analysis* ) (Gomes-Júnior et al. , 2022) (SORA, Gomes-Júnior et al. , 2022) é utilizado na etapa de análise e redução pós-fotometria para ajuste de modelos às curvas e extração de parâmetros físicos; sua saída pode alimentar bancos de dados e catálogos (como os do Grupo do Rio). Após a redução, as curvas são armazenadas em bancos de dados ou catálogos (por exemplo, VizieR, bancos de grupos de pesquisa), em formato tabular: cada registro associa uma observação a uma série de pontos (tempo, fluxo), além de metadados (objeto ocultante, data, observador, classificação positiva/negativa quando disponível, etc.).

Nesta dissertação, a *pipeline* de detecção (Capítulo 4) parte de curvas **já reduzidas** (pós-PRAIA/pós-SORA ou equivalentes), obtidas de fontes externas (Grupo do Rio, VizieR) e inseridas em um banco de dados relacional. Ou seja, o fluxo aqui descrito começa após a etapa de redução fotométrica e análise; a estrutura do banco, o fluxo de aquisição e a normalização do fluxo são detalhados no Capítulo 4. Aqui basta destacar que as curvas de luz são o insumo central para a etapa de classificação por ML: cada curva será convertida em um vetor de características e classificada como positiva ou negativa pelos modelos descritos no Capítulo 3.

## 2.3 Características Caracterização das curvas de luz e detecção positiva/negativade ocultação

Do ponto de vista observacional, uma curva de luz de ocultação é uma série temporal de fluxo(normalmente normalizado em relação ao , normalizada de modo que o nível fora da ocultação ).—(estrela + corpo) valha 1. Sua **morfologia** depende de vários fatores físicos — razão de brilho entre o corpo e a estrela, diâmetro aparente da estrela, difração de Fresnel, velocidade da sombra e presença de atmosfera, anéis ou satélites — e de

fatores instrumentais e ~~instrumentais~~atmosféricos — tempo de exposição e tempo morto, cintilação, nuvens e qualidade da fotometria.

**Detecção positiva e negativa.** Em uma detecção positiva, a curva exibe uma queda de fluxo durante o intervalo de tempo em que o trânsito ocorre. ~~A profundidade~~ Para um corpo sem atmosfera, muito maior que a estrela projetada e que a escala de Fresnel, qualquer corda que cruze o disco bloqueia toda a luz da estrela, seja ela central ou não: a posição da corda determina a *duração* da queda, e não a sua profundidade. A profundidade depende da razão entre os brilhos do corpo e da ~~queda está relacionada à fração do disco estelar ocultada e à geometria da corda; em ocultações centrais ou próximas do centro, a queda pode ser total (fluxo~~ estrela, porque a luz do próprio corpo continua a ser registrada na abertura. O patamar normalizado vale

$$F_{\rm oc} = \frac{F_{\rm c}}{F_\star + F_{\rm c}} = \frac{1}{1 + 10^{\,0{,}4\,(m_{\rm c} - m_\star)}},$$

em que $F$ e $m$ são o fluxo e a magnitude do corpo (c) e da estrela ($\star$). Na ocultação por Umbriel (Figura 2.2d), o patamar $\approx 0{,}24$ corresponde a $m_{\rm c} - m_\star \approx 1{,}2$ mag; para TNOs, em geral muito mais fracos que a estrela ocultada, o patamar fica próximo de zero~~no patamar)~~. Em uma detecção negativa, o observador está fora da sombra; a curva não apresenta queda significativa, apenas flutuações típicas de ruído fotométrico, atmosférico e instrumental. Tanto positivas quanto negativas são úteis: as primeiras fornecem cordas para o ajuste de forma e tamanho; as ~~segundas~~ últimas ajudam a delimitar a borda da sombra e a descartar falsos positivos (Gomes-Júnior et al. , 2022).

**Fatores que afetam a forma da curva.** O perfil da queda (ingresso e egresso) é influenciado pelo **diâmetro aparente da estrela** à distância do corpo ocultante: estrelas cujo diâmetro angular não é desprezível nessa escala produzem, em combinação com o ciclo de integração do detector, uma transição suave entre ~~o nível desocultado e o ocultado~~ os níveis de fluxo desocultado e ocultado da estrela. A **difração de Fresnel** nas bordas do corpo suaviza adicionalmente as regiões de ingresso e egresso; a escala típica é da ordem de ~~$f = \sqrt{D\lambda/2}$~~$L_{\rm F} = \sqrt{D\lambda/2}$, onde $D$ é a distância entre o observador e o corpo ocultante (ou, em convenções equivalentes, a distância estrela–corpo) e $\lambda$ o comprimento de onda (Roques et al. , 1987). Essa suavização faz com que os instantes de "meia-luz" não coincidam exatamente com o contato geométrico do limbo; exposições longas ou baixa resolução temporal podem mascarar os detalhes da difração e reduzir a precisão da extração dos tempos de imersão e emersão (Gomes-Júnior et al. , 2022). O **tempo de exposição** converte-se em resolução espacial ao longo da corda por meio da relação $r_t = V_S \times t_{\rm exp}$, onde $r_t$ é a resolução espacial efetiva, $V_S$ é a velocidade da sombra projetada na superfície terrestre e $t_{\rm exp}$ é o tempo de exposição~~;~~. Desse modo, exposições longas ~~podem suavizar ou mascarar~~ suavizam ou mascaram os detalhes da difração. A rigor, a amostragem ao longo da corda é determinada pelo **ciclo completo** entre quadros

sucessivos — o tempo de exposição somado ao tempo morto~~de leitura (*readout*)~~: enquanto $t_{\mathrm{exp}}$ borra cada ponto ao longo de um intervalo $V_S\, t_{\mathrm{exp}}$, o tempo morto abre lacunas entre medidas sucessivas, de modo que um tempo morto elevado degrada a resolução efetiva mesmo com exposições curtas — daí a busca por minimizá-lo na aquisição (Seção 2.2). A Figura 2.3 mostra como essas componentes — difração de Fresnel, tamanho angular da estrela e ciclo de integração do detector — se combinam no modelo de curva de luz utilizado pelo SORA (Gomes-Júnior et al. , 2022). Em corpos com atmosfera, refração e absorção modificam o perfil da curva e permitem inferir composição e estrutura da atmosfera~~(~~, como perfis de temperatura e densidade em função da altitude ~~)~~(Sicardy et al. , 2016); anéis ou satélites podem produzir quedas adicionais ou assimetrias (Braga-Ribas et al. , 2014). Para a detecção automatizada por ML, o que importa é que curvas com ocultação exibem uma queda sustentada e característica no fluxo, ao passo que curvas negativas ou com apenas ruído não exibem esse padrão; flutuações atmosféricas ou instrumentais podem, no entanto, imitar quedas pontuais, o que torna a tarefa de classificação não trivial e motiva o uso de modelos treinados e de características bem escolhidas (Capítulos 3 e 4).

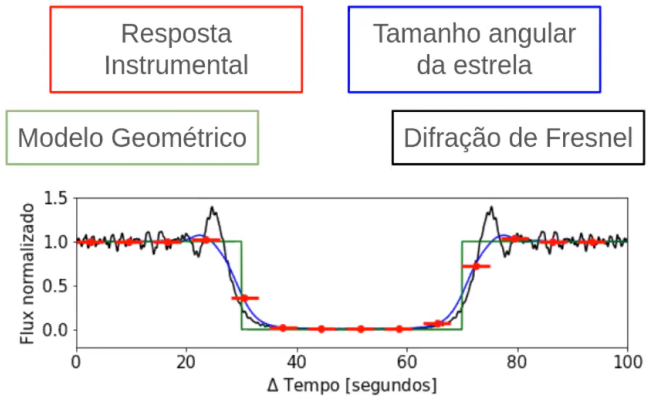



Figura 2.3: Componentes do modelo de curva de luz sintética utilizado pelo SORA (Gomes-Júnior et al. , 2022). Verde: modelo geométrico, o ponto de partida — corpo opaco de borda nítida ocultando uma estrela pontual, sem difração e com amostragem instantânea. Preto: o mesmo modelo incluindo a difração de Fresnel~~(azul), tamanho~~ . Azul: incluindo também o diâmetro angular da estrela. Traços vermelhos: resposta instrumental, ~~ciclo de integração do detector~~ e isto é, ~~resultante da combinação dessas componentes,~~ o ~~modelo geométrico~~ valor que cada exposição registraria ao integrar a curva azul durante o tempo de exposição (~~verde~~a largura de cada traço). ~~O ajuste simultâneo aos~~ A curva sintética ajustada a dados ~~observados~~ reais combina todos esses efeitos (~~pontos vermelhos) permite determinar com precisão os instantes de imersão e emersão, necessários para~~ Figura 2.4a~~reconstrução da forma do corpo~~).

**Exemplo numérico: escala de Fresnel.** Para um objeto transnetuniano (TNO) a uma distância típica de $D = 40\,\mathrm{UA} \approx 6{,}0 \times 10^{12}\,\mathrm{m}$, observado no visível ($\lambda = 500\,\mathrm{nm}$), a

escala de Fresnel é

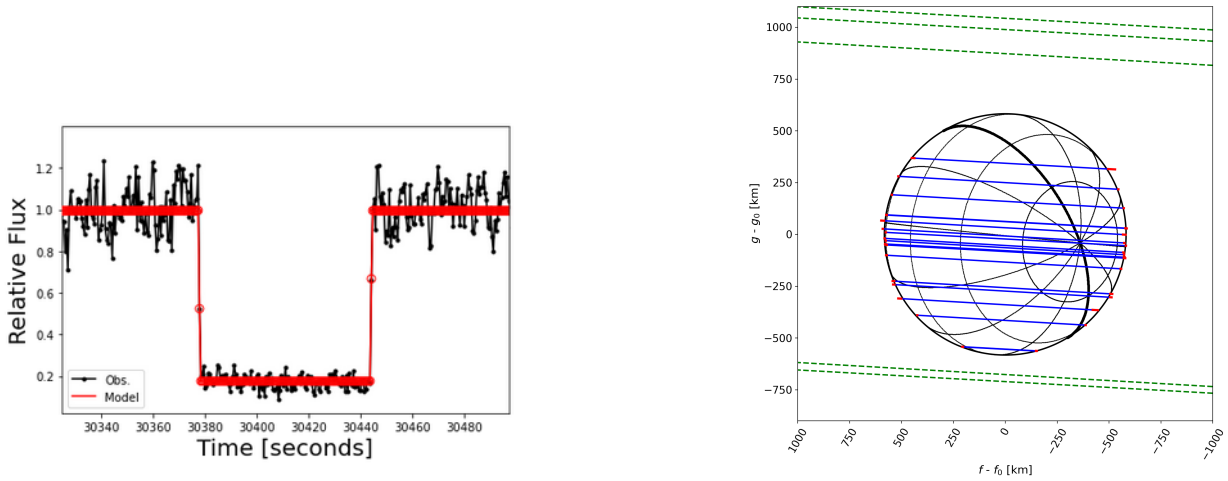
$$\not{f} L_{\mathrm{F}} = \sqrt{\frac{D\lambda}{2}} \approx \sqrt{\frac{6{,}0 \times 10^{12} \times 5{,}0 \times 10^{-7}}{2}} \approx 1{,}2 \text{ km}.$$

Essa escala define a resolução espacial efetiva na direção perpendicular à corda: detalhes menores que ~~$f$~~ $L_{\mathrm{F}}$ são suavizados pela difração. ~~Ainda assim, trata-se~~ Trata-se de uma resolução da ordem ~~de quilômetros~~ do quilômetro na observação de objetos a dezenas de unidades astronômicas de distância -~~–~~superior à de qualquer telescópio óptico terrestre operando por imageamento direto. É essa capacidade de resolução que torna as ocultações estelares uma ferramenta única para a caracterização de pequenos corpos do Sistema Solar.

**Relação sinal-ruído e qualidade.** A confiabilidade da detecção e da extração de parâmetros físicos depende da relação sinal-ruído (~~S/N~~SNR) da curva. Em astronomia, muitas medições são feitas no limite da detectabilidade instrumental, onde o ruído pode produzir excursões que imitam fontes tênues; definições operacionais claras de detecção e critérios estatísticos são essenciais para interpretar os dados de forma confiável (Wall & Jenkins , 2003). ~~S/N~~ SNR alto favorece a identificação clara da queda e a precisão dos instantes de imersão e emersão; curvas com ~~S/N~~ SNR baixo são mais difíceis de classificar e mais suscetíveis a falsos positivos e negativos. Na *pipeline* desta dissertação, as características extraídas de cada curva (estatísticas de fluxo, derivadas, métricas de forma, etc.) são escolhidas de forma a capturar padrões discriminativos entre curvas com e sem ocultação, mesmo na presença de ruído; o conjunto preciso de *features* e o protocolo de treinamento são apresentados no Capítulo 4. A Figura 2.4 ilustra, para a ocultação por Umbriel (2020), o ajuste de ~~um modelo a uma corda observada~~ uma curva sintética a uma das curvas de luz de ocultação observadas nessa campanha — com os instantes de imersão e emersão ~~marcados~~ indicados — e o perfil do limbo reconstruído a partir do conjunto de cordas (Assafin et al. , 2023).

## 2.4 Considerações finais do capítulo

Este capítulo situou as ocultações estelares como fenômeno astronômico e as curvas de luz de ocultação como produto observacional central. ~~Destacamos o papel da geometria da sombra~~ Vimos que a sombra do corpo, projetada em escala 1:1 no plano do céu, ~~das cordas e da precisão astrométrica na previsão e no aproveitamento dos eventos; a~~ é amostrada pelas cordas dos observadores, e que é a combinação de cordas positivas e negativas que permite reconstruir forma e tamanho. Vimos também que a astrometria estelar do Gaia, ao tornar mais precisa a previsão de onde a sombra passará, aumentou o número de eventos observados com sucesso. A coleta e o armazenamento dos dados foram descritos em linhas gerais~~;~~ , e foram resumidas as características observacionais das curvas. ~~O volume crescente de dados produzidos por redes de telescópios e catálogos como o Gaia~~




(a) (b)

Figura 2.4: (a) Curva de luz de ocultação observada (preto) e ~~modelo ajustado~~ curva sintética ajustada (vermelho) para uma das cordas da ocultação por Umbriel em 2020; os instantes de imersão e emersão ~~, marcados por circunferências, definem a geometria~~ encontram-se muito próximos dos pontos circulados nas bordas da ~~corda~~ queda (Assafin et al. , 2023). (b) Perfil do limbo de Umbriel ~~;~~ no plano do céu: cada linha azul ~~(incerteza vermelho) corresponde a~~ é uma corda positiva projetada~~no plano do céu~~ , com as incertezas dos extremos em vermelho (Assafin et al. , 2023).

~~torna~~ A combinação de previsões mais precisas, da popularização de câmeras rápidas e de baixo custo e das redes de observadores multiplicou o número de curvas de luz de ocultação a analisar e tornou penosa a triagem manual de todas ~~as curvas de luz; essa~~ elas. Essa escala motiva o desenvolvimento de métodos automatizados de detecção baseados em aprendizado de máquina, cujos fundamentos são apresentados no Capítulo 3 e cuja implementação é detalhada no Capítulo 4. ~~Notadamente, nos interessam também~~ Interessa-nos, em particular, que sejam apontados eventos tênues ~~e~~ que passaram despercebidos num primeiro tratamento ~~de curvas de luz - como ocorreu , por exemplo, na descoberta dos~~ das curvas — como ocorreu com os anéis de Quaoar~~, descrito em (Pereira et al. , 2023)~~: o primeiro (Q1R) foi revelado pela revisita de dados antigos motivada por uma observação nova (Morgado et al. , 2023), e o segundo (Q2R) foi descrito por Pereira et al. (2023). Um alerta confiável nesses casos é suporte para campanhas observacionais e descobertas importantes.

# Capítulo 3

# Modelos e algoritmos para ~~Machine Learning~~*Machine Learning*

O Capítulo 2 mostrou que as campanhas de ocultação estelar podem produzir diversas curvas de luz, sendo número suficiente para que haja dificuldade na análise completa e minuciosa. Cada curva pode ser resumida por um conjunto de características como profundidade da queda, estatísticas de ruído, entre outros. A questão é: *a partir dessa descrição numérica de uma curva, é possível decidir automaticamente se ela contém ou não uma ocultação?* Trata-se de um problema de **classificação binária**, e os métodos de *Machine Learning* são as ferramentas naturais para resolvê-lo. Com isso, os objetivos deste capítulo são: definir e situar os modelos de ~~Machine Learning~~ *Machine Learning* como ferramentas de descobertas e classificações no contexto científico; formalizar o problema de classificação (espaço de atributos, rótulos, regiões de decisão); apresentar os principais modelos utilizados na *pipeline*; e, por fim, discutir algoritmos de otimização, engenharia de características, validação de modelos e os riscos de falsos resultados.

## 3.1 Conceitos fundamentais de ~~Machine Learning~~*Machine Learning*

### 3.1.1 Definição e papel do aprendizado de máquina

O *Machine Learning* (ML), ou aprendizado de máquina, é uma área na qual os sistemas ~~"aprendem"~~“aprendem” a partir de dados, sem serem explicitamente programados para cada tarefa (Geron , 2019). Em vez de regras fixas codificadas à mão, um algoritmo de ML é alimentado com informações e **ajusta seus próprios parâmetros** (por exemplo, coeficientes de uma combinação linear) para capturar padrões ou relações nos dados. Esse processo de ajuste é o que se chama de **treinamento**; ao final, o modelo está ~~"treinado"~~“treinado” e pode ser aplicado a novas informações. O objetivo é permitir que

o sistema melhore seu desempenho à medida que mais dados ou mais iterações estejam disponíveis, especialmente em problemas sem solução analítica fechada ou com grandes volumes de dados.

No contexto científico, o ML costuma ser visto como ~~"~~uma "caixa-preta~~"~~" que produz previsões sem explicação. Essa visão reflete usos industriais em que a precisão preditiva prevalece sobre a interpretabilidade. Em ciência, porém, o ML pode ir além: atuando como ferramenta de descoberta, revelando padrões que podem ser incorporados a uma teoria formal de causa e efeito. O papel do ML não é substituir a teoria, mas auxiliar na sua formulação, identificando os padrões que a teoria buscará explicar. Segundo López (2020), por exemplo, o uso do ML na ciência pode ser agrupado em algumas funções típicas:

(i) **Sugerir a existência de algo novo:** ML pode encontrar padrões não previstos, indicando a existência de relações ainda não descobertas. Em matemática, algoritmos já auxiliaram a conjecturar teoremas que depois foram demonstrados formalmente (Gryak et al. , 2018).

(ii) **Descobrir o grau de importância das variáveis:** técnicas como a *Mean Decrease Accuracy* (MDA, a queda média da acurácia) permitem avaliar o impacto de cada variável (**~~feature~~*feature***, ou característica numérica de entrada) ~~na~~ sobre a saída do modelo de ML para exemplos novos — aqui, "predição" é a resposta do classificador, e não a previsão de uma ocultação a partir de efemérides. A ideia é treinar o modelo, medir o desempenho (por exemplo, via acurácia), embaralhar uma variável por vez e verificar se o desempenho cai; se cair, a variável é importante (Liu , 2004). Assim, o ML identifica quais variáveis devem compor explicações teóricas, mesmo sem revelar o mecanismo causal.

(iii) **Explorar relações de causa e efeito:** um modelo pode ser ajustado com dados nos quais um determinado efeito está ausente. Depois, quando esse efeito passa a atuar, o modelo continua produzindo previsões baseadas no cenário anterior. Se as observações reais passarem a divergir das previsões, essa diferença pode ser um indício de que o efeito alterou o sistema, sugerindo possíveis relações causais.

(iv) **Redução e simplificação:** em dados multidimensionais, técnicas de redução de dimensionalidade por exemplo, a Análise de Componentes Principais (PCA) projetam os dados em um espaço de menor dimensão, tornando mais evidentes padrões, agrupamentos e relações entre observações, o que favorece a interpretação dos resultados.

(v) **Encontrar padrões desconhecidos:** o ML pode atuar na detecção de eventos raros ou de anomalias em grandes volumes de dados -–por exemplo, na triagem automática de ~~ocultações estelares em levantamentos astronômicos - indicando~~ curvas

de luz de ocultação obtidas em campanhas dedicadas e por redes de observadores, como as reunidas no catálogo `B/occ/asteroid` do VizieR. A maioria dos levantamentos de imageamento de grande campo não tem tempos de exposição e cadência compatíveis com a duração de uma ocultação. O ML indica onde vale a pena investigar, mesmo sem explicar o mecanismo.

Neste trabalho, o ML atua sobretudo na função de **encontrar padrões desconhecidos** e de **descobrir o grau de importância das variáveis** (importância das *features* nos ~~ensembles~~ *ensembles* de árvores), em apoio à decisão do pesquisador sobre quais curvas merecem análise detalhada.

### 3.1.2 Aprendizado supervisionado versus não supervisionado

A taxonomia usual divide o ML em dois cenários, conforme a natureza dos dados:

- **Supervisionado:** cada exemplo possui um rótulo (classe ou valor numérico). Tarefas típicas: classificação (rótulo discreto) e regressão (rótulo contínuo). Exemplos: filtro de ~~spam~~ *spam*, previsão de preços, etc.
- **Não supervisionado:** os dados não possuem rótulos. Tarefas típicas: agrupamento (~~clustering~~ *clustering*), detecção de anomalias, redução de dimensionalidade. Exemplos: segmentação de clientes, identificação de *outliers* (anomalias), etc.

Neste trabalho, o foco é o **paradigma supervisionado**, em particular a **classificação binária**. No aprendizado supervisionado, o objetivo é encontrar uma função $f$ que mapeie cada entrada $\vec{x}$ (por exemplo, o vetor de características de uma curva de luz) a um rótulo $y$ (por exemplo, ~~"~~"ocultação presente~~"ou "ausente"~~" ou "ausente"). Os rótulos usados no treinamento vêm da classificação já conhecida dos dados (a chamada **verdade de campo**, ou *ground truth* - por exemplo, a análise prévia de um especialista ou o catálogo). O objetivo é que $f$ **generalize** bem, ou seja, que erre pouco em dados novos que não foram usados no ajuste. A detecção de ocultações estelares é formulada exatamente assim: a curva de luz contém um evento de ocultação (classe positiva, $y = 1$) ou não (classe negativa, $y = 0$). Os classificadores (Regressão Logística, Random Forest, XGBoost, CatBoost) recebem vetores de características extraídos das curvas e produzem uma classe ou uma probabilidade de classe, permitindo triagem e priorização automáticas no fluxo da *pipeline* (Capítulo 4). Neste trabalho, os classificadores foram usados por meio de bibliotecas Python de código aberto, cujas documentações pedem a citação dos trabalhos originais: *scikit-learn* (Pedregosa et al. , 2011), para a Regressão Logística e o Random Forest (Breiman , 2001); XGBoost (Chen & Guestrin , 2016); e CatBoost (Prokhorenkova et al. , 2018).

## 3.2 O problema da classificação supervisionada

Em notação matemática, uma amostra rotulada é representada ~~pela tupla~~ pelo par ordenado (ou **tupla**, isto é, uma sequência ordenada e finita de elementos) $Z = (X, Y)$. Aqui $X = \{\vec{x}_1, \vec{x}_2, \ldots, \vec{x}_n\}$ é o conjunto de **vetores de características**: cada $\vec{x}_i \in \mathbb{R}^d$ é um vetor com $d$ números que resumem, no caso deste trabalho, uma curva de luz ~~(amplitude,~~ de ocultação (por exemplo, a amplitude e o mínimo do fluxo suavizado~~, distância entre centróides, etc.~~). O número $d$ é a ~~"~~"dimensionalidade" do problema (neste ~~caso, 14 ou~~ trabalho, entre 11 e 28 ~~features~~*features*, conforme o experimento; Capítulo 5). O conjunto $Y = \{y_1, y_2, \ldots, y_n\}$ contém os **rótulos**: a classificação verdadeira de cada exemplo, com $y_i \in \{0, 1\}$ no caso binário (0 = sem ocultação, 1 = com ocultação).

Um **classificador** é uma função $f\colon \mathbb{R}^d \longrightarrow \{0, 1\}$ (ou, em geral, $\{1, \ldots, c\}$ para $c$ classes) que, dado um vetor $\vec{x}$, prediz um rótulo $y$. Em muitos modelos, $f$ produz primeiro uma **probabilidade** de a classe ser positiva; a decisão final é então ~~"positivo"~~ "positivo" se essa probabilidade for maior ou igual a 50 %, e ~~"negativo"~~ "negativo" caso contrário. O uso do valor 0,5 é intuitivo, mas ~~cabe frisar que~~ é arbitrário e depende~~rá~~do objetivo do modelo~~que está sendo treinado (esse limite decisório~~. Esse valor de corte é chamado de ~~'threshold' nas pipelines de ML, ou $\tau$,~~ **limiar** de decisão (em inglês, *threshold*) e será ~~melhor explorado mais adiante)~~denotado por $\tau$; seu ajuste é discutido na Seção 5.8.

A função $f$ divide o espaço $\mathbb{R}^d$ em **regiões de decisão**: $R_k = \{\vec{x} \in \mathbb{R}^d \mid f(\vec{x}) = k\}$. Cada região corresponde a uma classe. O objetivo do treinamento é estimar $f$ a partir dos dados $(X, Y)$ de modo que $f$ generalize bem para novos exemplos, não vistos durante o treinamento.

Antes de apresentar os modelos de classificação, convém fixar a ideia de treinamento como minimização de uma função de custo. O exemplo mais natural de ser explorado é a **regressão linear**. O modelo $y_i = \beta_0 + \beta_1 x_i + \epsilon_i$ descreve o comportamento médio de uma variável resposta $y$ dado $x$, com erro $\epsilon_i$ (Figura 3.1).

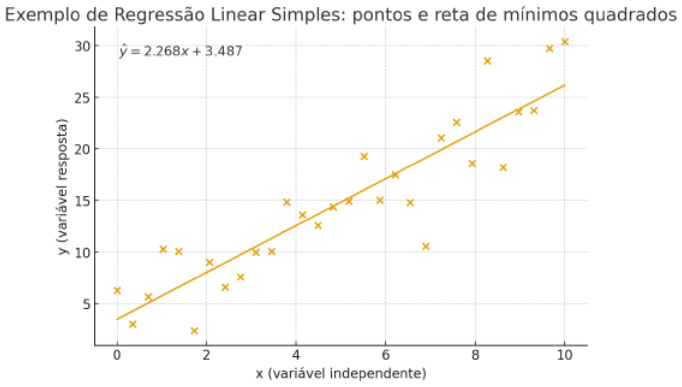



Figura 3.1: Exemplo ilustrativo de regressão linear simples~~(~~, com dados genéricos, não referentes a curvas de luz de ocultação~~) (Levada , 2026)~~. Fonte: Levada (2026).

Os parâmetros são estimados pelo método dos **mínimos quadrados**, que constitui o primeiro exemplo de algoritmo de otimização nesta dissertação. No caso mais simples ($\mathbb{R}^2$), o modelo é $y = \alpha + \beta x$ e a função de custo é

$$J(\alpha, \beta) = \sum_{i=1}^{n} (y_i - \alpha - \beta x_i)^2,$$

minimizada pela condição $\partial J/\partial\alpha = 0$, $\partial J/\partial\beta = 0$. A generalização para múltiplas variáveis ($d$ dimensões) segue naturalmente em notação vetorial. O modelo prediz $\hat{\vec{y}} = X\vec{\theta}$, com $X$ a matriz de dados (cada linha é um exemplo, cada coluna uma variável) e $\vec{\theta}$ o vetor de parâmetros. A função de custo torna-se

$$J(\vec{\theta}) = (\vec{y} - X\vec{\theta})^\top (\vec{y} - X\vec{\theta}) = \sum_{i=1}^{n} \left(y_i - \vec{\theta}^\top \vec{x}_i\right)^2.$$

A condição de mínimo, $\nabla_{\vec{\theta}} J = \vec{0}$, fornece a **equação normal**:

$$X^\top (X\vec{\theta} - \vec{y}) = \vec{0} \quad \Longrightarrow \quad \vec{\theta}^* = (X^\top X)^{-1} X^\top \vec{y}.$$

Após estimar $\vec{\theta}^*$, o modelo pode predizer $\vec{y}$ para novos dados de entrada (Geron , 2019; Levada , 2026).

Apesar dos problemas de regressão servirem de excelente passo inicial para se compreender as métricas e interpretações básicas dos modelos de ML, para problemas de classificação, não usamos regressão linear diretamente - a solução completa desses problemas ~~podem ser acompanhadas no capítulo~~ pode ser acompanhada no Capítulo 2 de ~~(Levada , 2026)~~Levada (2026). A **Regressão Logística** mantém a combinação linear, mas mapeia a saída ao intervalo $(0, 1)$ por uma ~~função sigmoide e~~ **função sigmoide**, $\sigma(z) = 1/(1 + e^{-z})$ — uma curva em forma de "S" que leva qualquer número real a um valor entre 0 e 1 (Figura 3.5) — e utiliza o custo de entropia cruzada, para o qual **não existe solução fechada** (o treinamento é feito por algoritmos iterativos - Seção 3.5).

## 3.3 Modelos de classificação utilizados na ~~pipeline~~*pipeline*

### 3.3.1 Regressão Logística

A Regressão Logística é um classificador probabilístico. Similarmente à Regressão Linear, este modelo calcula uma soma ponderada das características de entrada, mas, em vez de gerar o resultado diretamente, gera a *logística* (função sigmoide) desse resultado (Geron , 2019). Em outras palavras, ele transforma combinações lineares de características em probabilidades válidas.

Suponha um problema de classificação no qual se deseja identificar uma ocultação em

uma curva de luz a partir de uma única informação: o **tamanho da queda máxima do fluxo**. Curvas com maiores quedas possuem maior probabilidade de conter a ocultação. A ~~figura a seguir~~ Figura 3.2 ilustra a aplicação da regressão linear ~~em dados bem comportadosa~~ oito exemplos ilustrativos — quatro com grandes quedas de fluxo e quatro com quedas pequenas: nesse caso simples, o limiar 0,5 sobre a reta separa perfeitamente as duas classes, embora a reta produza valores fora do intervalo $[0, 1]$.

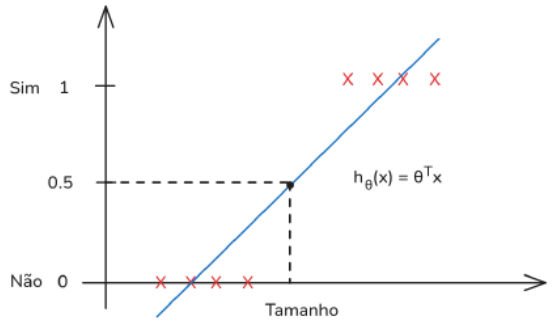



Figura 3.2: ~~Aplicação da regressão linear ao problema de classificação~~ Classificação binária ~~: a reta produz saídas fora~~ (sim $= 1$, não $= 0$) em função do ~~intervalo [0, 1] (Levada , 2026) . Nosso conjunto~~ tamanho da queda de ~~dados mostra quatro amostras de curvas de luz com grandes amplitudes e outras quatro com baixas amplitudes~~fluxo (marcadores). ~~Nesse exemplo simples, A~~ linha tracejada indica o ~~valor~~ limiar de $\theta$decisão; a linha contínua, ~~5 para~~ o ~~recorte~~ resultado da ~~reta divide perfeitamente as duas classificações~~regressão linear aplicada aos dados (ver discussão no texto). Fonte: adaptado de Levada (2026).

Nesse caso, teríamos infinitos valores sob uma reta para a variável de saída. Notadamente, a solução mais intuitiva seria considerar um limiar simples na metade da reta. No entanto, suponha que, no mesmo conjunto de dados, tivéssemos um elemento *outlier*, uma anomalia. A Figura 3.3 ilustra a situação: a curva discrepante tem amplitude muito diferente das demais (por exemplo, por falta de normalização), a reta se desloca e o limiar que antes funcionava deixa de ser útil, pois agora a metade da reta passa a considerar um ponto positivo como negativo ao mesmo tempo que ~~varios~~ vários pontos positivos ficam ~~proximos~~ próximos do limiar de decisão (e apenas o ~~outlier~~ *outlier* se encontra confortavelmente longe das curvas consideradas negativas), gerando dúvida para o analista que estuda os dados.

O que podemos ver é que a reta de regressão pode sofrer alterações significativas com a inclusão de novos pontos, evidenciando sua sensibilidade aos dados observados. Além disso, a regressão linear pode gerar valores fora do intervalo [0,1], incompatíveis com a interpretação probabilística. Dessa forma, sua sensibilidade a ~~outliers~~ *outliers* e a ausência de uma saída naturalmente probabilística fazem da regressão linear uma abordagem inadequada para as tarefas de classificação necessárias a este trabalho.

~~Nesse sentido~~Mais adequada a esse fim, a regressão logística funciona~~rá~~de forma similar aosmodelos lineares (saída contínua), mas ~~será adaptado~~é adaptada ao nosso problema.

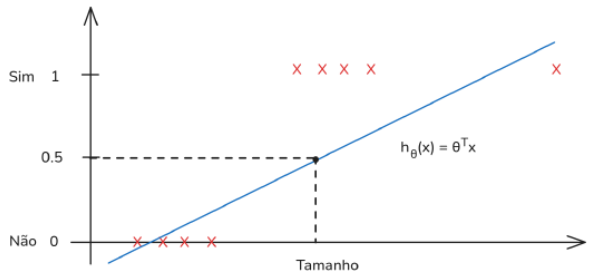



Figura 3.3: ~~O mesmo problema anterior~~Como na Figura 3.2, ~~mas~~ com a adição de um ponto discrepante (*outlier*). ~~Uma das curvas possui tamanho~~ Fonte: adaptado de ~~amplitude muito diferente da escala normal, possivelmente resultado da falta de normalização~~Levada (2026).~~Veja como a reta de regressão se desloca e o limiar 0,5 passa a classificar incorretamente (Levada , 2026).~~

O primeiro ponto ~~será~~ de adaptação é a interpretação probabilística: a combinação linear $\vec{\theta}^\top \vec{x}$ será mapeada pela logística ao intervalo $(0, 1)$, permitindo a interpretação como a probabilidade da classe ser positiva (Levada , 2026).

Em notação vetorial, seja $\vec{\theta}$ o vetor de parâmetros e $\vec{x}$ o vetor de atributos; a probabilidade predita é

$$\hat{p} = h_{\vec{\theta}}(\vec{x}) = \sigma(\vec{\theta}^\top \vec{x}) = \frac{1}{1 + e^{-\vec{\theta}^\top \vec{x}}}.$$

A decisão é: classe 1 se $\hat{p} \geqslant 0{,}5$, caso contrário classe 0. Isso ~~pois~~ porque a saída ~~continuará contínua , mas dessa vez limitada entre 0 e 1.~~ continua sendo contínua, mas agora limitada ao intervalo $(0, 1)$. Vale lembrar, também, que o valor de 0,5 pode ser alterado a depender da calibração do modelo, como discutido no início do capítulo.

O segundo ponto a ser adaptado será quanto ao treinamento. Nesse sentido, passa-se a utilizar outra função de custo ao invés dos mínimo quadrados: a **entropia cruzada** (ou *log loss*). Essa medida penaliza predições distantes do rótulo real. Por exemplo, se temos uma curva positiva (ocultação presente), e o modelo prediz uma probabilidade baixa de haver ocultação, o custo deverá ser alto.

$$loss = -[y \log p + (1 - y) \log (1 - p)]$$

onde p é a probabilidade prevista pelo modelo e a perda (loss) deverá ser aplicada a um único ponto. Ou seja, para um conjunto de ~~$m$~~ $n$ exemplos, com rótulos $y^{(i)} \in \{0, 1\}$ e probabilidades preditas $\hat{p}^{(i)}$, a função é

$$J(\vec{\theta}) = -\frac{1}{m}\frac{1}{n} \sum_{i=1}^{mn} \left[ y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right]. \tag{3.1}$$

A minimização de $J(\vec{\theta})$ é equivalente à maximização da log-verossimilhança (MLE), de modo que os parâmetros estimados são estatisticamente ótimos sob o modelo probabilístico assumido (Geron , 2019; Levada , 2026). O ajuste é realizado por gradiente

descendente (veremos mais detalhes sobre os algoritmos de otimização na Seção 3.5). O gradiente de $J$ em relação a $\vec{\theta}$ pode ser escrito na forma vetorial

$$\nabla_{\vec{\theta}} J(\vec{\theta}) = \tfrac{1}{m}\frac{1}{n} X^\top \big(\sigma(X\vec{\theta}) - \vec{y}\big),$$

onde $X$ é a matriz de dados e $\vec{y}$ o vetor de rótulos. A atualização dos parâmetros em cada iteração segue a **direção oposta ao gradiente** (direção de maior descida do custo), com passo controlado pela **taxa de aprendizado** $\eta$ (análogo ao passo de um método numérico de minimização). Trata-se, portanto, de uma medida sobre o quão distante da realidade estão as previsões do modelo.

**Exemplo numérico: uma iteração do gradiente descendente.** Para tornar o mecanismo concreto, considere ~~$m = 4$~~ $n = 4$ curvas com uma única ~~feature~~ *feature* $x$ (amplitude da queda de fluxo, normalizada entre 0 e 1) e rótulos $y$ (0 = sem ocultação, 1 = com ocultação):

| Curva | $x$ | $y$ | $\hat{p}$ (inicial) |
|---|---|---|---|
| 1 | 0,2 | 0 | 0,50 |
| 2 | 0,4 | 0 | 0,50 |
| 3 | 0,7 | 1 | 0,50 |
| 4 | 0,9 | 1 | 0,50 |

Tabela 3.1: Dados do exemplo numérico da Regressão Logística (valores ilustrativos).

Inicializamos $\theta_0 = 0$ e $\theta_1 = 0$. Como $\sigma(0) = 0{,}5$, todas as curvas recebem probabilidade $\hat{p} = 0{,}5$ e o custo inicial é $J = -\log(0{,}5) \approx 0{,}693$. Os gradientes são $\partial J/\partial\theta_0 = \frac{1}{4}\sum_i(\hat{p}_i - y_i) = 0$ e ~~$\partial J/\partial\theta_1 = \frac{1}{4}\sum_i(\hat{p}_i - y_i)\,x_i = -0{,}15$~~$\partial J/\partial\theta_1 = \frac{1}{4}\sum_i(\hat{p}_i - y_i)\,x_i = \frac{1}{4}(0{,}1 + 0{,}2 - 0{,}35 - 0{,}45) = -0{,}125$. Com taxa de aprendizado $\eta = 1{,}0$, a atualização dá ~~$\theta_1 \leftarrow 0 - 1{,}0 \times (-0{,}15) = 0{,}15$~~$\theta_1 \leftarrow 0 - 1{,}0 \times (-0{,}125) = 0{,}125$. Recalculando as probabilidades com o novo $\theta_1$, obtemos ~~$\hat{p}_1 = \sigma(0{,}03) \approx 0{,}507$, $\hat{p}_2 \approx 0{,}515$, $\hat{p}_3 \approx 0{,}526$ e $\hat{p}_4 \approx 0{,}534$~~$\hat{p}_1 = \sigma(0{,}025) \approx 0{,}506$, $\hat{p}_2 \approx 0{,}512$, $\hat{p}_3 \approx 0{,}522$ e $\hat{p}_4 \approx 0{,}528$. O custo cai para ~~$J \approx 0{,}685$~~$J \approx 0{,}678$. Uma única iteração já empurrou as probabilidades na direção correta: as curvas com maior amplitude (3 e 4, positivas) receberam $\hat{p}$ ligeiramente maior que as negativas. Após centenas de iterações, a separação se consolida ~~.~~e o custo diminui continuamente. Como esses quatro pontos são linearmente separáveis (todo $x \leqslant 0{,}4$ é negativo e todo $x \geqslant 0{,}7$ é positivo), sem regularização o custo só tende a zero no limite $|\theta_1| \to \infty$: não existe mínimo finito. Essa é uma das motivações para a regularização apresentada a seguir.

Sem nenhuma restrição, a Regressão Logística é livre para atribuir pesos enormes a qualquer ~~feature~~ *feature* — inclusive ao ruído. Se uma curva de treino por acaso tem um pico espúrio que coincide com a classe positiva, o modelo pode atribuir peso grande a essa flutuação; na hora de classificar curvas novas, esse peso exagerado prejudica a

predição. Esse é o problema do **sobreajuste** (~~overfitting~~*overfitting*, discutido em detalhe na Seção 3.6.3): o modelo "decorou" particularidades do treino em vez de aprender o padrão geral.

A solução é a **regularização**: impor um custo adicional aos pesos grandes, forçando o modelo a manter coeficientes moderados e a distribuir a "explicação" entre múltiplas *features* em vez de apostar tudo em uma (Geron , 2019; Levada , 2026). As três formas principais são:

- **Ridge ($\ell_2$):** adiciona ao custo o termo $\frac{\alpha}{2}\sum_j \theta_j^2$. Os coeficientes são *encolhidos* em direção a zero, mas raramente se anulam - todos os atributos permanecem no modelo. Funciona bem com *features* correlacionadas, reduzindo o sobreajuste sem descartar características.

- **Lasso ($\ell_1$):** adiciona $\alpha\sum_j |\theta_j|$. Tende a **zerar** coeficientes de atributos irrelevantes, atuando como seleção automática de variáveis. Em contrapartida, pode comportar-se de forma errática com muitas variáveis correlacionadas (tendendo a escolher apenas uma) ou quando o número de variáveis supera o de instâncias. A função de custo não é diferenciável em $\theta_j = 0$ (algoritmos de otimização não sabem a direção do próximo passo), exigindo subgradientes na otimização.

- **Elastic Net:** combina as penalidades Ridge e Lasso com uma taxa de mistura, unindo os benefícios de ambas. É recomendado quando há muitas variáveis correlacionadas.

Existe uma interpretação elegante dessas penalidades do ponto de vista bayesiano (Estimativa MAP — *Maximum A Posteriori*). Regularizar equivale a assumir uma crença prévia (*prior*) sobre os valores dos coeficientes do modelo *antes* de ver os dados. No Ridge, essa crença é uma **distribuição Gaussiana** centrada em zero: acredita-se que os pesos são provavelmente pequenos, mas não necessariamente nulos — as caudas suaves da Gaussiana permitem valores moderados (Figura 3.4). No Lasso, a crença é uma **distribuição de Laplace**, que tem um pico afiado em zero: acredita-se que muitos pesos *são* zero, e apenas alguns poucos são relevantes — daí o efeito de seleção automática de variáveis (Geron , 2019; Levada , 2026).

O hiperparâmetro $\alpha$ (ou $C = 1/\alpha$) controla a força da penalização: $\alpha$ grande significa uma crença prévia forte de que os pesos devem ser pequenos; $\alpha$ pequeno deixa o modelo mais livre. A escolha de $\alpha$ é feita por **validação cruzada**: treina-se o modelo com diferentes valores de $\alpha$ e seleciona-se aquele de melhor desempenho médio nas partições de validação ~~(as~~ (Geron , 2019), as chamadas *dobras* ~~, ou~~ (*folds*), definidas na Seção 3.6.5~~)~~ ~~(Geron , 2019)~~.

É essencial **escalonar** as *features* (~~e.g. StandardScaler~~por exemplo, com o `StandardScaler` do *scikit-learn*) antes de aplicar Ridge ou Lasso, pois ambas as penalidades são sensíveis

à escala dos dados~~.~~. Por exemplo, um atributo medido em quilômetros terá coeficiente numericamente muito diferente do mesmo atributo medido em metros, e a penalização tratará ambos de forma desigual. Nesta dissertação, utiliza-se Ridge ou Lasso na Regressão Logística, conforme documentado no Apêndice B.

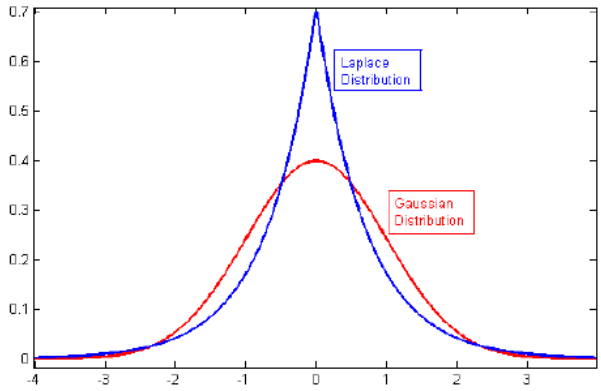



Figura 3.4: Comparação entre as distribuições Gaussiana e de Laplace, que correspondem aos priors assumidos sobre os parâmetros na interpretação bayesiana das regularizações Ridge e Lasso, respectivamente.

A Regressão Logística serve de base conceitual para entender classificadores probabilísticos e foi também treinada e avaliada na *pipeline* desta dissertação, junto com os demais modelos (Random Forest, XGBoost, CatBoost), que tipicamente oferecem melhor desempenho em dados tabulares e fornecem importância de variáveis. A função sigmoide $\sigma(z) = 1/(1 + e^{-z})$ está ilustrada na Figura 3.5.

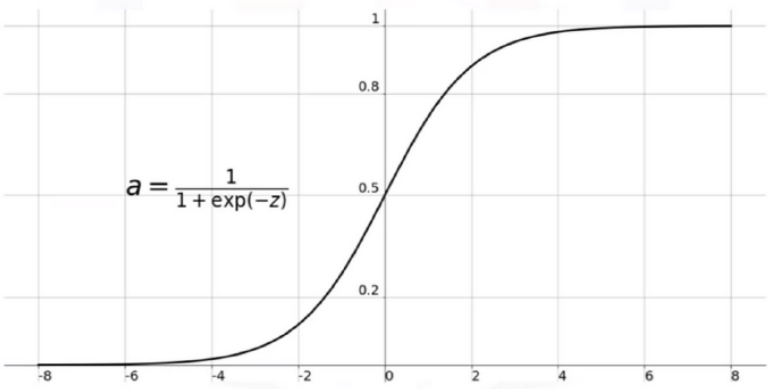



Figura 3.5: Função sigmoide $\sigma(z) = 1/(1 + e^{-z})$.

### 3.3.2 Árvores de decisão e critérios de impureza

Os modelos Random Forest, XGBoost e CatBoost são baseados em **árvores de decisão**. Uma árvore de decisão é um classificador que funciona como um fluxograma: em cada "“nó"—” pergunta-se o valor de uma das características (por exemplo, "“o mínimo

do fluxo suavizado é menor que 0,85?~~"~~"); conforme a resposta, segue-se por um dos dois ramos até um novo nó ou até uma ~~"folha"~~"folha", que devolve a classe predita. O espaço de entradas (vetores de características) fica assim particionado em regiões (chamadas de hiper-retângulos), cada uma associada a uma classe (Levada , 2026). Uma das maiores vantagens desses modelos é a **interpretabilidade**: as regras podem ser lidas e aplicadas manualmente, em contraste com modelos opacos como redes neurais. ~~O~~ A biblioteca Python *scikit-learn* (Pedregosa et al. , 2011), usada neste trabalho na Regressão Logística, no Random Forest, no K-means, na imputação, na padronização e no cálculo das métricas, utiliza o algoritmo **CART** (*Classification and Regression Trees*), que produz árvores binárias e escolhe, em cada nó, a divisão que maximiza a pureza dos nós filhos (Geron , 2019). Os dois critérios de impureza mais usados para classificação são o **coeficiente de Gini** e a **entropia**. Para um nó $i$ com proporções $p_{i,k}$ de exemplos na classe $k$ (com $k = 1, \ldots, c$ classes):

$$G_i = 1 - \sum_{k=1}^{c} p_{i,k}^2, \qquad H_i = -\sum_{k=1}^{c} p_{i,k} \log_2(p_{i,k}) \quad (\text{convenção } 0 \log 0 = 0). \tag{3.2}$$

Nessas expressões, $G_i$ é o coeficiente de Gini e $H_i$ é a **entropia** do nó $i$: ambos medem a impureza, ou seja, o quanto as classes estão misturadas naquele nó. Os dois valem zero quando o nó é puro (todos os exemplos pertencem a uma única classe) e crescem à medida que as classes ficam mais equilibradas entre si.

O Gini tende a isolar a classe mais frequente; a entropia tende a produzir árvores mais equilibradas. A divisão é feita de modo a maximizar a redução da impureza (ganho de informação). Árvores de decisão são instáveis e sensíveis a pequenas variações nos dados; sem restrições, tendem ao sobreajuste. Para regularizar, utilizam-se critérios de parada como a profundidade máxima ~~(*max_depth*) e~~ e o número mínimo de amostras por nó ou por folha ~~(*min_samples_leaf*) (Geron , 2019)~~ (Geron , 2019) — no *scikit-learn*, os hiperparâmetros `max_depth` e `min_samples_leaf`. A **poda** (*pruning*) pode ser aplicada após o crescimento completo, removendo ramos que não contribuem significativamente para a capacidade preditiva (Levada , 2026).

**Exemplo numérico: redução de impureza de Gini em uma divisão.** Considere um nó contendo 8 curvas de luz de treino, sendo 4 positivas (com ocultação) e 4 negativas. Como as classes estão perfeitamente misturadas, a impureza é máxima para o caso binário:

$$G_{\text{nó}} = 1 - \left[ \left(\tfrac{4}{8}\right)^2 + \left(\tfrac{4}{8}\right)^2 \right] = 1 - 0{,}5 = 0{,}5.$$

A árvore testa então uma divisão candidata — por exemplo, "profundidade da queda $> 0{,}5$?" — que separa as 8 curvas em dois nós-filhos:

- **Filho da esquerda** (3 curvas, todas positivas): $G_{\text{esq}} = 1 - [1^2 + 0^2] = 0$ (nó *puro*).

- **Filho da direita** (5 curvas, 1 positiva e 4 negativas): $G_{\text{dir}} = 1 - \left[\left(\frac{1}{5}\right)^2 + \left(\frac{4}{5}\right)^2\right] = 1 - 0{,}68 = 0{,}32$.

A impureza após a divisão é a média **ponderada** pelo número de exemplos em cada filho:

$$G_{\text{div}} = \frac{3}{8} \times 0 + \frac{5}{8} \times 0{,}32 = 0{,}20,$$

e o **ganho** (redução de impureza) é $\Delta G = G_{\text{nó}} - G_{\text{div}} = 0{,}50 - 0{,}20 = 0{,}30$. A divisão deixou os nós mais "puros" (o da esquerda já decide com certeza pela classe positiva). O algoritmo CART avalia todos os cortes candidatos de todas as *features* e escolhe aquele que produz o *maior* ganho; o processo se repete em cada nó-filho até atingir um critério de parada.

### 3.3.3 Random Forest (Floresta Aleatória)

Random Forest (Breiman , 2001) é um método de ~~ensemble~~***ensemble***: em vez de uma única árvore, treina-se um grande número de árvores de decisão (por exemplo, centenas), cada uma em uma **amostra aleatória** dos dados (amostragem com reposição, ou *bootstrap*) e usando, em cada divisão de nó, apenas um subconjunto aleatório das características (Geron , 2019; Levada , 2026). A predição final é obtida por **votação majoritária**: cada árvore ~~"vota"~~"vota" em uma classe e escolhe-se a classe que recebeu mais votos. A força da floresta está na **descorrelação** entre as árvores. Aqui, ***bagging*** (de *bootstrap aggregating*) é a técnica de treinar cada árvore em uma amostra distinta dos dados, sorteada com reposição, e depois agregar as predições por votação: como cada árvore vê um conjunto ligeiramente diferente, elas erram em casos diferentes. O *bagging* e a escolha aleatória de atributos em cada nó garantem essa diversidade; ao agregar centenas de árvores, o modelo mantém o baixo viés das árvores individuais e reduz drasticamente a **variância**. Uma vantagem colateral é a estimativa ~~**out-of-bag**~~***out-of-bag*** (OOB): como cada árvore é treinada em aproximadamente 63% dos dados (amostragem com reposição), cerca de 37% das instâncias ficam ~~"~~"fora da bolsa"~~"~~ para cada árvore e podem ser usadas como conjunto de validação interno, permitindo estimar o erro de generalização sem um conjunto de teste separado (Geron , 2019). Uma variante ainda mais aleatória são as **Extra-Trees** (*Extremely Randomized Trees*): em vez de procurar o melhor ponto de corte em cada atributo, sorteiam limiares ao acaso e ficam com o melhor entre os sorteados. Isso acelera o treinamento e pode reduzir ainda mais a variância, ao custo de um viés ligeiramente maior (Geron , 2019).

A aleatoriedade em dois níveis — nos dados e nos atributos — reduz a correlação entre as árvores e diminui a **variância** do ~~ensemble~~*ensemble*: ao agregar predições de árvores pouco correlacionadas, o erro global tende a ser menor que o de uma única árvore (Geron

, 2019). O **Teorema do Júri de Condorcet**[1] ilustra a ideia: se cada ~~"votante"~~"votante" (árvore) tem probabilidade maior que 50% de acertar, a decisão por maioria tende a melhorar com o número de votantes. Por exemplo, várias árvores com acurácia individual em torno de 65% podem produzir um ~~ensemble~~ *ensemble* com acurácia bem maior (da ordem de 90% ou mais), desde que os erros não sejam idênticos. O ganho depende da **diversidade** entre os modelos: se todos errarem nos mesmos casos, a votação não corrige.

Além da predição, Random Forest fornece uma estimativa natural da **importância das variáveis** (por exemplo, pela queda média na impureza ou na acurácia quando a variável é permutada), o que é útil para análise exploratória e seleção de ~~features~~ *features* (Capítulo 4). Nesta dissertação, a importância das variáveis nos modelos baseados em árvores (Random Forest, XGBoost, CatBoost) é obtida pela **queda média na impureza** (Gini) em cada divisão em que a variável participa; para a Regressão Logística, utilizam-se os coeficientes estimados (em valor absoluto ou normalizado). O critério adotado na *pipeline* é detalhado no Capítulo 4, e os resultados (ranking e magnitude das importâncias) são apresentados no Capítulo 5.

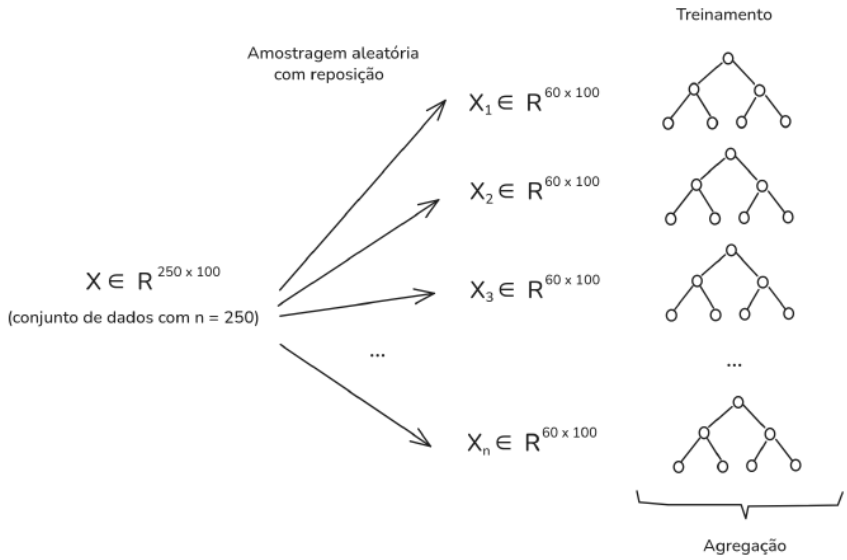



Figura 3.6: Esquema simplificado do funcionamento de ~~uma Floresta Aleatória~~um Random Forest.

[1] Formulado por Condorcet (1785): se $n$ votantes independentes acertam, cada um, com probabilidade $p > 1/2$, a probabilidade de a maioria acertar é $P_n = \sum_{k>n/2} \binom{n}{k} p^k (1-p)^{n-k}$, que cresce com $n$ e tende a 1. Para $n = 25$ e $p = 0{,}65$, obtém-se $P_{25} \approx 0{,}94$ (Figura 3.7). Como as árvores de uma floresta não são independentes, o ganho real é menor; ver Geron (2019).

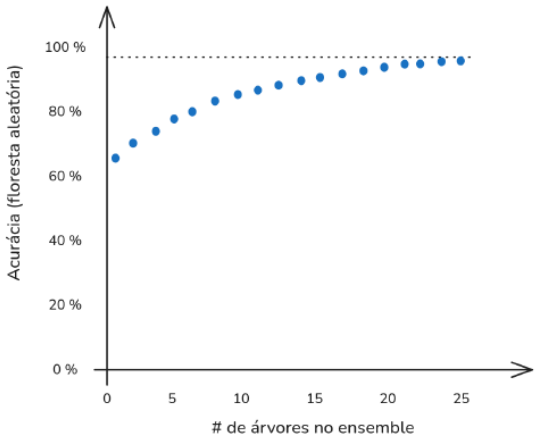



Figura 3.7: Exemplo ilustrativo: 25 árvores com acurácia individual em torno de 65% podem resultar em uma floresta com acurácia de aproximadamente 94%.

### 3.3.4 XGBoost (~~eXtreme Gradient Boosting~~*eXtreme Gradient Boosting*)

O ~~boosting~~***boosting*** é uma família de métodos em que os modelos são treinados **um após o outro**: cada novo modelo (por exemplo, uma árvore) é ajustado para corrigir os erros do modelo anterior, de modo que a combinação de todos forme um preditor forte a partir de vários preditores ~~"fracos"~~"fracos" (Geron , 2019; Levada , 2026). O ~~**gradient boosting**~~***gradient boosting*** utiliza um modelo aditivo: na iteração $m$,

$$~~F~~\hat{f}_m(\vec{x}) = ~~F~~\hat{f}_{m-1}(\vec{x}) + \eta\, h_m(\vec{x}), \tag{3.3}$$

onde $h_m$ é um novo aprendiz fraco (por exemplo, uma árvore rasa) ajustado ao **gradiente negativo** da função de perda em relação à predição atual ~~$F_{m-1}$~~ $\hat{f}_{m-1}$ — ou seja, aos ~~"~~"resíduos~~"~~" no espaço da perda (Geron , 2019). A taxa de aprendizado $\eta$ controla a contribuição de cada árvore (encolhimento, *shrinkage*), reduzindo o risco de ~~overfitting~~*overfitting*. O XGBoost (*eXtreme Gradient Boosting*) (Chen & Guestrin , 2016) aprimora esse esquema ~~ao adicionar termos de regularização explícitos na~~ de duas maneiras. Primeiro, adiciona à função de custo ~~(penalizando~~ termos de regularização explícitos, que penalizam o número de folhas e os pesos das folhas~~) e ao utilizar~~. Segundo, usa uma expansão de Taylor de segunda ordem da perda para ~~otimizar~~ calcular a atualização de cada árvore~~, o que resulta em~~. O resultado é uma implementação eficiente e de alto desempenho (Geron , 2019).

O XGBoost tornou-se muito popular em competições ~~e~~ de ciência de dados (como as da plataforma Kaggle) e em aplicações práticas, por sua precisão e velocidade. Em contraste com o Random Forest (que treina árvores em paralelo), o ~~boosting~~ *boosting* é inerentemente sequencial, o que pode limitar o paralelismo em certas fases, mas o ganho

de desempenho preditivo costuma compensar. ~~Na *pipeline* desta dissertação, o XGBoost é um dos quatro classificadores treinados e avaliados (Regressão Logística, Random Forest, XGBoost, CatBoost) (Capítulo 4).~~

### 3.3.5 CatBoost

CatBoost (*Categorical Boosting*) é um algoritmo de *gradient boosting* baseado em árvores, desenvolvido pela Yandex, empresa russa de tecnologia conhecida por seu mecanismo de busca (Prokhorenkova et al. , 2018), que incorpora inovações voltadas à robustez e à eficiência do treinamento.

Assim como o XGBoost, o CatBoost constrói árvores sequencialmente, minimizando uma função de perda por gradiente. Entretanto, sua arquitetura difere em dois aspectos ~~arquiteturais~~ relevantes:

- ***Ordered Boosting*:** para calcular os resíduos de cada exemplo, o CatBoost utiliza apenas os modelos treinados com exemplos anteriores a ele (em uma permutação aleatória do conjunto), combatendo o *target leakage* (vazamento de rótulo) — situação em que a predição de um exemplo é influenciada pelo próprio rótulo durante o treinamento. Esse procedimento reduz o viés otimista na estimativa dos resíduos, especialmente em conjuntos pequenos ou moderados.

- ***Oblivious Decision Trees*:** cada nível da árvore aplica a **mesma condição de divisão** a todos os nós daquele nível. Essa restrição produz árvores simétricas (ou ~~"oblivious"~~*oblivious*), que são mais rápidas de avaliar e atuam como regularização estrutural, reduzindo o risco de *overfitting*.

O CatBoost também possui tratamento nativo de variáveis categóricas — codificando-as internamente sem necessidade de *one-hot encoding* (codificação que substitui uma variável categórica com $c$ valores possíveis por $c$ colunas binárias, uma por categoria) — e opções para lidar com desbalanceamento de classes, como o parâmetro ~~*auto_class_weights='Balanced'*~~`class_weights='Balanced'`, que ajusta automaticamente os pesos das classes de forma inversamente proporcional à sua frequência. Neste trabalho, todas as *features* são numéricas, de modo que o tratamento de variáveis categóricas não é utilizado.

A métrica de importância de variáveis padrão do CatBoost é o ***PredictionValuesChange***: para cada *feature*, mede-se a mudança média nas predições do modelo ao longo de todos os *splits* que a utilizam. Os valores são normalizados para que a soma seja igual a 100. Neste trabalho, CatBoost é utilizado junto com Regressão Logística, Random Forest e XGBoost para comparação de desempenho e robustez na classificação de curvas de luz (Capítulo 4).

## 3.4 Agrupamento não supervisionado: K-means

O **K-means**, proposto originalmente por MacQueen (1967), é um algoritmo de **agrupamento** (~~clustering~~*clustering*) não supervisionado: não usa rótulos; apenas particiona os dados em ~~*k*~~ $K$ grupos de modo a minimizar a soma dos quadrados das distâncias de cada ponto ao centróide do seu grupo (Geron , 2019; Levada , 2026). A função objetivo, conhecida como *Within-Cluster Sum of Squares* (WCSS) ou inércia, é

$$J = \sum_{i=1}^{~~k~~K} \sum_{\vec{x} \in S_i} \|\vec{x} - \vec{\mu}_i\|^2, \tag{3.4}$$

onde $S_i$ é o conjunto de pontos atribuídos ao ~~cluster~~ *cluster* $i$ e $\vec{\mu}_i$ é o centróide desse ~~cluster~~*cluster*. O algoritmo alterna dois passos: (1) **atribuição** — cada ponto é atribuído ao centróide mais próximo; (2) **atualização** — cada centróide é recalculado como a média aritmética dos pontos a ele atribuídos. O custo $J$ não aumenta a cada iteração, de modo que o processo converge para um mínimo local (Geron , 2019; Levada , 2026). Em aplicações em que ~~*k*~~ $K$ não é fixo, a inércia (WCSS) é frequentemente usada no **método do cotovelo** (gráfico de WCSS em função de ~~*k*~~$K$) para sugerir o número de ~~clusters~~*clusters*; índices como o de ~~**Calinski-Harabasz**~~**Caliski–Harabasz**[2] medem a razão entre dispersão inter e ~~intra-cluster~~intra-*cluster*, indicando quão bem separados estão os grupos. Para mais detalhes sobre o funcionamento deste algoritmo, fica como recomendação o paper original: MacQueen (1967).

~~Nesta~~ Como mencionado na Seção 3.1, os classificadores utilizados neste trabalho são supervisionados; o uso do K-means limita-se à etapa de extração de características numéricas das curvas de luz de ocultação. Assim, nesta dissertação, o K-means **não** é usado como classificador de curvas, e sim como ferramenta para **extrair uma característica numérica**: aplica-se K-means com $K = 2$ aos valores do fluxo (ou do fluxo suavizado) de cada curva. Como não há rótulos, o algoritmo apenas descobre dois ~~"~~"níveis~~"~~" típicos nos dados (Levada , 2026). Os dois centróides representam, grosso modo, o nível ~~"~~"fora da ocultação~~"~~" e o nível ~~"~~"dentro da ocultação~~"~~". A **distância entre os centróides** é então usada como mais uma entrada (~~feature~~*feature*) para o classificador: curvas com ocultação bem definida tendem a ter distância maior; curvas apenas ruidosas tendem a ter distância menor. O detalhe está no Capítulo 4. No uso aqui, ~~*k* = 2~~ $K = 2$ e a variável é unidimensional (fluxo), o que simplifica a interpretação.

[2] Índice proposto por Caliński & Harabasz (1974): $CH(K) = \dfrac{\mathrm{tr}(B_K)/(K-1)}{\mathrm{tr}(W_K)/(n-K)}$, em que $B_K$ e $W_K$ são as matrizes de dispersão entre os grupos e dentro dos grupos, e $n$ é o número de pontos. Valores maiores indicam grupos mais compactos e mais bem separados.

## 3.5 Algoritmos de otimização no treinamento

Em um projeto de ML supervisionado, a etapa de **treinamento** corresponde, como explicado anteriormente, a ajustar os parâmetros do modelo de modo a minimizar uma função de custo (ou perda) nos dados. Nas seções anteriores foram apresentados os modelos e suas funções de custo; aqui unificamos os **algoritmos** usados para obter o minimizador. A minimização pode ser feita por **solução analítica fechada**, quando existe fórmula explícita (como na regressão linear, Seção 3.2), ou por **algoritmos iterativos**, quando não há forma fechada. Os três esquemas mais recorrentes são resumidos a seguir; a Tabela 3.2 indica em que contexto cada um aparece.

**1. Mínimos quadrados (solução fechada).** Conforme visto na regressão linear (Seção 3.2), modelos lineares com custo quadrático possuem minimizador analítico dado pela equação normal $\vec{\theta}^* = (X^\top X)^{-1} X^\top \vec{y}$. Na Regressão Logística e em modelos não lineares, recorre-se a métodos iterativos.

**2. Gradiente descendente (método de primeira ordem).** O **gradiente** da função de custo indica a direção de maior crescimento do custo; para minimizar, atualiza-se os parâmetros na **direção oposta**, com passo controlado pela taxa de aprendizado $\eta$:

$$\vec{\theta}_{t+1} = \vec{\theta}_t - \eta \, \nabla_{\vec{\theta}} J(\vec{\theta}_t).$$

As variantes principais distinguem-se pela quantidade de dados usada em cada atualização (Geron , 2019): ~~**Batch**~~ ***Batch*** **GD** utiliza todo o conjunto de treino (preciso, mas lento em grandes volumes); ~~**Stochastic**~~ ***Stochastic*** **GD** (SGD) utiliza uma única instância aleatória por passo (rápido e capaz de escapar de mínimos locais, mas com trajetória errática); ~~**Mini-batch**~~ ***Mini-batch*** **GD** processa pequenos subconjuntos aleatórios, equilibrando estabilidade e eficiência. A convergência depende crucialmente de $\eta$: se muito alta, o algoritmo pode divergir; se muito baixa, o treinamento torna-se excessivamente lento. Como vimos na Seção 3.3, a Regressão Logística é ajustada por gradiente descendente; o mesmo esquema é usado em redes neurais e no ~~gradient boosting~~ *gradient boosting* (cada árvore é ajustada ao gradiente da perda).

**3. Métodos de segunda ordem (Newton e variantes).** O **método de Newton** utiliza o gradiente e a **Hessiana**[3] da função de custo:

$$\vec{\theta}_{t+1} = \vec{\theta}_t - H^{-1}(\vec{\theta}_t) \, \nabla_{\vec{\theta}} J(\vec{\theta}_t).$$

Isso permite passos mais informados e convergência mais rápida perto do mínimo, às custas de calcular e inverter $H$ a cada iteração (custo $O(p^3)$ por passo). Variantes **quasi-Newton**~~(como BFGS)~~, como o BFGS[4], aproximam a Hessiana sem calculá-la explici-

[3] Matriz das derivadas parciais de segunda ordem da função de custo, $H_{ij} = \partial^2 J / \partial\theta_i \, \partial\theta_j$, que descreve a curvatura local de $J$ (Nocedal & Wright , 2006).

[4] Sigla de Broyden–Fletcher–Goldfarb–Shanno: o método atualiza, a cada iteração, uma

tamente. O XGBoost utiliza uma **expansão de Taylor de segunda ordem** da perda para determinar a atualização ótima de cada árvore no ~~gradient boosting~~ *gradient boosting* (Geron , 2019).

Tabela 3.2: Algoritmos de otimização e seu papel na etapa de treinamento em projetos de ML.

| **Família** | **Exemplo** | **Onde aparece** |
|---|---|---|
| Solução fechada | Mínimos quadrados (equação normal) | Regressão linear (Seção 3.2) |
| Primeira ordem (iterativo) | Gradiente descendente (~~Batch~~*Batch*, SGD, ~~Mini-batch~~*Mini-batch*) | Regressão Logística, redes neurais, ~~gradient boosting~~ *gradient boosting* |
| Segunda ordem (iterativo) | Newton, quasi-Newton (BFGS), Taylor | XGBoost (Taylor da perda); otimização convexa em geral |

~~Com isso, o leitor dispõe de uma referência clara sobre o significado das "contas"envolvidas no treinamento de modelos de aprendizado de máquina. Inicialmente, apresentamos um modelo simples de Regressão Linear ajustado por meio do método dos mínimos quadrados e discutimos suas limitações. Em seguida, introduzimos a Regressão Logística (Seção 3.3) como um exemplo direto de otimização por gradiente descendente. Por fim, abordamos os ensembles de árvores, que utilizam critérios de divisão para o aprendizado; no caso do XGBoost, o processo de otimização incorpora informações de segunda ordem da função de perda, resultando em atualizações mais eficientes (Geron , 2019).~~

## 3.6 Engenharia de características e validação de modelos

### 3.6.1 Engenharia de características

A criação e a seleção de **características numéricas** (atributos) a partir dos dados brutos são fundamentais para o sucesso do ML (Geron , 2019), uma vez que essas variáveis representam a informação que será efetivamente utilizada pelos algoritmos para identificar padrões e realizar previsões.

Essa etapa inclui: (i) escolher ou construir variáveis que capturem informação útil para a tarefa (por exemplo, estatísticas da curva de luz, derivadas, comparações entre segmentos); (ii) combinar ou transformar variáveis existentes (médias móveis, suavização, testes estatísticos entre quartis, etc.). No Capítulo 4, descreve-se em detalhe o conjunto de ~~features~~ *features* extraídas de cada curva de luz (amplitude, desvio padrão, ~~max drawdown~~queda máxima acumulada (*max drawdown*), distância entre centróides do

aproximação de $H^{-1}$ usando apenas as variações sucessivas dos parâmetros e do gradiente (Nocedal & Wright , 2006, cap. 6).

K-means no fluxo, estatísticas de derivadas, $p$-valores entre quartis, entre outras). Essas grandezas formam o vetor $\vec{x}$ de entrada dos classificadores.

### 3.6.2 Métricas de avaliação para classificação binária

A avaliação de um classificador binário parte da **matriz de confusão**, que contabiliza os acertos e erros do modelo em relação aos rótulos verdadeiros. Sejam $y_i \in \{0, 1\}$ os rótulos reais e $\hat{y}_i$ as predições; definem-se quatro contagens:

- **Verdadeiro Positivo (TP, do inglês *True Positive*):** $\hat{y}_i = 1$ e $y_i = 1$ (acerto na classe positiva).
- **Verdadeiro Negativo (TN, *True Negative*):** $\hat{y}_i = 0$ e $y_i = 0$ (acerto na classe negativa).
- **Falso Positivo (FP, *False Positive*):** $\hat{y}_i = 1$ e $y_i = 0$ (alarme falso).
- **Falso Negativo (FN, *False Negative*):** $\hat{y}_i = 0$ e $y_i = 1$ (evento perdido).

A partir dessas contagens, as métricas mais utilizadas em classificação binária são:
**Acurácia** — fração de predições corretas sobre o total:

$$\text{Acurácia} = \frac{TP + TN}{TP + TN + FP + FN}. \tag{3.5}$$

A acurácia é intuitiva, mas pode ser enganosa em conjuntos desbalanceados.

**Exemplo numérico:** considere 100 curvas de luz, das quais 10 contêm ocultação (positivas) e 90 não (negativas). Um classificador que **sempre** prediz ~~"negativo"~~"negativo" acerta 90 das 100 curvas — acurácia de 90% — mas não detecta *nenhuma* ocultação (sensibilidade = 0). Esse paradoxo evidencia que, em cenários desbalanceados, a acurácia sozinha é insuficiente para avaliar o modelo.

**Precisão** (*Precision*) — entre os exemplos classificados como positivos, qual fração é realmente positiva:

$$\text{Precisão} = \frac{TP}{TP + FP}. \tag{3.6}$$

**Sensibilidade** (*Recall*, ou sensibilidade) — entre os exemplos verdadeiramente positivos, qual fração foi corretamente detectada:

$$\text{Sensibilidade} = \frac{TP}{TP + FN}. \tag{3.7}$$

**F1-score** — média harmônica de precisão e sensibilidade, oferecendo um compromisso entre ambas:

$$F_1 = 2 \cdot \frac{\text{Precisão} \cdot \text{Sensibilidade}}{\text{Precisão} + \text{Sensibilidade}} = \frac{2\,TP}{2\,TP + FP + FN}. \tag{3.8}$$

**$F_\beta$-score** — generalização do F1-score que permite atribuir peso diferente à sensibilidade:

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precisão} \cdot \text{Sensibilidade}}{\beta^2 \cdot \text{Precisão} + \text{Sensibilidade}}. \tag{3.9}$$

Para $\beta > 1$, o $F_\beta$ penaliza mais os falsos negativos (favorecendo a sensibilidade); para $\beta < 1$, penaliza mais os falsos positivos. Em aplicações de triagem científica, como a detecção de ocultações estelares, $\beta = 2$ é uma escolha natural, pois o custo de perder um evento genuíno ~~(FN)~~ é maior que o de investigar um alarme falso~~(FP) (Saito & Rehmsmeier , 2015)~~ , isto é, o custo de um FN supera o de um FP.

**AUC-ROC** (*Area Under the Receiver Operating Characteristic Curve*) — mede a capacidade discriminativa do modelo ao longo de todos os limiares de decisão possíveis. A curva ROC plota a **taxa de verdadeiros positivos** (sensibilidade) contra a **taxa de falsos positivos** ($FP/(FP+TN)$) para cada limiar $\tau$. A área sob essa curva (AUC) varia de 0,5 (classificador aleatório) a 1,0 (classificador perfeito), como mostrado na ~~imagem 3.8.~~Figura 3.8: quanto mais a curva se aproxima do canto superior esquerdo, melhor a capacidade discriminativa do modelo. Um exemplo intuitivo é um teste médico: a taxa de verdadeiros positivos é a fração de doentes corretamente identificados, a taxa de falsos positivos é a fração de saudáveis apontados como doentes, e variar o limiar move o teste ao longo da curva, negociando a detecção de mais doentes contra um número maior de alarmes falsos. A AUC-ROC é útil por ser invariante ao limiar escolhido, mas pode ser otimista em conjuntos desbalanceados (Saito & Rehmsmeier , 2015).

**Curva Precisão-Sensibilidade** (*Precision-Recall*) — alternativa à curva ROC, recomendada quando a classe positiva é rara ou quando o custo dos erros é assimétrico (Saito & Rehmsmeier , 2015). Plota a precisão contra a sensibilidade para cada limiar $\tau$. A área sob a curva PR (AUC-PR) é mais informativa que a AUC-ROC em cenários desbalanceados, pois não é inflada por verdadeiros negativos abundantes.

Nesta dissertação, os modelos são avaliados por acurácia, precisão, sensibilidade, F1-score e AUC-ROC no conjunto de teste (Capítulo 5). Para a análise de ajuste de limiar, utilizam-se também o $F_2$-score e a curva ~~precision-recall~~*precision-recall*.

### 3.6.3 ~~Overfitting, underfitting~~ *Overfitting*, *underfitting* e o compromisso entre viés e variância

A identificação de **sobreajuste** (~~overfitting~~*overfitting*) e **subajuste** (~~underfitting~~*underfitting*) baseia-se na comparação do desempenho do modelo nos dados usados no treinamento versus nos dados reservados para teste ou validação (Geron , 2019). Em termos intuitivos: **sobreajuste** ocorre quando o modelo ~~"decora"~~"decora" os dados de treino (incluindo o ruído) e deixa de generalizar — o erro no treino fica muito baixo, mas o erro em dados

[4] ~~Disponível em: . Acesso em: 6 abr. 2026.~~

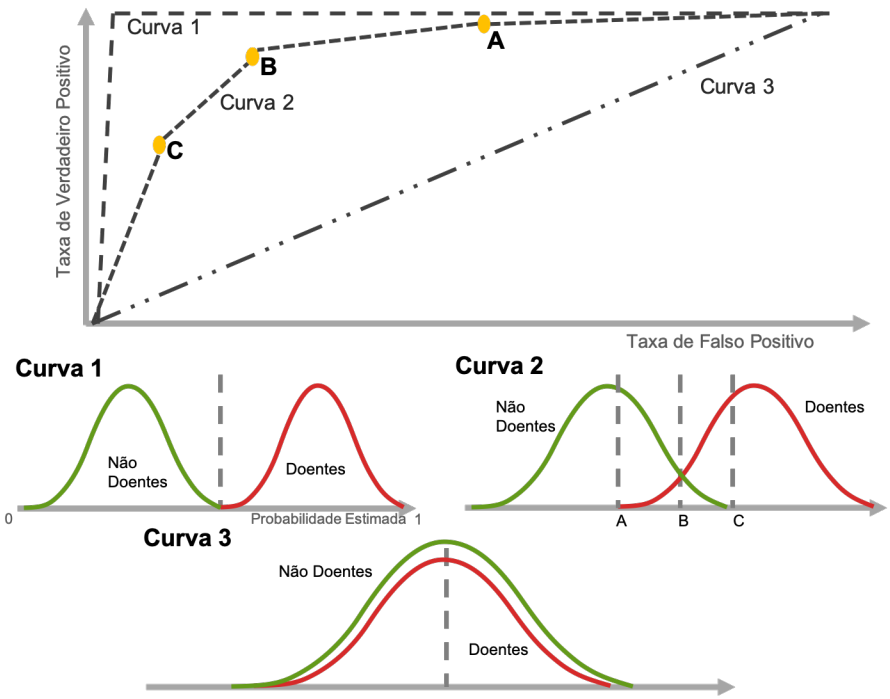



Figura 3.8: Exemplos de curvas ROC para classificadores de diferentes qualidades~~: quanto mais a curva se aproxima do canto superior esquerdo (maior área sob a curva, AUC), melhor a capacidade discriminativa do modelo~~; a diagonal corresponde a um classificador aleatório (AUC = 0,5). ~~Um exemplo intuitivo é um teste médico que classifica pacientes como doentes ou saudáveis~~Fonte: ~~a taxa~~ adaptado de ~~verdadeiros positivos mede a fração de doentes corretamente identificados, enquanto a taxa de falsos positivos mede a fração de saudáveis erroneamente apontados como doentes~~Ciência e Negócios. ~~Variar o limiar de decisão move o teste ao longo da curva, negociando a detecção de mais doentes contra um número maior de alarmes falsos~~Disponível em: `https://cienciaenegocios.com/curva-roc-e-auc-em-machine-learning`. ~~Fonte~~Acesso em: ~~adaptado de *Ciência e Negócios*~~6 abr. 2026.

novos permanece alto. **Subajuste** ocorre quando o modelo é simples demais e não captura o padrão verdadeiro dos dados — tanto o erro de treino quanto o de validação ficam altos.

O erro de generalização pode ser decomposto em **viés**, **variância** e **ruído irredutível** (Levada , 2026; Deisenroth et al. , 2019). Formalmente, ~~para um modelo~~ seja $f^\star(\vec{x})$ a relação verdadeira (desconhecida) entre entrada e saída e $\hat{f}$ ~~treinado em diferentes amostras~~ o modelo — a função $f$ da Seção 3.2 — estimado a partir de diferentes amostras de treino do mesmo processo gerador~~,~~; o erro quadrático médio esperado em um ponto $\vec{x}$ é:

$$\mathbb{E}\Big[(y - \hat{f}(\vec{x}))^2\Big] = \text{Viés}^2[\hat{f}(\vec{x})] + \text{Var}[\hat{f}(\vec{x})] + \sigma^2, \tag{3.10}$$

onde ~~$\text{Viés}^2[\hat{f}] = \Big(\mathbb{E}[\hat{f}(\vec{x})] - f(\vec{x})\Big)^2$~~ $\text{Viés}^2[\hat{f}] = \Big(\mathbb{E}[\hat{f}(\vec{x})] - f^\star(\vec{x})\Big)^2$ mede o erro sistemático do modelo (quão longe a predição média está do valor verdadeiro), $\text{Var}[\hat{f}]$ mede a

sensibilidade do modelo a flutuações nos dados de treino, e $\sigma^2$ é a variância do ruído intrínseco aos dados (irredutível por qualquer modelo). Modelos muito simples tendem a alto viés (subajuste); modelos muito flexíveis tendem a alta variância (sobreajuste). O objetivo prático é encontrar um equilíbrio que minimize o erro total.

**Diagnóstico.** O sinal mais claro de sobreajuste é um erro muito baixo no treino e um erro alto no teste ou validação (não generalizou). Esse diagnóstico é feito graficamente pelas **curvas de aprendizado** (Seção 3.6.4): o sobreajuste aparece como um **gap** entre a curva de treino e a de validação, enquanto o subajuste mantém ambas próximas e em um **platô alto** de erro.

A **validação cruzada** — detalhada na Seção 3.6.5 — complementa esse diagnóstico ao fornecer uma estimativa robusta do erro de generalização (média e desvio padrão sobre várias divisões dos dados), facilitando perceber quando o modelo começou a sobreajustar (Geron , 2019).

Em resumo: **sobreajuste** (~~overfitting~~*overfitting*) significa que o modelo se ajustou demais aos dados de treino (incluindo o ruído) e perde capacidade de generalizar; soluções típicas são obter mais dados, simplificar o modelo ou usar regularização (penalizar parâmetros grandes). **Subajuste** (~~underfitting~~*underfitting*) significa que o modelo é simples demais e não captura o padrão dos dados; as soluções envolvem aumentar a complexidade do modelo ou enriquecer as características de entrada. Os **hiperparâmetros** (por exemplo, força da regularização, profundidade máxima das árvores) controlam esse equilíbrio e podem ser escolhidos via validação cruzada.

A **regularização** restringe a liberdade do modelo (por exemplo, penalidade $\ell_2$ na Regressão Logística ou limites de profundidade e número de folhas nas árvores), sendo controlada por hiperparâmetros que podem ser ajustados via validação cruzada (Geron , 2019). Outras formas incluem a **parada antecipada**: interromper o treinamento quando o erro no conjunto de validação deixa de melhorar, evitando que o modelo ~~"decore"~~“decore” o ruído do treino. Nesta dissertação, o foco está na penalização de pesos em modelos lineares (Ridge/Lasso) e nos critérios de parada e poda nas árvores. A regularização $\ell_1$ e $\ell_2$ é **sensível à escala** das variáveis de entrada: atributos em escalas maiores tendem a receber pesos menores pela penalização. Em geral, recomenda-se **padronizar** as variáveis (subtrair a média e dividir pelo desvio padrão) antes de aplicar Ridge ou Lasso.

~~Na *pipeline* desta dissertação, as features não~~ No entanto, nesta dissertação a padronização é aplicada apenas à Regressão Logística; os modelos baseados em árvores recebem as *features* na escala original, pois são ~~padronizadas antes de todos os treinos~~ insensíveis a transformações monotônicas de cada variável (Capítulo 4)~~; a justificativa é discutida naquele capítulo~~.

### 3.6.4 Curvas de aprendizado

As **curvas de aprendizado** mostram o desempenho do modelo (acurácia ou erro) em função do tamanho do conjunto de treino (ou do número de épocas); são a ferramenta gráfica que torna visível o equilíbrio viés–variância discutido acima e auxiliam na decisão de coletar mais dados ou alterar a complexidade do modelo.

A Figura 3.9 ilustra o conceito com curvas de aprendizado ~~reais de três classificadores (XGBoost, Random Forest e Árvore de Decisão)em uma tarefa~~ de seis classificadores — (A) XGBoost, (B) Random Forest, (C) árvore de decisão, (D) *Extra Trees*, (E) *Naive Bayes* gaussiano e (F) Regressão Logística — aplicados a um problema de classificação em medicina (Zhong et al. , 2023). Em cada painel, a curva superior é o desempenho no **treino** e a inferior o desempenho em **validação cruzada**, ambos em função do número de amostras de treino. Dois padrões didáticos aparecem: (i) a curva de validação **sobe** à medida que mais dados ficam disponíveis e tende a um **platô**, ponto a partir do qual coletar mais exemplos pouco acrescenta; (ii) o **intervalo** (*gap*) entre a curva de treino (quase perfeita) e a de validação mede o **sobreajuste** — quanto maior o vão, mais o modelo decorou o treino em vez de generalizar. Um modelo bem ajustado é aquele cujas duas curvas convergem para um patamar alto com vão pequeno~~.—~~; na figura, é o caso do XGBoost (painel A), cujas curvas convergem para $\approx 0{,}96$. Os painéis E e F ilustram o subajuste: as curvas se encontram, mas num patamar baixo ($\approx 0{,}67$ e $\approx 0{,}74$). Com poucos exemplos de treino, todos os modelos mostram o vão largo típico do sobreajuste.

### 3.6.5 Validação de modelos

Para estimar o desempenho em dados não vistos e evitar otimismo, é essencial separar os dados usados no treino daqueles usados na avaliação (Geron , 2019). Imagine que treinamos o modelo usando todas as curvas de luz disponíveis e depois o avaliamos nessas mesmas curvas. O resultado seria enganosamente bom. Para estimar honestamente o desempenho, é preciso reservar dados que o modelo nunca viu. A **validação cruzada** sistematiza esse processo: em vez de uma única divisão treino/teste (que pode ser sortuda ou azarada), roda-se o experimento ~~$k$~~ $q$ vezes, cada vez deixando uma fatia diferente de fora (Figura 3.10). A média dos ~~$k$~~ $q$ resultados é uma estimativa muito mais robusta do erro de generalização.

As estratégias mais comuns são:

- **Conjunto de teste:** uma fração dos dados é reservada e nunca usada no treino; o desempenho nesse conjunto fornece uma estimativa do erro de generalização.

[4] ~~Disponível em: . Acesso em: 6 abr. 2026.~~
[4] ~~Disponível em: . Acesso em: 6 abr. 2026.~~

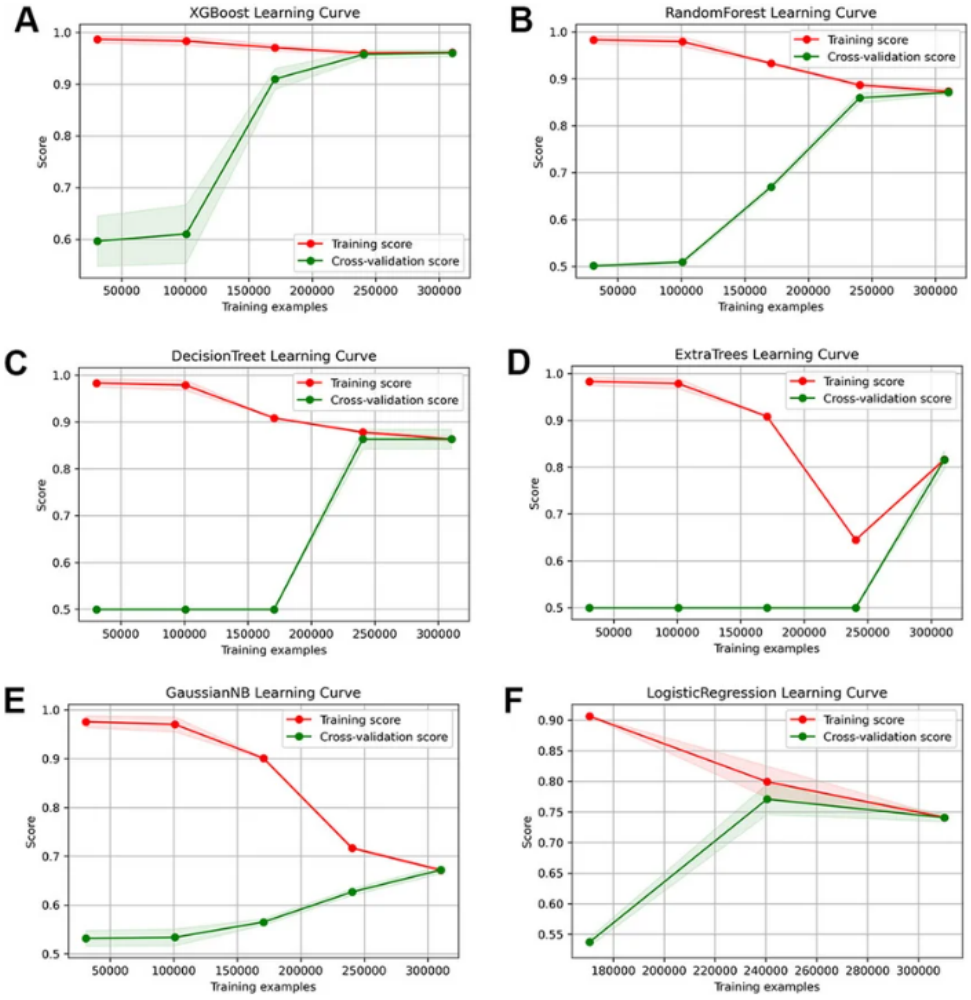



Figura 3.9: Exemplo de curvas de aprendizado (dados genéricos, não referentes a curvas de luz de ocultação) para ~~três seis~~ classificadores: (A) XGBoost, (B) Random Foreste , (C) ~~Árvore~~ árvore de ~~Decisão~~decisão, (D) *Extra Trees*, (E) *Naive Bayes* gaussiano e (F) Regressão Logística. Em cada painel, compara-se o desempenho no conjunto de treino com o obtido em validação cruzada, em função do número de amostras de treino~~; o vão entre as duas curvas indica o grau de sobreajuste, e o platô da curva de validação indica a partir de quando mais dados deixam de melhorar o modelo~~. Fonte: ~~adaptado de *RescarchGate*~~Zhong et al. (2023).

- **Validação cruzada ~~(k-fold):em~~ $q$ dobras** (em inglês, *k-fold cross-validation*): os dados são divididos em ~~$k$~~ $q$ partes (dobras); em cada rodada, ~~$k - 1$~~ $q - 1$ dobras são usadas para treino e uma para validação; a métrica final é a média sobre as ~~$k$~~ $q$ rodadas. Isso dá uma estimativa mais estável do desempenho e é útil para comparar modelos ou escolher hiperparâmetros (parâmetros do algoritmo que não são ajustados pelo treinamento, como a força da regularização).

- **Validação cruzada estratificada:** quando as classes são desbalanceadas (por exemplo, poucas curvas com ocultação e muitas sem), a divisão aleatória simples

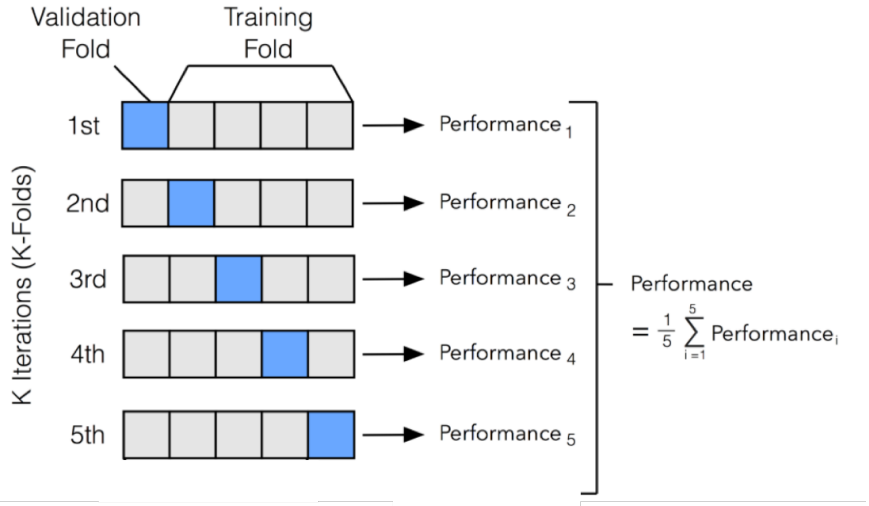



Figura 3.10: Esquema da validação cruzada com ~~$k = 5$~~$q = 5$ dobras: em cada rodada, uma fatia diferente (em destaque) é usada como conjunto de validação, e as demais como treino. Fonte: adaptado de Nishad (2023). Disponível em: `https://medium.com/@nishad009adi/k-fold-cross-validation-with-simple-example-e023bb2e2d43`. Acesso em: 6 abr. 2026.

pode produzir dobras em que uma das classes está sub ou sobrerrepresentada. A validação **estratificada** preserva a proporção de cada classe em cada dobra, tornando a estimativa mais confiável (Geron , 2019). Na detecção de ocultações, o uso de divisão estratificada é recomendado.

- **Busca em grade:** para escolher hiperparâmetros, avalia-se um conjunto de combinações (a ~~"grade"~~"grade") e escolhe-se a que maximiza a métrica de validação.

Na *pipeline* desta dissertação (Capítulo 4), a divisão treino/teste é feita de forma estratificada (mantendo a proporção de positivas e negativas em cada parte) ou por seleção manual de curvas para o teste. As métricas (acurácia, precisão, sensibilidade, F1-score, AUC-ROC) são calculadas no **conjunto de teste** — dados que o modelo nunca viu durante o treinamento.

Em resumo, o leitor dispõe de uma referência clara sobre o significado das "contas" envolvidas no treinamento de modelos de aprendizado de máquina. Inicialmente, apresentamos um modelo simples de Regressão Linear ajustado por meio do método dos mínimos quadrados e discutimos suas limitações. Em seguida, introduzimos a Regressão Logística (Seção 3.3) como um exemplo direto de otimização por gradiente descendente. Depois, abordamos os *ensembles* de árvores, que utilizam critérios de divisão para o aprendizado; no caso do XGBoost, o processo de otimização incorpora informações de segunda ordem da função de perda, resultando em atualizações mais eficientes (Geron , 2019). Por fim, esta seção reuniu as métricas de avaliação e os procedimentos de validação. Com isso, o

capítulo fornece a base conceitual necessária para interpretar a metodologia e os resultados descritos ~~no Capítulo 4.~~ nos Capítulos 4 e 5.

# Capítulo 4

# Metodologia: ~~Pipeline~~ *Pipeline* de Detecção

Este capítulo descreve a metodologia adotada para desenvolver o **fluxo de trabalho em etapas** (*pipeline*) de detecção automatizada de ocultações estelares em curvas de luz. A exposição está organizada em etapas sucessivas: desde a aquisição e o armazenamento dos dados até o **treinamento** (ajuste dos parâmetros dos classificadores) e a **avaliação** (medida do desempenho em dados não usados no ajuste).

Os objetivos deste capítulo são: (1) descrever a arquitetura completa da *pipeline*, da coleta dos dados ao modelo treinado; (2) detalhar o pré-processamento das curvas de luz, incluindo fontes de dados, armazenamento em banco relacional, normalização e critérios de classificação; (3) explicar a construção do conjunto de dados utilizado no treinamento, incluindo o recorte de segmentos negativos a partir de curvas positivas e a incorporação de curvas ~~sintéticas~~simuladas; (4) formalizar a engenharia de características (*feature engineering*) aplicada às curvas; e (5) especificar o protocolo de treinamento e avaliação dos classificadores, com as métricas e a divisão treino/teste adotadas. Ao final, o leitor terá uma visão clara de como os dados brutos são transformados em um conjunto de atributos e como os modelos são treinados e avaliados, preparando o terreno para a discussão dos resultados no Capítulo 5.

## 4.1 Visão geral da ~~pipeline~~*pipeline*

A metodologia está organizada em uma *pipeline* em etapas, na qual a saída de um bloco alimenta o seguinte. Essa estrutura facilita a manutenção, a reprodução e a extensão do trabalho. A Figura 4.1 ilustra o fluxo desde as fontes de dados até os modelos treinados.

**Etapa 1 Aquisição e armazenamento:** curvas de luz são obtidas a partir de portais externos (VizieR, banco do Grupo do Rio) e inseridas em um banco de dados relacional SQLite, com duas tabelas principais: `observations` (metadados da observação)

e `light_curves` (pontos tempo–fluxo).

**Etapa 2 Acesso e normalização:** um módulo de acesso aos dados consulta o banco, filtra curvas por objeto, data e tipo (positiva ou negativa), aplica normalização do fluxo ~~(baseline estimado por quartis)~~ pela linha de base (*baseline*, o nível de fluxo fora da ocultação), estimada por quartis, e remove *outliers* quando necessário.

**Etapa 3 Construção do conjunto de dados:** as curvas disponíveis são reunidas; segmentos sem ocultação são recortados de curvas positivas (com revisão manual) para ampliar a classe negativa; curvas ~~sintéticas negativas~~ negativas simuladas são incorporadas. Em seguida, cada curva é convertida em um vetor de características (lista de números que a descrevem), formando uma tabela em que cada linha é uma curva e cada coluna é uma dessas grandezas.

**Etapa 4 Extração de características:** para cada curva (série tempo–fluxo), ~~são calculadas~~ calculam-se grandezas numéricas que a resumem — estatísticas do fluxo, da série suavizada e de suas derivadas numéricas, e ~~medidas derivadas (suavização, derivadas,~~ comparações ~~entre quartis, etc.),~~ estatísticas entre quartis temporais —, gerando a tabela de vetores da etapa 3. O Capítulo 3 já introduziu a ideia de *features*; aqui detalhamos quais grandezas são extraídas.

**Etapa 5 Treinamento e avaliação:** o conjunto de dados final é dividido em **treino** (usado para ajustar os parâmetros dos classificadores) e **teste** (reservado para medir o desempenho). Os quatro classificadores são treinados na parte de treino; o desempenho é avaliado na parte de teste (acurácia, precisão, sensibilidade, F1-score, AUC-ROC). Os modelos ajustados e os resultados são salvos em disco para análise posterior.

As seções seguintes detalham cada uma dessas etapas.

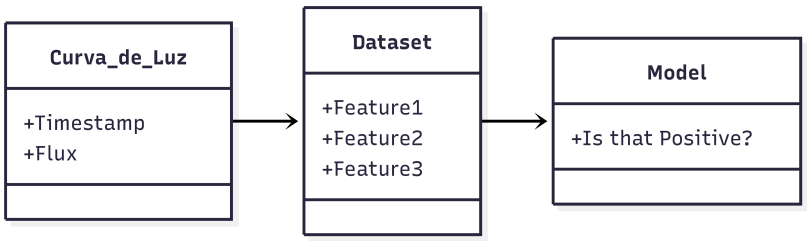



Figura 4.1: Esquema da *pipeline*: as curvas de luz são convertidas em vetores de características; o treinamento utiliza essas características, e não as séries temporais brutas.

## 4.2 Aquisição e armazenamento dos dados

### 4.2.1 Fontes de dados

As curvas de luz utilizadas neste trabalho provêm de duas origens principais.

**Grupo do Rio:** parte do conjunto foi obtida a partir do banco de dados em nuvem do Grupo do Rio, referente a campanhas de ocultações já analisadas na literatura: ocultação por Umbriel (satélite de Urano), observada em 2020 e analisada por Assafin et al. (2023), e ocultação por Chiron, observada em 2023 e analisada por Ortiz et al. (2023). Esses dados já possuem classificação (positiva ou negativa) e metadados consistentes.

**VizieR:** a maior parte das curvas foi adquirida no portal VizieR (*cdsarc.cds.unistra.fr*), catálogo `B/occ/asteroid` (Herald et al. , 2016), totalizando centenas de curvas em formato `.dat`, contendo séries temporais de tempo e fluxo. Para cada entrada do catálogo, os pontos da curva são baixados via serviço `vizgraph` e armazenados localmente. Convém esclarecer uma possível confusão de escalas: os milhares de registros mencionados na Introdução referem-se às **imagens (quadros)** que cada observação produz, das quais se extrai *uma* curva de luz por fotometria. O conjunto de dados deste trabalho é contado em **curvas**, não em quadros — são centenas de curvas, que, somadas ~~aos negativos por recorte e~~ às curvas ~~sintéticas~~negativas (cuja obtenção é descrita na Seção 4.4), totalizam as 1693 amostras detalhadas na Seção 5.4 (Capítulo 5).

O processamento e a integração dos dados foram realizados com as bibliotecas `Astropy` (Astropy Collaboration et al. , 2022) e `Astroquery` (Ginsburg et al. , 2019) (leitura e consulta a catálogos astronômicos), `asyncio` (biblioteca padrão do Python) e `aiohttp` (coleta assíncrona, reduzindo o tempo total de aquisição), e `Pandas` (McKinney , 2010) (manipulação tabular). ~~Scripts~~ As rotinas numéricas de suavização e de testes estatísticos (Seção 4.5) usam a biblioteca SciPy (Virtanen et al. , 2020). *Scripts* em Python foram desenvolvidos para automatizar a busca, o ~~download~~ *download* e a organização dos arquivos.

Vale observar nos fluxogramas (Figuras 4.2 e 4.3) um detalhe do procedimento: as curvas do VizieR foram baixadas e **triadas localmente** — com pré-visualizações (*previews*) em `.png` e arquivos `.dat` organizados em pastas — antes de um *script* de importação inseri-las no banco. Essa triagem prévia agilizou a inspeção de centenas de curvas que poderiam não ser positivas, estar em formato divergente ou sequer ser curvas de luz (por exemplo, curvas de rotação). Como efeito colateral, essas curvas foram registradas com origem *local*, de modo que nenhuma observação consta explicitamente como `VizieR` no campo de origem — um detalhe de proveniência corrigível por uma simples consulta SQL e sem efeito sobre os rótulos positivo/negativo. Registra-se, ainda, que as curvas marcadas como defeituosas no *download* foram descartadas sem caracterização do defeito nem tentativa de reaproveitamento (decisão de escopo e tempo que constitui uma fragilidade reconhecida da metodologia).

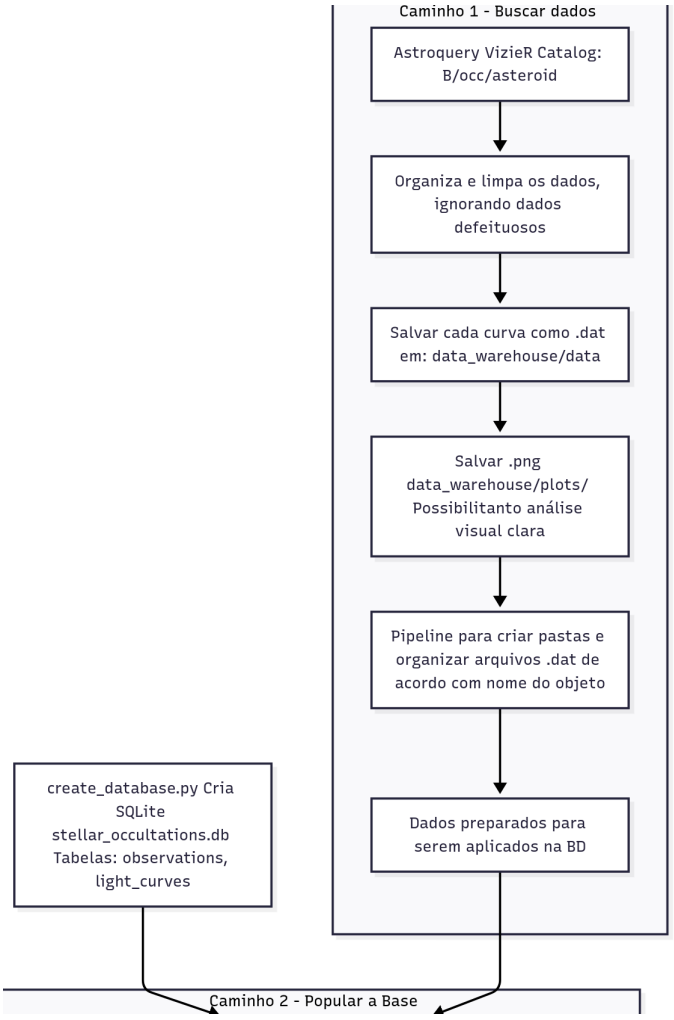



Figura 4.2: Fluxograma da aquisição e organização dos dados externos e da integração na etapa de inserção no banco de dados.

### 4.2.2 Estrutura do banco de dados

O banco de dados relacional foi implementado em SQLite (`.db`) e estruturado em duas tabelas principais, garantindo integridade referencial e consultas eficientes.

**Tabela `observations`:** armazena informações gerais de cada observação: identificador único, nome do objeto ocultador, data da observação, portal de origem, nome do observador e metadados adicionais (incluindo a classificação do evento como ocultação positiva ou negativa, quando disponível).

**Tabela `light_curves`:** contém os pontos individuais de cada curva de luz (tempo e fluxo), vinculados à observação correspondente por uma chave estrangeira (`observation_id`). Assim, uma mesma observação pode possuir muitos pontos, mantendo a normalização do esquema.

Durante a inserção, buscou-se preservar ao máximo os valores originais, realizando apenas ajustes mínimos de consistência: uniformização de unidades quando necessário, checagem do alinhamento temporal e validação de metadados, sem alterar as características intrínsecas das curvas. A Figura 4.3 resume o fluxo de consolidação do banco.

## 4.3 Acesso aos dados e normalização

Após a consolidação no banco, um módulo dedicado (*data access*) é responsável por consultar as curvas, filtrá-las por critérios (objeto, data, tipo positivo/negativo) e prepará-las para as etapas seguintes.

### 4.3.1 Consulta e classificação

As curvas são identificadas como **positivas** (com evento de ocultação) ou **negativas** (sem evento) com base em metadados armazenados na tabela `observations` (campo de metadados adicionais). Essa classificação pode provir de análises prévias (como no caso do Grupo do Rio) ou de anotações inseridas durante a organização dos dados. No Grupo do Rio, os rótulos decorrem das análises publicadas nas campanhas citadas (Umbriel, Chiron), realizadas por especialistas do grupo; no VizieR, a classificação positiva/negativa provém dos curadores do catálogo `B/occ/asteroid`. Não há estudo formal de concordância entre anotadores neste trabalho; as métricas de desempenho reportadas no Capítulo 5 são, portanto, relativas a esse *ground truth* e devem ser lidas com a ressalva de que erros ou inconsistências de rotulagem nos catálogos podem afetar tanto o treinamento quanto a avaliação dos modelos. Durante a organização dos dados não foram identificados casos confirmados de rótulo invertido (positiva rotulada como negativa ou vice-versa); contudo, como não se realizou uma reanálise independente de cada curva, não é possível descartar a existência de inconsistências pontuais herdadas dos catálogos de origem. O módulo de

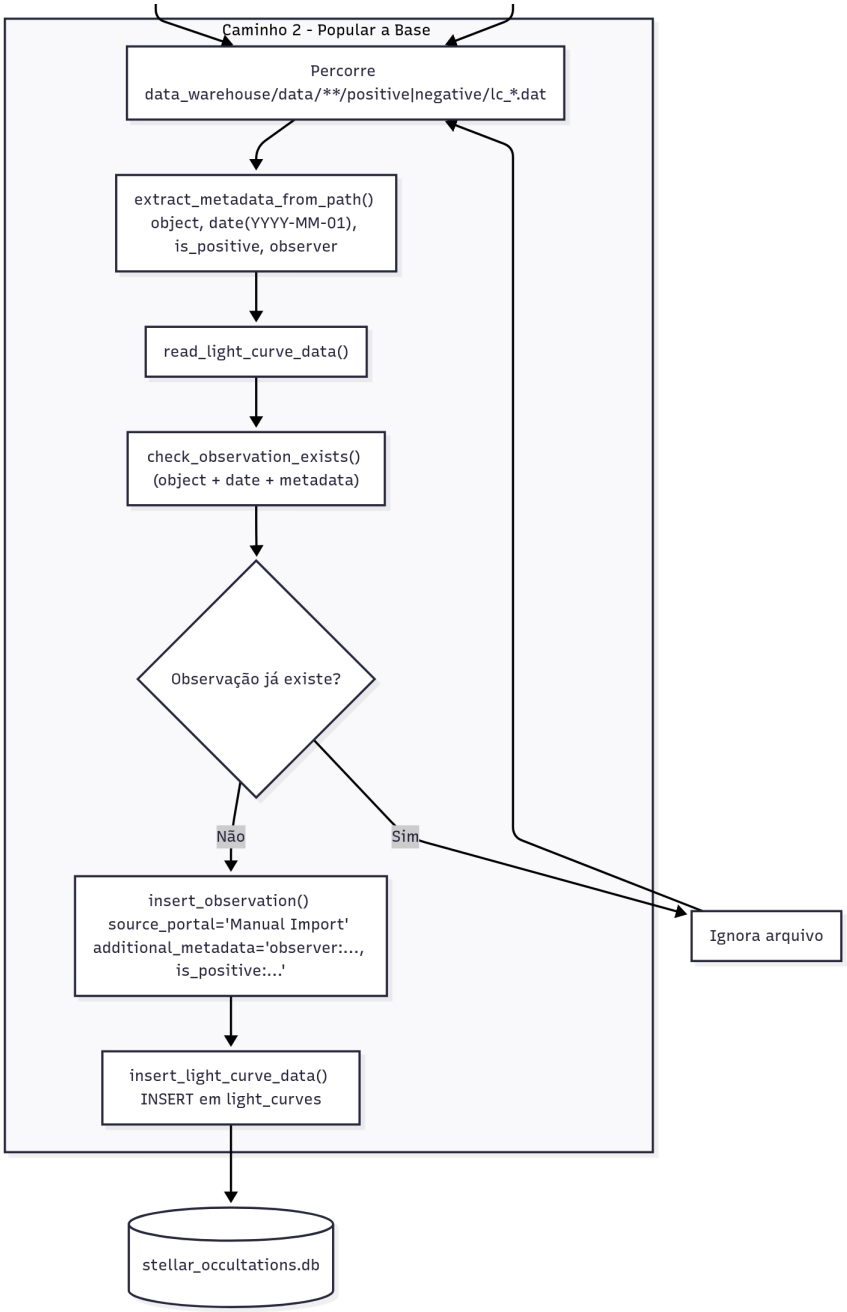



Figura 4.3: Fluxograma da *pipeline* responsável pela consolidação do banco de dados.

acesso permite listar objetos disponíveis, datas de observação por objeto e, para cada par objeto–data, recuperar as curvas com seus rótulos.

### 4.3.2 Normalização do fluxo

O fluxo fotométrico bruto varia em escala entre observações (diferentes telescópios, condições de céu, etc.). Para colocar todas as curvas em uma base comparável, aplica-se uma normalização que estima um *baseline* (nível de fluxo fora da ocultação) e divide-se a série pelo valor desse ~~baseline~~*baseline*.

A estratégia adotada é a seguinte: a série temporal do fluxo é dividida em quatro segmentos contíguos (quartis em ordem temporal). Calcula-se a média de cada segmento; os dois segmentos com maior média são considerados representativos do nível fora da ocultação. O *baseline* é definido como a média dessas duas médias. O fluxo inteiro é então dividido por esse ~~baseline~~*baseline*, de modo que, em curvas sem ocultação ou nos trechos externos à ocultação, o valor normalizado fique próximo de 1. Em curvas com ocultação, a região de queda aparece com valores inferiores a 1. Em casos degenerados (poucos pontos ou ~~baseline~~ *baseline* inválido), recorre-se à média global do fluxo como alternativa. A normalização coloca todas as curvas em uma escala comparável, facilitando a convergência dos algoritmos e a interpretação de resíduos; em modelos de regressão, a análise de resíduos é um diagnóstico fundamental para verificar a aderência do modelo aos dados (Morettin & Bussab , 2010). Esse procedimento é análogo a uma normalização "pós-fotometria", em que o nível de continuum já foi tratado.

### 4.3.3 Remoção de *outliers*

Quando necessário, pontos extremos do fluxo podem ser removidos com base em um critério de z-score: para um limiar $z_{\text{lim}}$ (por exemplo, 3), pontos cujo fluxo se afasta da média da série por mais de $z_{\text{lim}}$ desvios padrão são descartados. Isso reduz o impacto de picos ou quedas espúrias pontuais (defeitos de leitura, raios cósmicos, etc.) sem alterar a estrutura geral da curva.

## 4.4 Construção do conjunto de dados para aprendizado de máquina

O **conjunto de dados** (~~dataset~~*dataset*) é a tabela de exemplos (curvas de luz convertidas em vetores de características) com sua~~s~~classificação verdadeira (positiva ou negativa), usada para treinar e avaliar os classificadores. Devido à escassez de curvas negativas nos catálogos disponíveis, adotaram-se duas estratégias complementares: recorte de segmentos sem ocultação a partir de curvas positivas e geração de curvas ~~sintéticas negativas~~ negati-

vas simuladas com o simulador ~~físico de Gomes-Ferrante & Braga-Ribas (Gomes-Ferrante & Braga-Ribas , 2~~ de curvas de luz de ocultação de Gomes-Ferrante & Braga-Ribas (2023). Os detalhes de cada estratégia são descritos nas subseções seguintes.

### 4.4.1 Tamanho e composição do conjunto de dados

Para interpretar sobreajuste, significância das métricas e generalização no Capítulo 5, é essencial conhecer o tamanho e a composição do conjunto utilizado. A composição final (quantidade de curvas positivas, negativas do banco, negativos por recorte manual, curvas ~~sintéticass~~simuladas, total de amostras e proporção treino/teste) é apresentada na Seção 5.4 (Configuração experimental), onde os números dos experimentos são detalhados junto com a distribuição por fonte.

### 4.4.2 Curvas positivas e negativas do banco

As curvas classificadas como positivas são obtidas do banco via consultas que filtram observações com rótulo correspondente. As curvas negativas são obtidas da mesma forma. O número de curvas negativas "nativas" (observações realmente sem ocultação) costuma ser menor que o de positivas ~~em muitos catálogos~~no catálogo utilizado neste trabalho. Além disso, o catálogo obtido via VizieR, que constitui a principal fonte de dados deste trabalho, contém exclusivamente curvas positivas.

### 4.4.3 Recorte de segmentos negativos a partir de curvas positivas

Uma estratégia para obter exemplos negativos de alta qualidade consiste em extrair, das próprias curvas positivas, os trechos localizados **fora** da região de ocultação (isto é, anteriores à imersão e posteriores à emersão). Esses segmentos representam observações do mesmo sinal na ausência do evento e, portanto, constituem candidatos adequados para compor a classe negativa (denominados **negativos por recorte**). Contudo, por serem derivados da mesma curva de luz de ocultação, os recortes compartilham com ela a mesma estrela (brilho, cor e diâmetro aparente), o mesmo instrumento e as mesmas condições de imagem daquela noite (PSF, *seeing*, ~~compartilham características intrínsecas da observação, como instrumentação~~cadência, ~~condições observacionais e propriedades da estrela observada. Assim, para evitar vazamento de informação (*data leakage*) e reduzir o risco de viés nos modelos, a curva positiva que originou os recortes deve ser removida do conjunto de dados utilizado no treinamento e na avaliação~~cintilação e transparência).

O procedimento adotado foi o seguinte: ~~para~~ em cada curva positiva ~~simples, identifica-se~~ de uma amostra aleatória, a região de ocultação ~~de forma manual. Os segmentos antes e depois dessa~~ é delimitada automaticamente pelo primeiro e pelo último ponto com fluxo

normalizado inferior a 0,78, após a remoção de *outliers* a $3\sigma$; os segmentos anterior e posterior a essa região, com pelo menos 20 pontos cada, são ~~recortados. Quando isso ocorre~~exibidos e aceitos ou rejeitados manualmente. Se ao menos um segmento é aceito, a curva positiva original é excluída do conjunto de ~~positivos usados no treino, para evitar~~ positivas, para que a mesma observação não apareça ~~tanto como positiva quanto~~ como positiva e como negativa (vazamento de informação~~). Essa abordagem reduz o risco de viés e garante que os negativos por recorte correspondam de fato a trechos sem ocultação.~~ , ou *data leakage*).

### 4.4.4 Curvas ~~sintéticas~~ negativas simuladas

Para diversificar ainda mais a classe negativa e cobrir condições variadas de ruído e amostragem, foram incorporadas **curvas de luz ~~sintéticas~~simuladas** geradas por um simulador físico (Gomes-Ferrante & Braga-Ribas , 2023). O simulador combina: (i) a geometria do evento ~~(tempo,~~ o instante de cada exposição é convertido na posição do observador ao longo da corda~~);~~ , $x(t) = V_S\,(t - t_0)$, medida no plano do céu a partir do centro do evento; o corpo é representado por uma faixa opaca de largura igual ao seu diâmetro, o que equivale a uma corda central; (ii) a difração de Fresnel nas bordas de imersão e emersão, o diâmetro aparente da estrela e a integração durante a exposição; (iii) um modelo de ruído ~~(Poisson~~branco (Poisson da estrela e do céu, ruído de leitura ~~,~~ gaussiano e cintilação ~~atmosférica~~multiplicativa lognormal); (iv) a normalização pós-fotometria. Para a classe negativa, ~~são geradas curvas sem evento (ou com o~~ diâmetro ~~efetivo nulo)~~é nulo: a geometria e a difração não intervêm, e cada curva simulada consiste apenas na base de tempo (20 a 60 s de duração, com exposições de 0,~~submetidas ao mesmo tipo de~~ 05 a 0,25 s) e no ruído, com parâmetros sorteados uniformemente (estrela de magnitude 11 a 14,5; *seeing* de 0,8″ a 1,6″). Não foram incluídos ruído ~~e normalização das curvas reais~~correlacionado, variações de transparência nem nuvens. As curvas ~~sintéticas~~ simuladas são exportadas em formato `.dat` e carregadas pelo módulo de construção do ~~dataset~~*dataset*, sendo tratadas como negativas. Essa etapa permite aumentar o tamanho e a variedade do conjunto de treino e melhorar a generalização do modelo em condições diversas.

O conjunto final reúne, portanto: positivas do banco (excluindo as usadas para recorte), negativas do banco, negativos por recorte (segmentos recortados e aprovados) e curvas ~~sintéticas negativas~~ negativas simuladas. Cabe destacar que o catálogo `B/occ/asteroid` do VizieR, principal fonte das curvas positivas, é um agregador público de campanhas independentes envolvendo diferentes corpos, observadores, telescópios, datas e localizações geográficas; a diversidade observacional do conjunto positivo é, portanto, intrínseca à natureza do catálogo, e não há fonte observacional única que possa enviesar essa classe. Os pontos onde existe acoplamento entre amostras são explicitamente assumidos e mitigados:

(i) os negativos por recorte herdam telescópio, estrela e noite da curva positiva da qual foram recortados, motivo pelo qual a positiva original é excluída do conjunto de treino; (ii) as curvas ~~sintéticas~~simuladas, geradas por simulador físico, podem apresentar características de ruído distintas das observações reais, limitação retomada e quantificada no Experimento 3 (Capítulo 5). Todas as curvas passam pela mesma etapa de extração de características, descrita na próxima seção.

## 4.5 Engenharia de características

~~As curvas de luz são séries temporais (tempo × fluxo).~~ Os modelos de classificação utilizados neste trabalho (Regressão Logística, Random Forest, XGBoost, CatBoost) recebem **vetores numéricos**, e não as séries brutas~~. Por~~ ; por isso, cada curva é convertida ~~em um vetor de **características** (features) extraídas da série — por exemplo, amplitude, mínimo do fluxo suavizado, distância entre centróides do K-means, etc. O resultado é uma tabela em que cada linha é uma curva e cada coluna é uma dessas grandezas.~~ no vetor de características descrito nesta seção. As características foram padronizadas (média zero, desvio unitário) **apenas para a Regressão Logística**, via `StandardScaler` — o transformador da biblioteca `scikit-learn` (Pedregosa et al. , 2011) que, para cada variável, subtrai a média e divide pelo desvio padrão do conjunto de treino —, uma vez que a regularização $\ell_2$ é sensível à escala das variáveis (Geron , 2019). Os modelos baseados em árvores (Random Forest, XGBoost, CatBoost) recebem as *features* em escala original, pois são insensíveis a transformações monotônicas de cada variável.

Adiante veremos que algumas características extraídas carregarão informações redundantes. Isto é proposital, pois motivará uma demonstração sobre redução de dimensionalidade nos modelos finais.

### 4.5.1 ~~Estatísticas básicas~~ Amplitude e desvio padrão do fluxo

Seja $F(t)$ a série de fluxo (preferencialmente normalizado) com $N$ pontos. Definem-se:

$$\mu = \frac{1}{N}\sum_{i=1}^{N} F(t_i), \qquad \sigma = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}\left(F(t_i)-\mu\right)^2}, \tag{4.1}$$

$$F_{\min} = \min_{1\leqslant i\leqslant N} F(t_i), \qquad F_{\max} = \max_{1\leqslant i\leqslant N} F(t_i). \tag{4.2}$$

A amplitude da curva é $\mathrm{Amp} = F_{\max} - F_{\min}$. Em curvas com ocultação, a amplitude tende a ser maior (queda pronunciada); em curvas ~~apenas ruidosas,~~ negativas, o desvio padrão ($\sigma$) e a amplitude ~~podem~~ tendem a ser menores. O desvio padrão $\sigma$ e a amplitude são portanto candidatos naturais a *features* discriminativas.

### 4.5.2 Suavização ~~e médias móveis~~(filtro de Savitzky-Golay)

Para reduzir o ruído de alta frequência sem perder a forma geral da curva, aplica-se uma ~~janela de~~ suavização. ~~A suavização~~ Ela atua como um filtro passa-baixa ~~: atenua as flutuações —~~ um filtro que deixa passar as variações lentas (baixas frequências) e atenua as rápidas (altas frequências): as flutuações ponto a ponto~~(componentes de alta frequência, associadas ao~~ , dominadas pelo ruído~~) e preserva~~ , são atenuadas, enquanto as variações lentas~~(baixa frequência)~~, que descrevem a forma da ocultação~~. A janela~~ , são preservadas. Utiliza-se o filtro de Savitzky-Golay (Savitzky & Golay , 1964), implementado na função `savgol_filter` da biblioteca SciPy (Virtanen et al. , 2020): em cada janela móvel de $w$ pontos ajusta-se, por mínimos quadrados, um polinômio de grau 2, e o ponto central é ~~adaptativa ao tamanho da~~ substituído pelo valor do polinômio. Por isso o filtro preserva melhor a profundidade de picos e vales do que uma média móvel simples. A largura da janela depende do número $N$ de pontos da série: ~~utiliza-se $N/40$ para séries longas (ex.: $N > 120$ pontos) e $N/3$ para séries mais curtas, com~~ $w = \max(3, \lfloor N/3 \rfloor)$ para $N < 40$ e $w = \max(3, \lfloor N/40 \rfloor)$ para $N \geqslant 40$, somando-se 1 quando o resultado é par (o filtro simétrico exige $w$ ímpar). O valor mínimo de 3 ~~pontos e ímpar para filtros simétricos. Com essa janela, calculam-se:~~

~~Média móvel do fluxo; extraem-se o máximo e o mínimo da~~ refere-se à janela, e não à série~~suavizada. Filtro de Savitzky-Golay (polinômio de ordem 2), que preserva melhor os picos e vales; do fluxo suavizado por Savitzky-Golay~~ ; curvas com menos de 5 pontos são descartadas da extração de características. Do fluxo suavizado extraem-se máximo, mínimo e desvio padrão. A suavização é especialmente importante para as derivadas (subseção seguinte): a diferenciação numérica amplifica as componentes de alta frequência, nas quais domina o ruído, de modo que derivadas da série bruta refletiriam sobretudo o ruído, e não a forma da queda.

Essas medidas resumem o comportamento "suave" da curva e são menos sensíveis a flutuações pontuais.

### 4.5.3 ~~Max drawdown~~Queda máxima acumulada (*max drawdown*)

O *max drawdown*, termo emprestado da análise de séries financeiras, é a maior queda do fluxo em relação ao máximo ~~acumulado até cada instante~~atingido *anteriormente* na série:

$$\text{drawdown}(i) = F(t_i) - \max_{1 \leqslant j \leqslant i} F(t_j), \qquad \text{Max_Drawdown} = \min_i \text{drawdown}(i). \tag{4.3}$$

A motivação é física: em uma ocultação, a queda ocorre a partir de um patamar alto (a estrela ainda visível). Ao contrário da amplitude, o *max drawdown* respeita a ordem

temporal — só contam quedas que vêm depois de um máximo, como uma imersão. Em curvas com ocultação, o ~~max drawdown~~ *max drawdown* tende a ser mais negativo; em curvas sem evento, permanece próximo de zero (apenas flutuações).

### 4.5.4 K-means no fluxo suavizado

Conforme definido na Seção 3.4 (Capítulo 3), o K-means é um algoritmo de agrupamento não supervisionado; nesta etapa, ele é empregado não como classificador, mas como **extrator de característica**. Aplica-se o K-means com $K = 2$ aos valores do fluxo suavizado (Savitzky-Golay), tratando cada ponto como um vetor unidimensional. A escolha de $K = 2$ é motivada pela física do problema: espera-se que uma curva com ocultação apresente dois regimes de fluxo (o nível fora do evento~~(~~, ou *baseline*~~)~~, e o nível ~~rebaixado durante a queda~~de fluxo menor durante o evento). A **distância entre os dois centróides** resultantes é então usada como *feature*: curvas com ocultação bem definida tendem a apresentar distâncias maiores, enquanto curvas negativas, cujas variações de fluxo se devem apenas ao ruído, apresentam distâncias menores. É importante notar que o K-means sempre encontra dois grupos, mesmo em ruído puro: para uma série gaussiana de desvio padrão $\sigma_s$ (o da série suavizada), os centróides ficam em $\pm\sigma_s\sqrt{2/\pi}$ em torno da média, e a distância ~~maior; curvas apenas ruidosas, distância menor. Em casos de~~ entre eles vale $\approx 1{,}6\,\sigma_s$. A característica só discrimina, portanto, quando a queda é algumas vezes maior que o ruído da série suavizada. Em quedas tênues, ~~essa~~ com profundidade comparável a $1{,}6\,\sigma_s$, a distância ~~pode ficar menor e se confundir com ruído, ou pode possuir distância grande o suficiente para se destacar. Esse tipo de questão~~ de uma curva positiva pode coincidir com a de uma negativa ruidosa; verificar se, mesmo assim, a característica contribui para a classificação é o ~~que motiva o objetivo dessa~~ objetivo desta extração~~: testar a utilidade do algoritmo no contexto de ocultações~~.

### 4.5.5 Derivadas

A partir do fluxo suavizado, calculam-se a primeira e a segunda derivada (numericamente, e.g. via `np.gradient`). Uma ocultação típica produz picos negativos na primeira derivada (queda) e inflexões na segunda. São extraídas estatísticas dessas derivadas: mínimo, máximo, média, desvio padrão, assimetria (*skewness*, que mede o quanto a distribuição se afasta da simetria em torno da média) e curtose ~~da~~ (*kurtosis*, que mede o peso das caudas da distribuição em relação ao da gaussiana; a implementação usa o excesso de curtose, nulo para uma gaussiana) da primeira derivada; mínimo, máximo e desvio padrão da segunda derivada. Essas *features* capturam a "forma" da transição de fluxo e ajudam a distinguir quedas reais de quedas espúrias ou ruidosas.

### 4.5.6 Comparação entre quartis (testes estatísticos)

A série é dividida em quatro quartis temporais. Para cada par de quartis, aplicam-se ~~um~~ o teste $t$ de Welch ~~(diferença de~~ (Welch , 1947), que compara as médias ~~) e um~~ sem supor variâncias iguais, e o teste de Kolmogorov-Smirnov (K-S) para duas amostras (Kolmogorov , 1933; Smirnov , 1948), que compara as funções de distribuição acumulada~~das duas amostras e fornece~~ ; ambos estão implementados no SciPy (`ttest_ind` e `ks_2samp`) (Virtanen et al. , 2020) e fornecem um $p$-valor sob a hipótese de que as ~~distribuições são iguais~~ duas amostras vêm da mesma população (Wall & Jenkins , 2003; Morettin & Bussab , 2010). Em curvas com ocultação, pelo menos um quartil (o que contém a queda) tende a diferir dos demais; os $p$-valores dos testes tendem a ser pequenos. Define-se então a *feature* como o menor $p$-valor entre todos os pares, em escala logarítmica: $-\log_{10}(p_{\min} + \epsilon)$, para evitar $\log(0)$. Valores altos indicam forte evidência de diferença entre segmentos, compatível com a presença de um evento.

### 4.5.7 Métricas de profundidade, duração e ajuste de modelos

Além das *features* descritas até aqui, a *pipeline* incorpora um conjunto de métricas inspiradas em critérios observacionais discutidos em fóruns entre astrônomos amadores para análise de curvas difíceis de ocultação. Não há documento oficial que defina essas grandezas como critério definitivo para classificar uma curva como positiva; no texto referimo-nos a elas simplesmente como **métricas de profundidade, duração e ajuste de modelos**, isto é, grandezas adotadas neste trabalho e inspiradas em critérios práticos de análise. A implementação concreta segue o módulo de engenharia de características do projeto (arquivo `occ_features.py`), cujas fórmulas são resumidas abaixo.

**~~Baseline~~*Baseline*.** O nível de fluxo fora da ocultação é estimado pela mediana da série: baseline = mediana($F$). Em implementações alternativas pode-se usar a média dos pontos com fluxo acima da mediana. A mediana é robusta a *outliers* — valores extremos isolados, como quedas ou picos pontuais causados por raios cósmicos ou defeitos de leitura —, que pouco a deslocam, ao contrário do que ocorreria com a média.

**Profundidade ~~do dip~~da queda (*dip*).** A queda do fluxo em relação ao ~~baseline~~ *baseline* é

$$\text{depth} = \max\big(0,\ \text{baseline} - \min_i F(t_i)\big). \tag{4.4}$$

Em curvas sem ocultação, depth $\approx 0$; em curvas com evento, depth reflete a magnitude da queda.

**Desvio do ~~baseline~~*baseline*.** O desvio padrão $\sigma_{\text{baseline}}$ é calculado apenas sobre os pontos com fluxo $\geqslant$ baseline (fora do ~~dip~~*dip*). Essa grandeza estima o **ruído fotométrico fora do evento**: a dispersão natural do fluxo medido quando não há ocultação, decorrente das incertezas de medição (ruído de fótons, de leitura e de cintilação atmosférica). Usá-la

como referência evita contaminar a medida de ruído com a própria queda. Se houver poucos pontos acima do ~~baseline~~*baseline*, usa-se o desvio padrão de toda a série.

**SNR do ~~dip~~*dip*.** Neste trabalho, define-se a razão sinal-ruído da queda como o quociente entre a profundidade do *dip* e o ruído estimado fora do evento:

$$\text{SNR_dip} = \frac{\text{depth}}{\sigma_{\text{baseline}}}. \tag{4.5}$$

Valores altos indicam queda significativa em relação ao ruído, compatível com evento real.

**Sequências abaixo do ~~baseline~~*baseline*.** Uma *sequência* (*run*) é um bloco consecutivo de índices em que $F(t_i) <$ baseline. Ocultações típicas produzem uma sequência longa; ruído tende a sequências curtas ou isoladas. Define-se o **número de ~~frames~~ *frames* na maior sequência** como a quantidade de pontos do maior bloco consecutivo abaixo do ~~baseline~~*baseline*.

**Duração em segundos.** A duração da maior sequência é $\Delta t = t_{\text{fim}} - t_{\text{incio}}$, onde $t_{\text{incio}}$ e $t_{\text{fim}}$ são os instantes do primeiro e do último ponto dessa sequência. Essa grandeza está ligada à geometria do evento (tamanho do objeto, velocidade da sombra, posição da corda, etc.).

**Fluxo mínimo e razão.** Sejam $F_{\min} = \min_i F(t_i)$ e a razão $F_{\min}/\text{baseline}$ (quando baseline $> 0$). Em curvas normalizadas com ocultação, a razão tende a ser menor que 1.

**Chiš do modelo constante.** O modelo "sem evento" assume fluxo constante igual ao ~~baseline~~*baseline*. Define-se

$$\chi^2_{\text{const}} = \sum_i \frac{(F_i-\mu)^2}{\sigma^2} \frac{(F_i - \mu)^2}{s^2_{\text{MAD}}}, \tag{4.6}$$

com $\mu =$ baseline e ~~$\sigma$~~ $s_{\text{MAD}}$ uma escala robusta dos resíduos (por exemplo, MAD — mediana do valor absoluto dos desvios em relação à mediana).

**Chiš do modelo "square well".** O modelo com ~~dip~~ *dip* assume nível constante fora da janela da maior sequência e nível constante (média do fluxo) dentro da janela, em formato de "poço retangular". ~~A predição pred$_i$ vale baseline fora da janela e~~ Esse modelo é a versão mais simples do modelo geométrico da Figura 2.3 (curva verde): bordas nítidas, sem difração nem integração temporal; difere dele por ajustar o nível interno (a média ~~do fluxo na janela~~ dos pontos da janela) em vez de fixá-lo, o que acomoda a luz residual do corpo ocultante. No modelo de poço retangular, o valor previsto para cada ponto, $\hat{F}_i$, é igual ao *baseline* fora da janela e à média dos fluxos dentro dela. Define-se

$$\chi^2_{\text{sw}} = \sum_i \frac{(F_i-\text{pred}_i)^2}{\sigma^2} \frac{(F_i - \hat{F}_i)^2}{s^2_{\text{MAD}}}. \tag{4.7}$$

**Razão chiš.** A grandeza

$$\chi^2_{\text{ratio}} = \frac{\chi^2_{\text{const}}}{\chi^2_{\text{sw}}} \tag{4.8}$$

mede o ganho de ajuste ao incluir o ~~dip~~*dip*: valores $\chi^2_{\text{ratio}} > 1$ indicam que o modelo com ~~dip~~ *dip* explica melhor os dados. Na implementação, a razão é limitada ao intervalo [0,1, 10] para evitar valores extremos. Curvas sem ~~dip~~ *dip* tendem a $\chi^2_{\text{ratio}} \approx 1$.

No ~~dataset~~ *dataset* gerado pela *pipeline*, essas grandezas aparecem em colunas cujos nomes no código possuem o prefixo `Occ_` (de *occultation*), por exemplo `Occ_depth`, `Occ_duration_s`. No texto, tratamos todas elas como métricas de profundidade, duração e ajuste de modelos.

### 4.5.8 Resumo das *features*

O vetor final de *features* inclui: amplitude e desvio padrão do fluxo; máximo~~e~~, mínimo ~~da média móvel e do~~ e desvio padrão do fluxo suavizado por Savitzky-Golay; ~~max drawdown~~*max drawdown*; distância entre centróides do K-means no fluxo; mínimos $-\log_{10}(p)$ dos testes $t$ e K-S entre quartis; as estatísticas das derivadas primeira e segunda listadas acima; e as métricas de profundidade, duração e ajuste de modelos (profundidade do ~~dip~~*dip*, SNR do ~~dip~~*dip*, duração em segundos da maior sequência abaixo do ~~baseline~~*baseline*, número de ~~frames~~ *frames* na maior sequência, desvio padrão do ~~baseline~~*baseline*, fluxo mínimo, razão fluxo mínimo/~~baseline~~*baseline*, $\chi^2$ do modelo constante, $\chi^2$ do modelo square well, razão $\chi^2_{\text{const}}/\chi^2_{\text{sw}}$). Todas as colunas de *features* são numéricas; metadados como identificador da curva e rótulo (positivo/negativo) são mantidos em colunas separadas e não são usados como entrada do classificador. O resultado da etapa de engenharia de características é um arquivo tabular (por exemplo, `dataset_final.csv`) em que cada linha é uma curva e as colunas são as *features* mais o rótulo e o identificador.

## 4.6 Treinamento e avaliação dos modelos

### 4.6.1 Preparação dos dados e divisão treino/teste

O ~~dataset~~ *dataset* final é carregado a partir do arquivo gerado na etapa anterior. As colunas de metadados (identificador da curva, fonte, rótulo) são separadas das colunas de *features*. Valores faltantes (NaN) são preenchidos pela **mediana** calculada no conjunto de treino (`SimpleImputer` com `strategy='median'`), estratégia robusta a *outliers*. O rótulo binário (ocultação presente = 1, ausente = 0) é definido como variável alvo.

A divisão entre treino e teste pode ser feita de duas formas:

1. **Divisão estratificada:** os dados são divididos aleatoriamente em conjunto de

treino e conjunto de teste (por exemplo, 80% treino e 20% teste), mantendo a proporção das classes em cada parte (*stratified split*). Uma semente fixa (*random state*) é utilizada na divisão para garantir reprodutibilidade; a mesma semente (ou sementes documentadas) é empregada no treinamento de cada modelo (Regressão Logística, Random Forest, XGBoost, CatBoost), de modo que os experimentos sejam reproduzíveis.

2. **Seleção manual de teste:** um subconjunto de curvas é fixado como conjunto de teste (por exemplo, curvas de um objeto ou data específicos). O restante compõe o treino. Essa opção é útil para avaliar a generalização para cenários de interesse conhecido.

As métricas de desempenho (acurácia, precisão, sensibilidade, F1-score, AUC-ROC) são calculadas no conjunto de teste e reportadas no Capítulo 5. Quando a escolha de hiperparâmetros for feita por validação cruzada, pode-se reportar também a média e o desvio padrão da métrica nas dobras de validação, para dar noção da variabilidade; no conjunto de teste, as métricas são reportadas como valor único, condicionado à divisão treino/teste adotada.

### 4.6.2 Modelos utilizados

Foram utilizados quatro algoritmos de classificação supervisionada, descritos no Capítulo 3:

- **Regressão Logística:** classificador linear probabilístico com regularização $\ell_2$ (Ridge) ou $\ell_1$ (Lasso); hiperparâmetro de força de regularização ($C$ ou $\alpha$) escolhido via validação cruzada.
- **Random Forest:** ~~ensemble~~ *ensemble* de árvores de decisão com *bagging*; hiperparâmetros típicos incluem número de árvores (~~*n_estimators*~~`n_estimators`), profundidade máxima (~~*max_depth*~~`max_depth`) e peso das classes (~~*class_weight='balanced'*~~`class_weight='balanced'`) para lidar com desbalanceamento.
- **XGBoost:** *gradient boosting* com árvores; controle de profundidade, taxa de aprendizado (~~*learning_rate*~~`learning_rate`), subamostragem de linhas e colunas.
- **CatBoost:** outro algoritmo de *boosting*, com opção de ponderação automática das classes (~~*auto_class_weights='Balanced'*~~`auto_class_weights='Balanced'`).

Os valores concretos dos hiperparâmetros utilizados nos experimentos (e eventual busca em grade ou validação cruzada) estão documentados no Apêndice B. A escolha desses modelos deve-se ao seu bom desempenho em problemas de classificação com dados

tabulares, à interpretabilidade (importância de variáveis) e à disponibilidade em bibliotecas estáveis (scikit-learn, XGBoost, CatBoost). Para quantificar a interpretabilidade, a **importância das variáveis** é calculada da seguinte forma: nos ~~ensembles~~ *ensembles* baseados em árvores (Random Forest, XGBoost, CatBoost), utiliza-se a importância por **queda média na impureza** (Gini) em cada divisão em que a variável participa, conforme fornecido pelas respectivas bibliotecas; na Regressão Logística, utilizam-se os coeficientes estimados (em valor absoluto). Os rankings e as magnitudes das importâncias são apresentados e discutidos no Capítulo 5.

### 4.6.3 Métricas de avaliação

As definições formais das métricas de classificação binária utilizadas neste trabalho — acurácia, precisão, sensibilidade, F1-score, $F_\beta$-score, AUC-ROC e curva ~~precision-recall~~ *precision-recall* — foram apresentadas na Seção 3.6.2 (Capítulo 3). Aqui, discutem-se as escolhas específicas adotadas na *pipeline* à luz do problema de detecção de ocultações.

Em problemas de detecção de eventos, a matriz de confusão e as métricas dela derivadas permitem avaliar a capacidade do modelo de tomar decisões confiáveis (Cazeneuve et al. , 2023). A **curva ROC** visualiza o compromisso (*trade-off*) entre a taxa de verdadeiros positivos (sensibilidade) e a taxa de falsos positivos à medida que o limiar de decisão varia; a **área sob a curva** (AUC-ROC) resume a capacidade de ordenação do modelo de forma independente do limiar escolhido (Sudjianto & Liu , 2025). Em problemas com **classe positiva rara** — isto é, quando os exemplos de interesse (positivos) são minoria expressiva frente aos negativos —, a **curva** ~~**precision-recall**~~***precision-recall*** pode ser mais informativa, pois não é inflada pelo grande número de verdadeiros negativos.

No contexto desta dissertação, em que os dados são desbalanceados (muitas curvas sem ocultação), a **acurácia** sozinha pode ser enganosa, conforme ilustrado pelo exemplo numérico do Capítulo 3. Para detecção de ocultações estelares, a **sensibilidade** tem prioridade, pois não perder eventos raros é essencial; a **precisão** evita excesso de falsos positivos que sobrecarregariam a triagem humana. O **F1-score** equilibra precisão e sensibilidade e é adequado para comparar os quatro modelos em cenário desbalanceado. A **AUC-ROC** avalia a capacidade de separação das classes de forma global.

### 4.6.4 Fluxo de treinamento e persistência

O fluxo de treinamento é: (1) carregar o conjunto de dados; (2) separar as colunas de características das colunas de rótulo (positivo/negativo); (3) dividir em treino e teste (por exemplo, 80% treino, 20% teste); (4) treinar cada um dos quatro modelos no conjunto de treino; (5) avaliar cada modelo no conjunto de teste, calculando as métricas acima; (6) gerar visualizações (curvas ROC, matrizes de confusão, importância de cada variável); (7) salvar os modelos ajustados e um resumo das métricas em disco. Os modelos salvos

podem ser carregados depois para classificar novas curvas ou para comparação com outras configurações. Os resultados numéricos e as figuras são a base da discussão do Capítulo 5. O ambiente de software (versões do Python e das bibliotecas) necessário para reproduzir esse fluxo está documentado no Apêndice C.

Este capítulo apresentou a metodologia completa da *pipeline* de detecção de ocultações estelares em curvas de luz: desde a aquisição e o armazenamento em banco relacional até o treinamento e a avaliação de classificadores. A arquitetura em etapas (aquisição → acesso e normalização → construção do ~~dataset~~ *dataset* → engenharia de características → treinamento) permite reproduzir o fluxo e estendê-lo com novas fontes de dados, novas *features* ou novos modelos. A combinação de dados reais (VizieR, Grupo do Rio), negativos por recorte (obtidos manualmente) e curvas ~~sintéticas~~ simuladas visa um conjunto de treino equilibrado e representativo. A extração de características baseada em estatísticas, suavização, derivadas e testes entre quartis fornece uma representação vetorial adequada aos modelos de árvore e à Regressão Logística utilizados. No próximo capítulo, são apresentados os resultados obtidos com os quatro modelos treinados (Regressão Logística, Random Forest, XGBoost, CatBoost), incluindo comparação de desempenho, análise de importância das variáveis e discussão sobre generalização e possível ~~overfitting~~ *overfitting*.

# Capítulo 5

# Resultados

## 5.1 Visão geral dos experimentos

Este capítulo apresenta os resultados obtidos com a *pipeline* de detecção automatizada de ocultações estelares em curvas de luz, descrita no Capítulo 4. O problema é formulado como **classificação binária** em que foram treinados e avaliados quatro classificadores — Regressão Logística, Random Forest, XGBoost e CatBoost — cujo desempenho foi medido exclusivamente no **conjunto de teste**, isto é, em dados que não participaram do treinamento.

As métricas adotadas são: **acurácia** (fração de acertos no total), **precisão** (entre as curvas classificadas como positivas, quantas eram realmente positivas), **sensibilidade** ou *recall* (entre todas as curvas realmente positivas, quantas foram detectadas), **F1-score** (média harmônica entre precisão e sensibilidade) e **área sob a curva ROC** (AUC-ROC), que resume a capacidade do modelo de separar as duas classes para diversos limiares de decisão. As **matrizes de confusão** mostram quantos verdadeiros positivos (TP), verdadeiros negativos (TN), falsos positivos (FP) e falsos negativos (FN) cada modelo produziu.

A montagem de um conjunto de dados equilibrado foi a primeira grande dificuldade do projeto: nos catálogos disponíveis (VizieR, Grupo do Rio), o número de curvas negativas é muito inferior ao de positivas — o banco dispunha de 802 curvas positivas e apenas 3 negativas. Para contornar esse desbalanceamento, adotaram-se duas estratégias complementares, descritas no Capítulo 4: recorte de segmentos sem ocultação a partir das curvas positivas (com revisão manual), gerando 186 negativos por recorte, e incorporação de 702 curvas ~~sintéticas negativas~~ negativas simuladas geradas pelo simulador físico de ~~Gomes-Ferrante & Braga-Ribas (Gomes-Ferrante & Braga-Ribas , 2023)~~ Gomes-Ferrante & Braga-Ribas (2023). O *dataset* final reúne **1693 amostras** (802 positivas e 891 negativas), com divisão treino/teste estratificada por curva (80%/20%).

Ao todo, foram realizados **seis experimentos**, que variam o conjunto de *features* (de

28 a 11 variáveis) e a composição do conjunto de teste (misto ou exclusivamente real). Como o desempenho se mostrou notavelmente estável entre eles, este capítulo apresenta **em detalhe os três experimentos mais informativos**, que sintetizam a progressão lógica do trabalho:

- **Experimento 1:** conjunto completo de 28 *features* (Tabela 5.1), divisão 80%/20%. É a **linha de base** de desempenho, com todas as variáveis disponíveis.
- **Experimento 2:** 11 *features* (sem `Feature_Savgol_Min` e sem `kmeans_centroid_dist`), divisão 80%/20%. É a **configuração enxuta** adotada como referência operacional — a mais simples sem perda de desempenho, e a usada no estudo de caso de Quaoar.
- **Experimento 3:** 11 *features*, porém com **teste somente em curvas reais** (as ~~sintéticas~~ simuladas permanecem no treino). É o teste de **generalização** mais próximo do uso real.

Os demais três experimentos — a redução intermediária de *features* para 14 (Experimento 4) e 13 variáveis (Experimento 5) e a ablação isolada da *feature* `kmeans_centroid_dist` (Experimento 6) — conduzem às mesmas conclusões e estão documentados em detalhe no Apêndice A, que reúne ainda uma variante do Experimento 3 com divisão 65%/35%, usada como ablação do tamanho do conjunto de treino. Suas métricas são consolidadas, ao lado das dos três experimentos-destaque, na Tabela 5.10 (Seção 5.10).

Em todos os experimentos, o mesmo protocolo foi aplicado: divisão treino/teste estratificada, preenchimento de valores faltantes pela mediana calculada no treino, e padronização das características (média zero, desvio unitário) apenas para a Regressão Logística, conforme detalhado no Capítulo 4.

Além dos experimentos de classificação, este capítulo apresenta uma análise de **ajuste do limiar de decisão** (Seção 5.8), que investiga como reduzir o limiar de classificação para minimizar falsos negativos — uma demanda natural em cenários de triagem científica, onde perder uma ocultação real é mais custoso do que revisar um falso alarme. Por fim, um **estudo de caso** (Seção 5.9) aplica a *pipeline* a curvas reais de uma ocultação estelar pelo objeto transnetuniano (50000) Quaoar ~~(Pereira et al. , 2023)~~ (Pereira et al. , 2023; Morgado et al. , 2023) — dados externos ao banco de treino e morfologicamente complexos (corpo principal e estruturas de anel) — como teste de generalização operacional e como demonstração da capacidade de revisitar curvas em busca de eventos sutis.

## 5.2 Descrição das *features* e análise de redundância

### 5.2.1 Conjunto completo de *features*

A *pipeline* de extração de características produz 28 variáveis numéricas a partir de cada curva de luz normalizada. Essas variáveis podem ser agrupadas em cinco famílias, descritas na Tabela 5.1.

Nas definições acima, $f$–$F$ denota o vetor de fluxo normalizado , $\tilde{f}$ (como no Capítulo 4), $\tilde{F}$ o fluxo suavizado por filtro Savitzky-Golay (polinômio de grau 2, janela adaptativa), $f'$ e $f''$ $F'$ e $F''$ a primeira e segunda derivadas numéricas de $\tilde{f}\tilde{F}$, e *baseline* a mediana de $f$$F$.

### 5.2.2 Análise de redundância

A presença de variáveis redundantes — altamente correlacionadas ou conceitualmente equivalentes — pode inflar a dimensionalidade sem acrescentar informação discriminativa, aumentar o risco de *overfitting* e dificultar a interpretação da importância das *features*. Para identificar pares redundantes, foram analisadas tanto as definições matemáticas quanto as correlações empíricas entre as 28 variáveis. Os pares identificados são apresentados a seguir.

1. **`Feature_Amp` vs. `Occ_depth`.** A amplitude é definida como $\max(f) - \min(f)$$\max(F) - \min(F)$, enquanto a profundidade observacional é baseline − $\min(f)$, com baseline = mediana($f$ $\min(F)$, com baseline = $\text{mediana}(F)$. Para curvas de fluxo normalizado próximo de 1, o máximo tende a coincidir com a mediana, tornando as duas métricas praticamente idênticas. **Removida:** `Feature_Amp`. **Mantida:** `Occ_depth`, por possuir motivação física direta na caracterização de ocultações.

2. **`Feature_Flux_std` vs. `Occ_baseline_std`.** O desvio padrão do fluxo total ($\sigma(f)$$\sigma(F)$) inclui os pontos dentro do dip*dip*, inflando a medida de ruído quando há ocultação. O desvio padrão do *baseline* ($\sigma(f \mid f \geqslant \text{mediana})$$\sigma(F \mid F \geqslant \text{mediana})$) estima o ruído fotométrico fora do evento, sendo mais adequado fisicamente. **Removida:** `Feature_Flux_std`. **Mantida:** `Occ_baseline_std`.

3. **`Occ_flux_min` vs. `Feature_Savgol_Min`.** O mínimo do fluxo bruto e o mínimo do fluxo suavizado por Savitzky-Golay são altamente correlacionados, pois o filtro preserva a posição dos mínimos. A versão suavizada é mais robusta a ruído pontual. **Removida:** `Occ_flux_min`. **Mantida:** `Feature_Savgol_Min`.

4. **`Occ_flux_min_over_baseline`.** Esta razão ($\min(f)/\text{baseline}$$\min(F)/\text{baseline}$) é algebricamente derivável de `Occ_depth` e do valor do *baseline*, não acrescentando informação independente. **Removida.**

Tabela 5.1: Conjunto completo de 28 *features* extraídas pela *pipeline*. A coluna "Família" indica o grupo funcional; a coluna "Definição" resume o cálculo.

| Família | ~~Feature~~*Feature* | Definição |
|---|---|---|
| Fluxo bruto | `Feature_Amp` | ~~$\max(f) - \min(f)$~~ $\max(F) - \min(F)$ (amplitude total do fluxo) |
| | `Feature_Flux_std` | ~~$\sigma(f)$~~ $\sigma(F)$ (desvio padrão do fluxo) |
| | `Max_Drawdown` | ~~$\min(f_i - \max_{j\leqslant i} f_j)$~~ $\min(F_i - \max_{j\leqslant i} F_j)$ (maior queda acumulada) |
| Savitzky-Golay | `Feature_Savgol_Max` | ~~$\max(\tilde{f})$~~ $\max(\tilde{F})$ (máximo do fluxo suavizado) |
| | `Feature_Savgol_Min` | ~~$\min(\tilde{f})$~~ $\min(\tilde{F})$ (mínimo do fluxo suavizado) |
| | `Feature_Savgol_std` | ~~$\sigma(\tilde{f})$~~ $\sigma(\tilde{F})$ (desvio padrão do fluxo suavizado) |
| ~~K-Means~~ K-means | `kmeans_centroid_dist` | Distância entre os 2 centróides do ~~K-Means aplicado a $\tilde{f}$~~ K-means aplicado a $\tilde{F}$ |
| Testes de quartis | `MinLogP_Ttest` | $-\log_{10}(\min p_{\text{t-test}})$ entre pares de quartis |
| | `MinLogP_KS` | $-\log_{10}(\min p_{\text{KS}})$ entre pares de quartis |
| Derivadas | `Deriv_Min` | ~~$\min(f')$~~ $\min(F')$ |
| | `Deriv_Max` | ~~$\max(f')$~~ $\max(F')$ |
| | `Deriv_Mean` | ~~$\bar{f}'$~~ $\bar{F}'$ |
| | `Deriv_Std` | ~~$\sigma(f')$~~ $\sigma(F')$ |
| | `Deriv_Skew` | Assimetria de ~~$f'$~~ $F'$ |
| | `Deriv_Kurtosis` | Curtose de ~~$f'$~~ $F'$ |
| | `SecondDeriv_Min` | ~~$\min(f'')$~~ $\min(F'')$ |
| | `SecondDeriv_Max` | ~~$\max(f'')$~~ $\max(F'')$ |
| | `SecondDeriv_Std` | ~~$\sigma(f'')$~~ $\sigma(F'')$ |
| Observacionais | `Occ_depth` | ~~baseline $-$ $\min(f)$~~ baseline $-$ $\min(F)$ (profundidade do ~~dip~~*dip*) |
| | `Occ_SNR_dip` | depth/$\sigma_{\text{baseline}}$ (razão sinal-ruído do ~~dip~~*dip*) |
| | `Occ_duration_s` | Duração do maior bloco abaixo do ~~baseline~~ *baseline* (em segundos) |
| | `Occ_n_frames_below_baseline` | Maior nž de ~~frames~~ *frames* consecutivos abaixo do ~~baseline~~ *baseline* |
| | `Occ_baseline_std` | ~~$\sigma(f \mid f \geqslant \text{baseline})$~~ $\sigma(F \mid F \geqslant \text{baseline})$ (ruído fora do ~~dip~~*dip*) |
| | `Occ_flux_min` | ~~$\min(f)$~~ $\min(F)$ |
| | `Occ_flux_min_over_baseline` | ~~$\min(f)/\text{baseline}$~~ $\min(F)/\text{baseline}$ |
| | `Occ_chi2_constant` | $\chi^2$ do modelo constante (sem evento) |
| | `Occ_chi2_square_well` | $\chi^2$ do modelo de poço retangular |
| | `Occ_chi2_ratio` | $\chi^2_{\text{const}}/\chi^2_{\text{sw}}$ (razão de ajuste) |

5. **`Occ_n_frames_below_baseline` vs. `Occ_duration_s`.** Para observações com cadência uniforme, a duração em segundos é proporcional ao número de ~~frames~~ *frames* ($\Delta t_{\text{dur}} \approx n_{\text{frames}} \times \delta t$). A duração em unidade física (segundos) é mais interpretável e invariante a mudanças de cadência. **Removido:** `Occ_n_frames_below_baseline`. **Mantida:** `Occ_duration_s`.

6. **`Feature_Savgol_Max`.** Para fluxo normalizado, o máximo do sinal suavizado é próximo de 1,0 em quase todas as curvas, resultando em variância muito baixa e poder discriminativo desprezível. **Removida.**

7. **Nove *features* de derivadas (`Deriv_*` e `SecondDeriv_*`).** No Experimento 4, a remoção das nove *features* de derivadas não causou perda de desempenho (F1 mantido em 0,99). Além disso, as métricas observacionais de profundidade, duração e razão $\chi^2$ já capturam as variações abruptas do fluxo que as derivadas buscam codificar, caracterizando redundância conceitual. **Removidas** todas as nove variáveis de derivadas.

Após a remoção, o conjunto final utilizado no treinamento contém **13 *features***, listadas na Tabela 5.2.

Tabela 5.2: Conjunto de 13 *features* após a análise de redundância. O conjunto de referência operacional (Experimento 2) remove ainda `Feature_Savgol_Min` e `kmeans_centroid_dist`, restando 11.

| **Família** | ~~**Features**~~ ***Features*** |
|---|---|
| Savitzky-Golay | `Feature_Savgol_Min`, `Feature_Savgol_std` |
| Fluxo bruto | `Max_Drawdown` |
| ~~K-Means~~ K-means | `kmeans_centroid_dist` |
| Testes de quartis | `MinLogP_Ttest`, `MinLogP_KS` |
| Observacionais | `Occ_depth`, `Occ_SNR_dip`, `Occ_duration_s`, `Occ_baseline_std`, `Occ_chi2_constant`, `Occ_chi2_square_well`, `Occ_chi2_ratio` |

É importante ressaltar que as 15 variáveis removidas **continuam sendo calculadas e armazenadas** no arquivo CSV do *dataset*. A exclusão ocorre apenas no momento do treinamento, por meio de uma lista configurável no código (`EXCLUDED_FEATURES`). Essa decisão de projeto permite re-incluir qualquer variável em estudos futuros de ablação sem necessidade de reprocessar as curvas de luz.

## 5.3 Importância das *features*: metodologia

Uma das vantagens de modelos baseados em árvores e de modelos lineares é a possibilidade de quantificar a contribuição individual de cada variável para a decisão do

classificador. Entretanto, cada família de modelos emprega uma definição distinta de "importância", e valores obtidos por métodos diferentes **não devem ser comparados diretamente em escala absoluta**. Esta seção descreve o cálculo adotado internamente por cada um dos quatro classificadores utilizados neste trabalho.

### 5.3.1 Random Forest — *Mean Decrease in Impurity* (MDI)

No caso do Random Forest, cada nó de cada árvore realiza uma divisão (*split*) em uma variável, e essa divisão reduz a impureza (medida pelo índice de Gini) dos subconjuntos resultantes. Para uma *feature* $x_j$, a importância MDI é definida como a **média**, sobre todas as árvores e todos os nós, da redução de impureza atribuída a $x_j$:

$$\mathrm{MDI}(x_j) \;=\; \frac{1}{T}\sum_{t=1}^{T}\sum_{n\in\mathcal{N}_t(x_j)}\Delta I(n),$$

onde $T$ é o número de árvores, $\mathcal{N}_t(x_j)$ é o conjunto de nós da árvore $t$ que utilizam $x_j$ para o *split*, e $\Delta I(n)$ é a redução ponderada de impureza no nó $n$.

Na implementação do `scikit-learn`, os valores de MDI são **normalizados para que a soma seja igual a 1**. Consequentemente, com 13 *features*, nenhuma variável individual ultrapassa tipicamente valores de importância em torno de 0,20–0,30, e o eixo horizontal dos gráficos de importância varia de 0 a ≈0,25.

Uma limitação conhecida do MDI é o viés em favor de variáveis contínuas ~~de alta cardinalidade~~ou com muitas categorias, que oferecem mais ~~candidatos para *split*~~pontos de corte candidatos, e de variáveis correlacionadas (Strobl et al. , 2007).

### 5.3.2 XGBoost — *Weight* (contagem de *splits*)

O XGBoost constrói árvores de forma sequencial por *gradient boosting*. Na interface do `scikit-learn` (`XGBClassifier`), o atributo `feature_importances_` retorna, por padrão, a métrica ***weight***: para cada *feature* $x_j$, conta-se o número total de vezes em que $x_j$ foi selecionada como variável de *split* em todas as árvores do *ensemble*. Os valores são então normalizados para somar 1.

A métrica *weight* mede a **frequência de uso** da variável, não a magnitude da melhoria que cada *split* proporciona. Alternativas disponíveis no XGBoost incluem *gain* (melhoria média de perda) e *cover* (número médio de amostras afetadas). Neste trabalho, utilizou-se o *weight* padrão.

### 5.3.3 CatBoost — *PredictionValuesChange*

O CatBoost, assim como o XGBoost, é um algoritmo de *gradient boosting*. Porém, a métrica de importância padrão retornada por `feature_importances_` é o ***Prediction-***

***ValuesChange***: para cada *feature* $x_j$, calcula-se a mudança média no valor de predição do modelo quando os valores de $x_j$ são permutados aleatoriamente. Essa abordagem é conceitualmente próxima da **importância por permutação**, sendo mais fidedigna à contribuição preditiva real da variável do que métricas baseadas em contagem de *splits*.

Os valores de *PredictionValuesChange* são normalizados para somar 100 na implementação do CatBoost, de modo que a escala dos gráficos difere da adotada pelo Random Forest e XGBoost.

### 5.3.4 Regressão Logística — valor absoluto dos coeficientes

Na Regressão Logística, cada *feature* $x_j$ possui um coeficiente $\beta_j$ no modelo log-linear (a mesma notação do Capítulo 3):

$$\log \frac{P(y=1)}{P(y=0)} = \beta_0 + \sum_{j=1}^{p} \beta_j \, x_j.$$

Como as *features* são previamente padronizadas (média zero, desvio unitário) pelo `StandardScaler`, os coeficientes estão em escalas comparáveis. A importância de $x_j$ é então estimada como $|\beta_j|$: quanto maior o módulo do coeficiente, maior a influência da variável na probabilidade de ocultação.

Ao contrário dos modelos baseados em árvores, esses valores **não são normalizados para somar 1 ou 100**, e por isso a escala do gráfico pode variar de 0 a valores como 3 ou 4, dependendo do problema.

### 5.3.5 Comparação entre os métodos

A Tabela 5.3 sintetiza as diferenças entre os quatro métodos.

Tabela 5.3: Comparação dos métodos de cálculo de importância de *features* utilizados por cada modelo.

| Modelo | Métrica | Normalização | Escala típica |
|---|---|---|---|
| Random Forest | MDI (Gini) | $\sum = 1$ | 0 a ≈0,25 |
| XGBoost | *Weight* (contagem de *splits*) | $\sum = 1$ | 0 a ≈0,15 |
| CatBoost | *PredictionValuesChange* | $\sum = 100$ | 0 a ≈30 |
| Regressão Logística | $\lvert\beta_j\rvert$ (coeficientes) | Sem normalização | 0 a ≈4 |

É fundamental notar que **a mesma *feature* pode ter importância relativa diferente entre modelos**, sem que isso represente contradição: cada algoritmo captura padrões distintos nos dados, e a métrica de importância reflete a forma como cada modelo utiliza internamente a variável. A análise de importância, portanto, deve ser interpretada dentro de cada modelo, e não como um *ranking* absoluto.

## 5.4 Configuração experimental

Para os experimentos foram processadas **802 curvas positivas** do banco de dados (VizieR e Grupo do Rio); a partir delas, foram obtidos **186 negativos por recorte** manual de segmentos sem ocultação. Além disso, foram utilizadas **3 curvas negativas** do próprio banco e **702 curvas ~~sintéticas~~ negativas simuladas** geradas pelo simulador de ~~Gomes-Ferrante & Braga-Ribas (Gomes-Ferrante & Braga-Ribas , 2023)~~Gomes-Ferrante & Braga-Ribas ( . A Tabela 5.4 resume a composição do *dataset*.

Tabela 5.4: Distribuição das amostras por fonte no *dataset* utilizado nos experimentos.

| **Fonte** | **Quantidade** |
|---|---|
| Curvas positivas do banco (db_positive) | 802 |
| Curvas ~~sintéticas negativas (synthetic~~negativas simuladas (`synthetic`) | 702 |
| Negativos por recorte (manual) | 186 |
| Curvas negativas do banco (db_negative) | 3 |
| **Total** | 1693 |

A divisão entre treino e teste foi feita de forma **estratificada por curva** (80% treino, 20% teste): cada curva aparece integralmente em treino ou em teste, nunca nos dois, evitando vazamento de informação. O conjunto de teste contém 339 curvas (aproximadamente 178–179 negativas e 160–161 positivas).

Os hiperparâmetros (número de árvores, profundidade máxima, regularização, etc.) foram fixados conforme o Apêndice B. Utilizou-se ponderação das classes nos quatro algoritmos para lidar com o desbalanceamento residual. A semente aleatória foi fixada em 42 para reprodutibilidade. Todas as métricas referem-se exclusivamente ao **conjunto de teste**.

## 5.5 Resultados do Experimento 1 (linha de base, 28 *features*)

No Experimento 1 foram utilizadas as 28 *features* descritas na Tabela 5.1 (conjunto completo). A Tabela 5.5 resume as métricas de desempenho no conjunto de teste.

Tabela 5.5: Métricas de desempenho no conjunto de teste — Experimento 1. Conjunto completo de 28 *features*, divisão 80%/20%.

| **Modelo** | **Acurácia** | **Precisão** | **Sensibilidade** | **F1-score** | **AUC-ROC** |
|---|---|---|---|---|---|
| Random Forest | **0,9941** | 1,0000 | 0,9875 | **0,9937** | 0,9997 |
| CatBoost | 0,9912 | 0,9937 | 0,9875 | 0,9906 | 0,9998 |
| Regressão Log. | 0,9882 | 1,0000 | 0,9750 | 0,9873 | 1,0000 |
| XGBoost | 0,9853 | 0,9814 | 0,9875 | 0,9844 | 0,9997 |

O melhor desempenho em F1-score e acurácia foi obtido pelo **Random Forest** (F1 = 0,9937; acurácia = 0,9941). A sensibilidade de 98,75% indica que a grande maioria das curvas com ocultação foi corretamente identificada; a precisão de 100% indica ausência de falsos positivos nesse modelo. A Regressão Logística também alcança precisão de 100%. A AUC-ROC muito alta ($\geqslant$ 0,9997) confirma excelente capacidade de separação das classes.

A Figura 5.1 apresenta as matrizes de confusão dos quatro modelos no Experimento 1. Em cada matriz, as linhas indicam a classe real e as colunas a classe predita. A diagonal principal contém os acertos: verdadeiros negativos no canto superior esquerdo e verdadeiros positivos no canto inferior direito. Fora da diagonal ficam os erros: falsos positivos (superior direito) e falsos negativos (inferior esquerdo). Por exemplo, o Random Forest classifica corretamente as 179 curvas negativas e 158 das 160 positivas, com 2 falsos negativos e nenhum falso positivo.

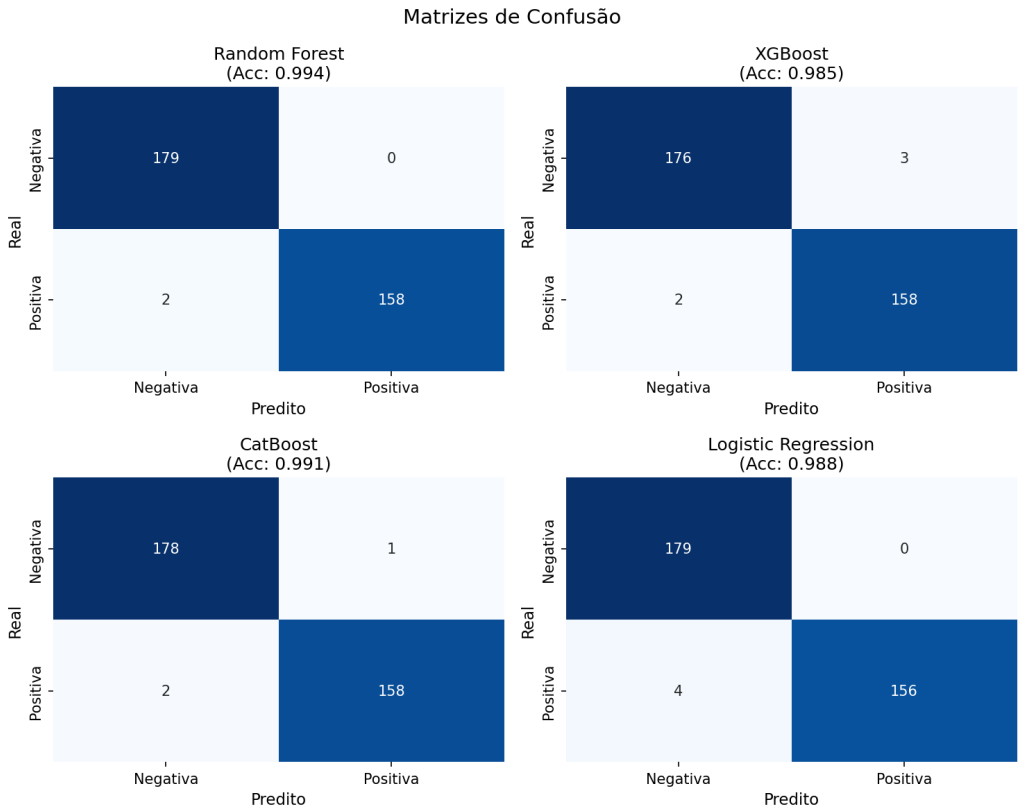



Figura 5.1: Matrizes de confusão dos quatro modelos no Experimento 1 (28 *features*). Eixos: real (vertical) e predito (horizontal). Classes: Negativa (0) e Positiva (1).

## 5.6 Redução do conjunto de *features*

Partindo das 28 *features* do Experimento 1, o conjunto de variáveis foi progressivamente reduzido, removendo aquelas identificadas como redundantes ou pouco discrimi-

nativas (Seção 5.2). O Experimento 4 (14 *features*) elimina amplitude, desvio padrão do fluxo bruto, máximo do Savitzky-Golay e as derivadas; o Experimento 5 (13 *features*) remove adicionalmente `Feature_Savgol_Min`; e o Experimento 6 (12 *features*) ablaciona a `kmeans_centroid_dist`. Em todos eles, o melhor F1-score permanece em torno de 0,99 — ou seja, **a remoção dessas variáveis não degrada o desempenho**. Os resultados completos desses três experimentos (tabelas e matrizes de confusão) estão no Apêndice A; a Tabela 5.10 (Seção 5.10) consolida os valores.

Um achado merece destaque por ser, à primeira vista, contraintuitivo. A `kmeans_centroid_dist` — distância entre os dois centróides obtidos pela aplicação de ~~K-Means~~ K-means ($K = 2$) ao fluxo suavizado — figura consistentemente entre as variáveis *mais importantes* nos modelos de árvore, chegando a concentrar boa parte da importância no XGBoost. Ainda assim, removê-la (Experimento 6) **não piora o desempenho**: a capacidade preditiva é simplesmente redistribuída para variáveis colineares, sobretudo `Feature_Savgol_std` — que captura o mesmo fenômeno físico, a dispersão do fluxo causada pelos dois patamares dentro e fora do *dip*. A lição é importante: **a importância atribuída por um algoritmo de árvore não é sinônimo de insubstituibilidade**; uma variável pode concentrar importância apenas por oferecer um ponto de corte conveniente, enquanto outras compensam sua ausência. Na prática, isso permite simplificar a *pipeline* eliminando o passo de ~~K-Means~~ K-means sem custo de desempenho.

### 5.6.1 Configuração enxuta: Experimento 2 (11 *features*)

O Experimento 2 leva essa simplificação ao limite, removendo *simultaneamente* `Feature_Savgol_Min` e `kmeans_centroid_dist` e restando **11 *features***, com divisão 80%/20%. É esta a **configuração enxuta** adotada como referência operacional do trabalho — a mais simples que preserva o desempenho da linha de base — e a empregada no estudo de caso de Quaoar (Seção 5.9). A Tabela 5.6 apresenta as métricas.

Tabela 5.6: Métricas de desempenho no conjunto de teste — Experimento 2. 11 *features* (sem `Feature_Savgol_Min` e sem `kmeans_centroid_dist`), divisão 80%/20%.

| Modelo | Acurácia | Precisão | Sensibilidade | F1-score | AUC-ROC |
|---|---|---|---|---|---|
| XGBoost | **0,9912** | 0,9937 | 0,9875 | **0,9906** | 0,9995 |
| CatBoost | **0,9912** | 1,0000 | 0,9813 | **0,9905** | 0,9997 |
| Random Forest | 0,9882 | 0,9937 | 0,9813 | 0,9874 | 0,9996 |
| Regressão Log. | 0,9853 | 1,0000 | 0,9688 | 0,9841 | 1,0000 |

XGBoost e CatBoost mantêm F1 $\geqslant$ 0,990, confirmando que a ablação combinada não degrada o desempenho mesmo com menos de metade das variáveis originais. A matriz de confusão dos quatro modelos nesta configuração (Figura A.4), assim como as dos demais experimentos, encontra-se no Apêndice A.

## 5.7 Teste de generalização: curvas exclusivamente reais

### 5.7.1 ~~Motivação~~

No *dataset* utilizado, cerca de 78% das curvas negativas são ~~sintéticas~~**simuladas** (geradas pelo simulador de ~~Gomes-Ferrante & Braga-Ribas (Gomes-Ferrante & Braga-Ribas , 2023)~~ Gomes-Ferrante & Braga-Ribas , 2023). Assim, o conjunto de teste padrão mistura curvas reais e ~~sintéticas~~simuladas. Para avaliar se o desempenho se mantém em cenários onde apenas curvas **reais** são classificadas — situação típica de uso em campanhas observacionais — foi realizado o Experimento 3: os modelos são treinados com 11 *features* e a avaliação é feita **somente em curvas reais**. A divisão adotada é a mesma dos demais experimentos (80%/20%, estratificada por curva), de modo que os resultados sejam diretamente comparáveis; a fração aplica-se às curvas elegíveis para o teste (positivas reais, negativos por recorte e negativas nativas), enquanto as curvas ~~sintéticas~~ simuladas são alocadas integralmente ao treino. O conjunto de teste resultante contém 198 curvas reais (160 positivas e 38 negativas). Uma variante com divisão 65%/35%, que amplia o número de curvas reais avaliadas ao custo de reduzir o treino, é analisada no Apêndice A como ablação do tamanho do conjunto de treino.

### 5.7.1 ~~Resultados do Experimento 3~~

~~A~~ Os resultados estão na Tabela 5.7, que apresenta as métricas no conjunto de teste restrito a curvas reais.

Tabela 5.7: Métricas de desempenho no conjunto de teste — Experimento 3. Teste somente em curvas reais (11 *features*, divisão 80%/20%; teste com 198 curvas: 160 positivas e 38 negativas).

| Modelo | Acurácia | Precisão | Sensibilidade | F1-score | AUC-ROC |
|---|---|---|---|---|---|
| XGBoost | **0,9848** | 0,9937 | 0,9875 | **0,9906** | 0,9984 |
| Random Forest | 0,9798 | 0,9937 | 0,9812 | 0,9874 | 0,9965 |
| CatBoost | 0,9798 | 0,9937 | 0,9812 | 0,9874 | 0,9980 |
| Regressão Log. | 0,9646 | 0,9935 | 0,9625 | 0,9778 | 0,9974 |

### 5.7.1 ~~Interpretação~~

O desempenho em teste exclusivamente real é **essencialmente preservado**: o melhor F1 (XGBoost, 0,9906) coincide com o obtido no teste misto do Experimento 2, e a AUC-ROC permanece $\geqslant$ 0,9965 em todos os modelos. Isso indica que as *features* capturam sinal discriminativo real e não apenas artefatos da distribuição ~~sintética. Vale registrar que a queda de ≈2 pontos percentuais observada na~~ simulada. Na variante 65%/35% do

Experimento 3 (Apêndice A)~~decorre, em parte,da redução do conjunto~~ , o melhor F1 passa de 0,9906 para 0,9838, uma diferença de ~~treino~~0,~~e~~ 7 ponto percentual (pp). Como essa variante altera ao mesmo tempo o treino ($-19\,\%$ das curvas reais de treino, ou $-10\,\%$ do treino total) e o conjunto de teste (198 → 347 curvas), não ~~apenas da composição do teste~~é possível atribuir a diferença à redução do treino: trata-se de uma hipótese, não de um resultado. A diferença tem, de todo modo, a mesma ordem da flutuação entre partições (desvio padrão do F1 na validação cruzada de 0,006 a 0,010; Tabela 5.11). Como ressalva, o teste desta configuração contém apenas 38 curvas negativas reais, de modo que as métricas ligadas a falsos positivos possuem maior incerteza amostral. O Experimento 3 reforça que a *pipeline* é adequada para triagem em dados observacionais reais.

## 5.8 Minimização de falsos negativos

### 5.8.1 Motivação: assimetria de custo entre erros

Nos experimentos anteriores, todos os modelos utilizam o limiar de decisão padrão $\tau = 0{,}5$: uma curva é classificada como positiva se a probabilidade estimada $\hat{p} = P(y = 1 \mid \mathbf{x})$ for $\geqslant 0{,}5$. Esse limiar implica que falsos positivos e falsos negativos possuem o mesmo custo — hipótese que raramente se verifica em aplicações reais.

No contexto de triagem de curvas de luz para análise de ocultações estelares, os custos são **assimétricos**:

- **Falso negativo** (ocultação real classificada como negativa): a curva deixa de ser prioridade para análise mais detalhada. O dado não é perdido — desde que não seja descartado, a informação continua disponível —, mas o evento pode demorar a ser "redescoberto", com risco de o grupo perder a oportunidade de um resultado inédito (como ilustra a ~~própria~~ história ~~dos anéis~~ do primeiro anel de Quaoar, ~~revelados só numa revisita dos dados~~ revelado numa revisita de dados antigos (Morgado et al. , 2023)).

- **Falso positivo** (curva sem evento classificada como positiva): a curva recebe prioridade indevida e será examinada por um analista humano, que rapidamente descartará o falso alarme. O custo é apenas tempo de análise.

Essa assimetria é análoga à de sistemas de triagem médica (*screening*), nos quais se prefere aceitar mais falsos alarmes a perder diagnósticos reais. Formalmente, o objetivo é **maximizar a sensibilidade** (minimizar falsos negativos), aceitando uma redução controlada da precisão (aumento de falsos positivos).

### 5.8.2 Método: variação do limiar de classificação

Todos os quatro classificadores utilizados neste trabalho produzem, além do rótulo binário, uma **probabilidade estimada** $\hat{p} = P(y = 1 \mid \mathbf{x}) \in [0, 1]$ de a curva conter uma ocultação. Essa saída contínua é obtida pelo método `predict_proba` das bibliotecas `scikit-learn`, XGBoost e CatBoost, que converte a pontuação interna de cada modelo em uma probabilidade. A classificação binária é, então, obtida pela aplicação de um limiar $\tau$ sobre $\hat{p}$:

$$\hat{y} = \begin{cases} 1 & \text{se } \hat{p} \geqslant \tau, \\ 0 & \text{caso contrário.} \end{cases}$$

Ao **reduzir** $\tau$ abaixo de 0,5, o modelo torna-se mais "sensível": classifica como positiva qualquer curva com probabilidade moderada de conter uma ocultação. Isso tem dois efeitos simultâneos:

1. A **sensibilidade aumenta**: curvas com evento que estavam na zona de incerteza passam a ser corretamente classificadas como positivas.

2. A **precisão diminui**: algumas curvas sem evento cuja probabilidade estimada está entre $\tau$ e 0,5 passam a ser classificadas como positivas (falsos positivos adicionais).

O compromisso entre precisão e sensibilidade pode ser visualizado por meio da **curva precisão-sensibilidade** (*precision-recall curve*), que é parametrizada por $\tau$. Diferentemente da curva ROC — que compara taxa de verdadeiros positivos (TPR) com taxa de falsos positivos (FPR) —, a curva precisão-sensibilidade é mais informativa para conjuntos de dados com leve desbalanceamento e quando o objetivo é minimizar falsos negativos (Saito & Rehmsmeier , 2015).

Adicionalmente, para cada valor de $\tau$, podem-se recalcular todas as métricas de classificação (acurácia, precisão, sensibilidade, F1-score), gerando **curvas de métrica em função do limiar** que permitem escolher o ponto operacional mais adequado ao cenário de uso.

### 5.8.3 Escolha do ponto operacional

A seleção do limiar ótimo depende do critério adotado. Três abordagens comuns são:

1. **Sensibilidade mínima garantida:** fixar um patamar desejado de sensibilidade (ex.: $\geqslant 99{,}5\%$) e identificar o maior $\tau$ que o satisfaz. Esse é o critério mais natural para triagem científica.

2. **Maximização do $\mathbf{F}_\beta$-score:** utilizar uma versão ponderada do F1 que atribui peso

$\beta^2$ à sensibilidade:

$$F_\beta = (1+\beta^2)\cdot\frac{\text{precisão}\cdot\text{sensibilidade}}{\beta^2\cdot\text{precisão}+\text{sensibilidade}}.$$

Com $\beta = 2$, a sensibilidade vale quatro vezes mais que a precisão; o limiar ótimo é aquele que maximiza $F_2$.

3. **Análise de custo:** atribuir ~~custos explícitos~~ um custo $c_{\text{FN}}$ ~~e~~ a cada falso negativo (evento perdido) e um custo $c_{\text{FP}}$ ~~e minimizar o custo esperado total~~ a cada falso positivo (alarme a ser revisado) e escolher $\tau$ que minimize o custo total $C(\tau) = c_{\text{FN}}\,\text{FN}(\tau) + c_{\text{FP}}\,\text{FP}(\tau)$. Se as probabilidades forem bem calibradas, o limiar ótimo é $\tau^\star = c_{\text{FP}}/(c_{\text{FP}}+c_{\text{FN}})$; por exemplo, $c_{\text{FN}} = 9\,c_{\text{FP}}$ dá $\tau^\star = 0{,}1$. Quando $c_{\text{FN}} \gg c_{\text{FP}}$, o limiar ótimo desloca-se para baixo.

Neste trabalho, adota-se o critério de **sensibilidade mínima garantida**, por ser o mais transparente e o mais fácil de comunicar: "o modelo detecta pelo menos $X\%$ das ocultações reais, ao custo de $Y$ falsos alarmes adicionais".

### 5.8.4 Resultados

O ajuste de limiar foi aplicado aos quatro modelos do Experimento 5 (13 *features*, divisão 80%/20%; detalhado no Apêndice A). Para cada modelo, foram calculadas as métricas em função do limiar $\tau \in [0{,}01,\ 0{,}99]$. A Tabela 5.8 resume pontos operacionais representativos.

Tabela 5.8: Pontos operacionais selecionados na análise de limiar (Experimento 5, 13 *features*). Para cada modelo: limiar padrão ($\tau = 0{,}5$), limiar sensível ou ótimo, e métricas associadas (FP = falsos positivos, FN = falsos negativos).

| Modelo | Limiar | Precisão | Sensibilidade | F1 | $F_2$ | FP / FN |
|---|---|---|---|---|---|---|
| Random Forest | $\tau = 0{,}50$ (padrão) | 1,0000 | 0,9813 | 0,9905 | 0,9849 | 0 / 3 |
| | $\tau = 0{,}45$ (sensível) | 0,9938 | 0,9938 | 0,9938 | 0,9938 | 1 / 1 |
| | $\tau = 0{,}01$ (máx. sens.) | 0,7921 | 1,0000 | 0,8840 | 0,9501 | 42 / 0 |
| Regressão Log. | $\tau = 0{,}16$ | 0,9938 | 1,0000 | 0,9969 | 0,9988 | 1 / 0 |
| XGBoost | $\tau = 0{,}30$ | 0,9875 | 0,9875 | 0,9875 | 0,9875 | 2 / 2 |
| CatBoost | $\tau = 0{,}37$ | 0,9875 | 0,9875 | 0,9875 | 0,9875 | 2 / 2 |

**Destaque:** A Regressão Logística com $\tau = 0{,}16$ atinge sensibilidade de 100% com apenas 1 falso positivo — ponto operacional ideal para triagem em que minimizar falsos negativos é prioritário.

As Figuras 5.2 e 5.3 ilustram as curvas precisão-sensibilidade e as métricas em função do limiar (Experimento 5). Variando $\tau$ de 1 a 0, percorre-se cada curva precisão-sensibilidade da esquerda (alta precisão, baixa sensibilidade) para a direita (alta sensibilidade, menor precisão); quanto mais a curva se aproxima do canto superior direito, melhor

o modelo. Como todos os modelos operam em valores muito altos, a região de interesse concentra-se acima de 0,9 em ambos os eixos.

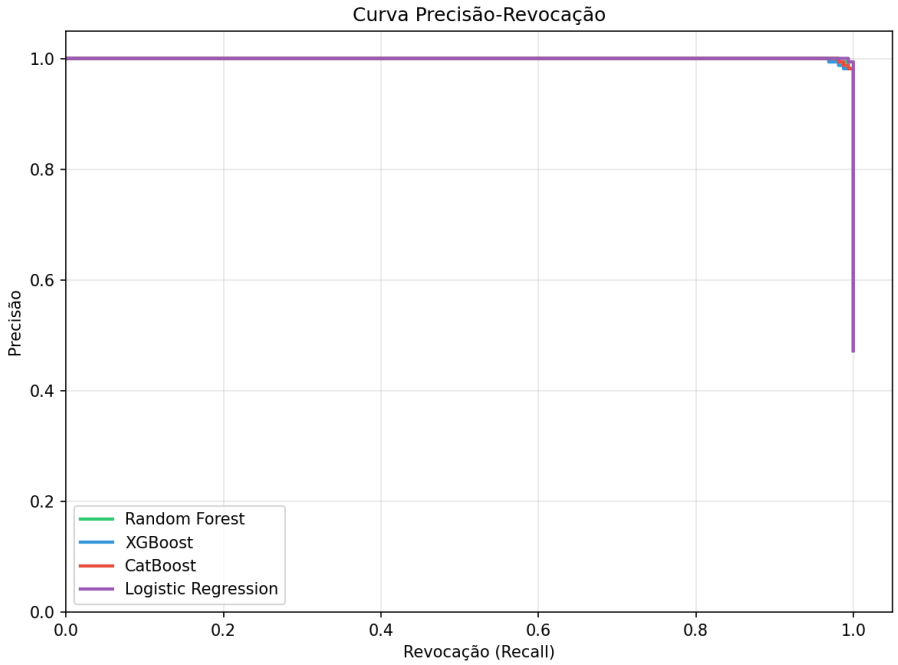


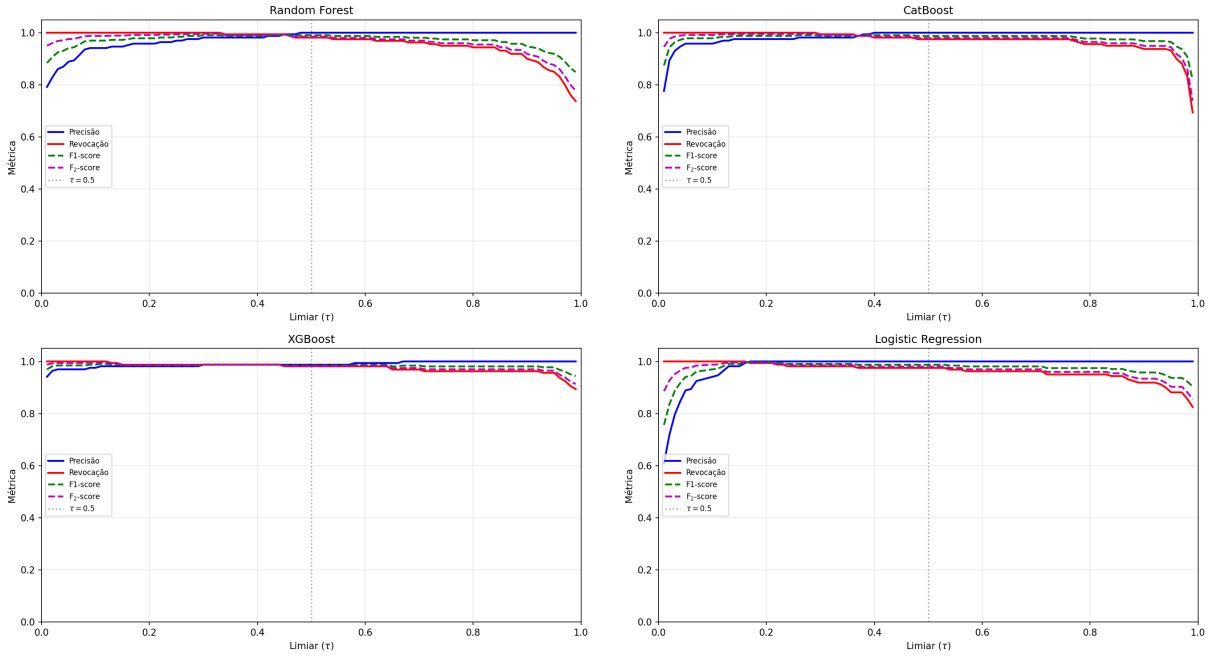

Figura 5.2: Curvas precisão-sensibilidade dos quatro modelos ~~(~~no Experimento~~5).~~ 5. Cada ~~ponto da~~ posição ao longo de uma curva corresponde a um valor ~~de~~ do limiar $\tau$~~: variando $\tau$ de 1 a 0.~~ No gráfico, ~~percorre-se a curva da esquerda (alta precisão, baixa~~ "revocação" é sinônimo de sensibilidade ~~) para a direita~~ (~~alta sensibilidade, menor precisão~~*recall*).~~O ponto ideal é o canto superior direito (precisão e sensibilidade próximas de 1); quanto mais a curva se aproxima dele, melhor o modelo. Como todos os modelos operam em valores muito altos, a região de interesse concentra-se acima de 0,9 em ambos os eixos.~~

### 5.8.5 Implicações para uso operacional

O ajuste de limiar é uma operação de **pós-processamento**: não requer retreinamento dos modelos, não altera os pesos aprendidos e pode ser realizado instantaneamente. Na prática, isso significa que a *pipeline* pode operar em **dois modos**:

- **Modo balanceado** ($\tau = 0{,}5$): otimiza o equilíbrio entre precisão e sensibilidade. Adequado para relatórios de desempenho geral e comparação entre modelos.
- **Modo triagem** ($\tau < 0{,}5$): maximiza a detecção de ocultações reais, aceitando falsos alarmes adicionais que serão descartados na etapa de análise manual. Adequado para o cenário operacional de priorização de curvas.

A escolha do limiar pode, inclusive, ser adaptada por campanha observacional: em campanhas com poucas curvas (dezenas), um limiar mais baixo é aceitável porque o custo




(a) Random Forest e XGBoost (b) CatBoost e Regressão Logística

Figura 5.3: Precisão, sensibilidade, F1-score e $F_2$-score em função do limiar de decisão $\tau$ (Experimento 5). (a) Random Forest e XGBoost; (b) CatBoost e Regressão Logística.

de revisar falsos positivos é pequeno; em campanhas com milhares de curvas, pode-se optar por um limiar intermediário que limite o volume de falsos alarmes a um nível gerenciável pela equipe de análise.

## 5.9 Estudo de caso: aplicação a uma curva real de Quaoar

### 5.9.1 Motivação

Os seis experimentos descritos nas seções anteriores avaliam a *pipeline* sobre curvas oriundas do banco de dados de treinamento (catálogo `B/occ/asteroid` do VizieR e Grupo do Rio), respeitando a partição treino/teste. Para complementar essa avaliação, apresenta-se nesta seção um **estudo de caso** no qual a *pipeline* é aplicada a um conjunto de curvas **externas ao banco**: registros de uma ocultação estelar pelo objeto transnetuniano **(50000) Quaoar**, que não estavam armazenadas no VizieR nem no *drive* do Grupo do Rio. O evento é especialmente interessante porque, além da queda profunda associada ao corpo principal, apresenta assinaturas adicionais atribuídas aos dois anéis recentemente descobertos em torno de Quaoar (Pereira et al. , 2023; Morgado et al. , 2023), configurando uma curva complexa com múltiplas assinaturas físicas (corpo principal, anéis e o limite para a existência de uma possível atmosfera) em uma única série temporal. Foram analisadas três curvas do mesmo evento: CFHT/WIRCam (banda $K_s$)

e Gemini-Norte/'Alopeke, nos canais azul (filtro $r$) e vermelho (filtro $z$).

Este estudo de caso tem três objetivos:

1. Verificar se o modelo ~~treinado~~ CatBoost do Experimento 2 (11 *features*; na análise de recortes, também o XGBoost do mesmo experimento) generaliza para curvas reais **nunca vistas** — isto é, fora do conjunto de teste e fora da distribuição dos catálogos de origem.

2. Avaliar o comportamento da *pipeline* em uma curva **morfologicamente complexa**, com componentes não típicos da maioria das curvas de treino (anéis e estruturas secundárias).

3. Demonstrar o uso da *pipeline* como ferramenta de **recorte interativo**, permitindo isolar segmentos específicos da curva (anel, atmosfera, corpo ocultante) e reaplicar o classificador a cada janela.

Os dados de Quaoar foram **gentilmente cedidos pelo autor principal** de Pereira et al. (2023), no qual a redução fotométrica completa do evento e a caracterização dos dois anéis Q1R e Q2R são apresentadas. Reforçando o que já se mencionou, a curva de Quaoar não figura no banco SQLite usado para treinar os modelos, garantindo que o teste é, de fato, sobre dados externos à distribuição de treino.

### 5.9.2 Procedimento

A aplicação da *pipeline* ao caso Quaoar foi implementada em um *script* dedicado (`pipeline/model_in_practice/test_quaoar.py`), escrito em estilo ~~*top-down* linear para~~ linear (*top-down*), de cima para baixo, para facilitar a inspeção passo a passo — inclusive a colocação interativa de pontos de parada no VS Code. O fluxo é:

1. **Leitura das curvas:** cada arquivo `.dat` é lido com `np.genfromtxt`, extraindo apenas a coluna de tempo (segundos do dia, em UTC) e a de fluxo normalizado. Pontos marcados com *flag* de baixa qualidade pelo PRAIA (valores extremos como 99,999) são ~~removidos por um clip físico~~ descartados por um filtro de validade física: mantêm-se apenas os pontos com fluxo normalizado no intervalo $[-2,\ 5]$, preservando ocultações profundas reais (fluxo $\to 0$) sem cortar o *dip* verdadeiro.

2. **Recorte opcional por janela temporal:** um dicionário `CUT_RANGES`, preenchido manualmente pelo autor a partir da inspeção visual da curva, permite restringir cada curva a um intervalo $(t_{\text{incio}},\ t_{\text{fim}})$~~em segundos UTC~~, em segundos contados a partir de 00:00:00 UTC de 2022-08-09 (segundos do dia). O recorte é aplicado antes do plot e da extração de *features*, permitindo isolar separadamente o *dip* principal do corpo, possíveis estruturas de anel à esquerda/direita do evento principal ou eventuais assinaturas atmosféricas nas bordas do *dip*.

3. **Extração de *features*:** as mesmas funções utilizadas no treinamento (`extract_features` e `compute_occ_features`, no pacote `model_training`) são reaproveitadas, garantindo coerência exata com o pré-processamento aprendido pelos modelos.

4. **Inferência:** o modelo **CatBoost** treinado no Experimento 2 (11 *features*, sem `Feature_Savgol_Min` e sem `kmeans_centroid_dist`) é carregado a partir do diretório `outputs/resultado5`; o *imputer* — objeto `SimpleImputer` do *scikit-learn*, ajustado no treino, que substitui valores ausentes de uma *feature* pela mediana do conjunto de treino — é recuperado, com recurso alternativo à mediana local caso ocorra incompatibilidade de versão do `scikit-learn`.

A escolha do CatBoost-Exp. 2 é deliberada: trata-se da configuração mais enxuta da *pipeline* (menor número de *features*, sem o passo de ~~K-Means~~K-means), reforçando que o desempenho elevado não depende das variáveis identificadas como dispensáveis pelo estudo de ablação (Seção 5.6).

A Figura 5.4 apresenta a curva analisada após o pré-processamento, com o *dip* central do corpo principal e as estruturas rasas dos anéis. A detecção da **curva inteira** como ocultação é trivial — corpo central mais dois anéis fornecem um sinal forte — e serve apenas como *baseline* (primeira linha da Tabela 5.9, $\hat{p} \approx 0{,}995$ no CatBoost). O interesse real está em **sondar os anéis por recortes**, isolando cada assinatura física.

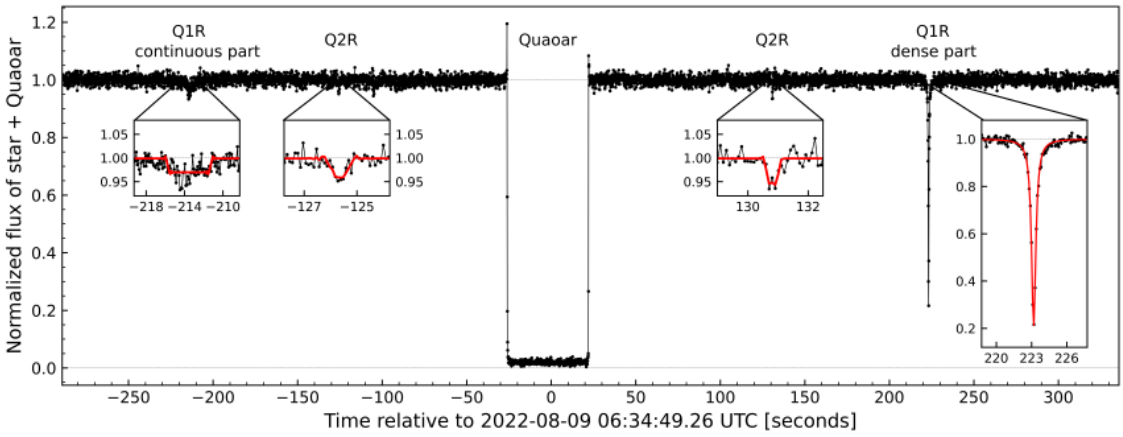



Figura 5.4: Curva de luz da ocultação estelar por Quaoar utilizada no estudo de caso, com o tempo referenciado ao instante 2022-08-09 06:34:49,26 UTC. O *dip* central (anotado como "Quaoar"), de duração $\approx 48$ s e profundidade $\approx 100\%$, corresponde à ocultação pelo corpo principal. As estruturas secundárias rasas, isoladas nos quatro quadros ampliados (*insets*), são as assinaturas dos dois anéis~~denominados~~, Q1R (Morgado et al. , 2023) e Q2R ~~em~~ (Pereira et al. , 2023), sendo Q1R observado em duas porções (parte contínua e parte densa) e Q2R em duas travessias antes e depois do corpo. A linha pontilhada indica o *baseline* (~~$f_{\text{norm}} = 1$~~$F_{\text{norm}} = 1$).

### 5.9.3 Aplicação a recortes da curva: sondando os anéis

A demonstração mais interessante deste estudo de caso explora justamente a ideia anunciada na Introdução: usar a *pipeline* para **(re)visitar uma curva complexa por recortes**, isolando trechos específicos e perguntando ao modelo, janela a janela, se há indício de evento. O dicionário `CUT_RANGES` do *script* permite reexecutar a inferência sobre uma janela temporal arbitrária — a operação é uma única linha (`cut_curve(time, flux, t_ini, t_fim)`) e pode ser feita interativamente no ~~Debug Console~~ *Debug Console* do VS Code, sem retreinar o modelo nem reescrever a *pipeline*.

Tomando a curva Gemini-'Alopeke *Red-z* — a de maior SNR —, definiram-se **seis janelas** que cobrem assinaturas físicas distintas: um trecho de *baseline* sem evento (controle negativo), as duas porções do anel Q1R (contínua e densa), as duas travessias do anel fino Q2R (antes e depois do corpo) e um trecho curto de ruído ~~visualmente parecido com uma microocultação~~que se assemelha visualmente a uma ocultação, adjacente ao Q2R (janelas destacadas na Figura 5.5). A inferência foi aplicada a cada janela com os modelos CatBoost (o modelo de referência da Seção 5.9) e XGBoost do Experimento ~~2.~~2; o Random Forest e a Regressão Logística não foram avaliados nos recortes. A Tabela 5.9 reúne as probabilidades atribuídas.

Tabela 5.9: Probabilidade de ocultação $\hat{p}$ atribuída por CatBoost e XGBoost (Exp. 2) à curva inteira (*baseline*) e a cada recorte da curva Gemini-'Alopeke *Red-z*. Janelas em segundos relativos ao instante 2022-08-09 06:34:49,26 UTC.

| **Recorte (janela, s)** | **Assinatura física** | $\hat{p}$ **CatBoost** | $\hat{p}$ **XGBoost** |
|---|---|---|---|
| Curva inteira | corpo central + 2 anéis (*baseline*) | 0,9951 | 0,9996 |
| −290 a −250 | *baseline* (controle negativo) | 0,029 | 0,0006 |
| −225 a −200 | Q1R — parte contínua | 0,685 | 0,9601 |
| −129 a −114 | Q2R — anel fino (real) | 0,058 | 0,0434 |
| −109 a −099 | ruído parecido com anel | 0,005 | 0,0005 |
| +121 a +146 | Q2R — anel fino (real) | 0,456 | 0,0807 |
| +191 a +251 | Q1R — parte densa | 0,987 | 0,9994 |

A leitura da tabela revela três regimes. **(i) Anéis fortes (Q1R):** tanto a parte contínua quanto a densa são detectadas com folga ($\hat{p} = 0{,}96$ e 0,999 no XGBoost) — o sinal é profundo ou longo o bastante para o modelo. **(ii) Controle e ruído:** o *baseline* sem evento e o trecho de ruído recebem probabilidades ínfimas ($\hat{p} \approx 6 \times 10^{-4}$ e $5 \times 10^{-4}$ no XGBoost), confirmando que o classificador não tem viés pró-positivo. **(iii) Anéis finos (Q2R):** aqui está o ponto sutil. Ambos os anéis Q2R recebem veredito *negativo* no limiar padrão $\tau = 0{,}5$ ($\hat{p} = 0{,}043$ e 0,081 no XGBoost).

À primeira vista, isso pareceria uma falha. Mas o que importa para a revisita de curvas **não é o valor absoluto da probabilidade, e sim a separação entre eventos e não-eventos**. O anel fino real (Q2R, $\hat{p} = 0{,}043$) recebe probabilidade cerca de **86 vezes maior** do que o trecho de ruído adjacente, de duração comparável ($\hat{p} = 0{,}0005$) — quase duas ordens de grandeza de diferença. Em outras palavras, mesmo "errando" a

classificação binária, o modelo *ordena corretamente* as janelas: atribui ao anel verdadeiro uma probabilidade pequena em termos absolutos (4–8%), mas enorme frente à do ruído que mais se parece com uma ocultação (0,05%). É exatamente esse contraste que um analista revisitando a curva precisa enxergar.

Essa separação é o que torna o **ajuste de limiar** (Seção 5.8) tão eficaz neste caso. Baixando o limiar para $\tau = 0{,}03$ — um valor entre as duas escalas de probabilidade —, os dois anéis Q2R passam a ser corretamente classificados como ocultações, *sem* que o controle ($6 \times 10^{-4}$) ou o ruído ($5 \times 10^{-4}$) cruzem o novo limiar. A sensibilidade sobre os quatro eventos físicos da curva (Q1R contínuo, Q1R denso, $\mathrm{Q2R}_1$ e $\mathrm{Q2R}_2$) sobe de 50% para 100%, sem nenhum falso alarme nos dois recortes-controle. Esta é a manifestação prática, em dados reais externos ao treino, do mecanismo de *threshold tuning* apresentado na Seção 5.8. A Figura 5.5 ilustra os seis recortes coloridos sobre a curva, com o veredito em $\tau = 0{,}5$ e em $\tau = 0{,}03$.

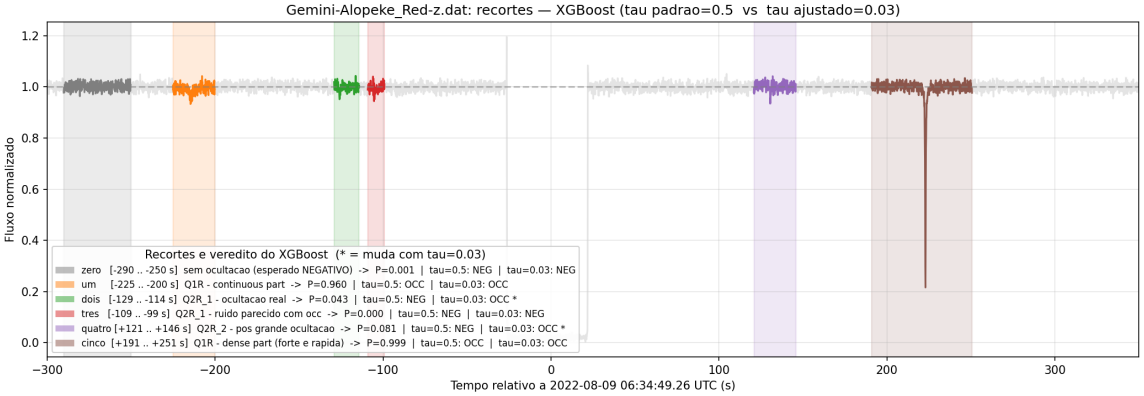



Figura 5.5: Recortes da curva Gemini-'Alopeke canal *Red*- filtro $z$ (visível-vermelho) de Quaoar, com o veredito do XGBoost em cada janela colorida. A legenda indica, por recorte, a probabilidade $\hat{p}$ e a classificação no limiar padrão ($\tau = 0{,}5$) e no limiar ajustado ($\tau = 0{,}03$). Os anéis finos Q2R, negativos em $\tau = 0{,}5$, são recuperados em $\tau = 0{,}03$, enquanto o controle e o trecho de ruído permanecem corretamente negativos. Tempo relativo a 2022-08-09 06:34:49,26 UTC.

**A janela curta não é a causa.** Uma hipótese natural para a baixa probabilidade dos anéis finos seria que as janelas correspondentes (10–25 s) são curtas demais, prejudicando a extração de *features*. Essa hipótese foi testada e **refutada**: ao ampliar as janelas dos recortes Q2R por um fator $\sim 3$, a probabilidade atribuída pelo XGBoost ao anel real $\mathrm{Q2R}_1$ *caiu* de 0,043 para 0,035, e a do $\mathrm{Q2R}_2$ caiu de 0,081 para 0,029 — ou seja, ampliar a janela tornou o veredito negativo ainda mais forte. A explicação é estrutural: o conjunto de *features* é dominado por **estatísticas globais** da janela (mínimo do Savitzky-Golay, *Max Drawdown*, profundidade, SNR do *dip*). Um *dip* de poucos segundos diluído em dezenas de segundos de *baseline* limpo torna-se um *outlier* estatisticamente desprezível,

e o modelo responde com *mais* confiança que se trata de *baseline* puro. A fronteira de detecção é, portanto, regida pela razão entre a duração do *dip* e a duração da janela — não pelo tamanho absoluto da janela. Esse diagnóstico motiva diretamente as direções de trabalho futuro discutidas no Capítulo 6 (janela deslizante e *features* locais).

**Por que XGBoost.** A análise de recortes destaca o XGBoost porque suas probabilidades são mais **polarizadas**: a razão entre a probabilidade do anel real e a do ruído no $Q2R_1$ é de $\sim 86\times$ no XGBoost, contra $\sim 12\times$ no CatBoost (Tabela 5.9). Os dois modelos concordam qualitativamente — ambos detectam o corpo e os anéis fortes e rejeitam controle e ruído —, mas a maior separação do XGBoost torna o ajuste de limiar mais robusto. Isso é coerente com a leve vantagem do XGBoost no teste em curvas exclusivamente reais (Experimento 3). A subseção anterior, sobre a curva completa, usou o CatBoost-Exp. 2; aqui, ambos classificam o corpo principal com $\hat{p} > 0{,}99$, e o XGBoost é preferido apenas para a discussão dos eventos sutis.

### 5.9.4 Interpretação

O estudo de caso reforça, na prática, quatro conclusões dos experimentos formais:

1. **Generalização para dados externos:** mesmo treinado em curvas predominantemente ~~sintéticas~~ simuladas e oriundas dos catálogos VizieR/Grupo do Rio, o modelo do Experimento 2 classifica corretamente registros de uma campanha observacional distinta, em três bandas espectrais e dois telescópios diferentes.

2. **Robustez a complexidade morfológica:** a presença de estruturas secundárias na curva (anéis) não confunde o classificador, que mantém probabilidades $> 0{,}99$ para o evento principal.

3. **Capacidade de triagem de eventos sutis por recorte:** ainda que os anéis finos Q2R fiquem abaixo do limiar padrão, o modelo lhes atribui probabilidade até $\sim 86\times$ maior do que a um trecho de ruído semelhante. Essa separação — e não o valor absoluto — é o que permite recuperá-los com um simples ajuste de limiar, validando a *pipeline* como ferramenta para revisitar curvas em busca de descobertas.

4. **Utilidade operacional:** o procedimento — leitura do `.dat`, limpeza, recorte opcional e inferência — é executado em segundos e pode ser incorporado a campanhas reais com modificação mínima.

Como limitação, vale registrar que se trata de um $n$ pequeno (três curvas, todas do mesmo evento) e que os modelos atuais não estão otimizados para detectar separadamente os eventos de anel: o ajuste de limiar resolve o caso prático, mas não substitui melhorias

estruturais como janela deslizante, *features* locais ou classificação multi-classe. Uma avaliação sistemática nesse regime — inclusive incorporando dados de campanhas similares no banco de treino — é uma das direções recomendadas no Capítulo 6.

## 5.10 Comparação entre os experimentos

A Tabela 5.10 permite uma visão comparativa do F1-score por modelo nos seis experimentos realizados.

Tabela 5.10: Comparação do F1-score entre os seis experimentos, por modelo. Exp. 1: 28 *features*; Exp. 2: 11 *features* (ablação combinada); Exp. 3: teste somente em curvas reais (198 curvas: 160 positivas e 38 negativas; ~~sintéticas~~ simuladas alocadas ao treino); Exp. 4: 14 *features*; Exp. 5: 13 *features*; Exp. 6: 12 *features* (sem `kmeans`).

| **Modelo** | **Exp. 1** 28 f. 80/20 | **Exp. 2** 11 f. 80/20 | **Exp. 3** teste real 80/20 | **Exp. 4** 14 f. 80/20 | **Exp. 5** 13 f. 80/20 | **Exp. 6** 12 f. 80/20 |
|---|---|---|---|---|---|---|
| Random Forest | **0,9937** | 0,9874 | 0,9874 | 0,9874 | **0,9905** | **0,9906** |
| XGBoost | 0,9844 | **0,9906** | **0,9906** | 0,9875 | 0,9843 | **0,9906** |
| CatBoost | 0,9906 | 0,9905 | 0,9874 | **0,9906** | 0,9873 | 0,9874 |
| Regressão Log. | 0,9873 | 0,9841 | 0,9778 | 0,9873 | 0,9873 | 0,9873 |

Cinco observações principais emergem da análise conjunta:

1. **Estabilidade entre conjuntos de *features*.** A redução de 28 (Experimento 1) para 14 *features* (Experimento 4) preserva o desempenho, com F1 entre 0,98 e 0,99.

2. **Impacto da remoção de `Feature_Savgol_Min`.** No Experimento 5 (13 *features*), o Random Forest lidera com F1 = 0,9905; a remoção dessa *feature* não causa degradação acentuada.

3. **Dispensabilidade do ~~K-Means~~K-means.** Nos Experimentos 6 e 2, a remoção de `kmeans_centroid_dist` (e, no Exp. 2, também de `Feature_Savgol_Min`) não degrada o desempenho: XGBoost e CatBoost mantêm F1 $\geqslant$ 0,990. A colinearidade funcional com `Feature_Savgol_std` explica essa redundância.

4. **Generalização em curvas reais.** O Experimento 3 (teste somente em curvas reais) preserva o melhor F1 do teste misto (XGBoost, 0,9906) e mantém AUC-ROC $\geqslant$ 0,9965, indicando que as *features* capturam sinal real e não apenas artefatos da distribuição ~~sintéticas~~simulada. A queda de ~~≈2 pp observada~~ 0,7 pp na variante 65/35 (Apêndice A) ~~decorre, em parte, da~~ é compatível com a flutuação entre partições; atribuí-la à redução do ~~conjunto de~~ treino é uma hipótese, pois a variante também altera o conjunto de teste.

5. **AUC-ROC.** Em todos os experimentos com teste misto (todos exceto o Experimento 3), a AUC-ROC permanece $\geqslant$ 0,9995; no Experimento 3, $\geqslant$ 0,9965.

A Figura 5.6 apresenta as curvas ROC da linha de base em dados mistos (Experimento 1), com separação quase perfeita entre as classes. As curvas ROC dos demais experimentos — incluindo a variante 65/35 do Experimento 3 (Figura A.6) — estão reunidas no Apêndice A.

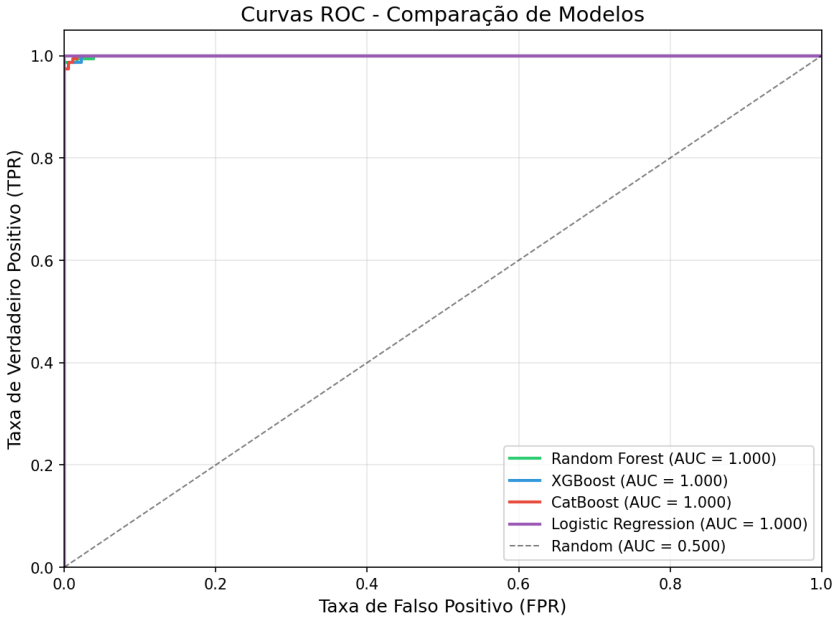



Figura 5.6: Curvas ROC — Experimento 1 (28 *features*).

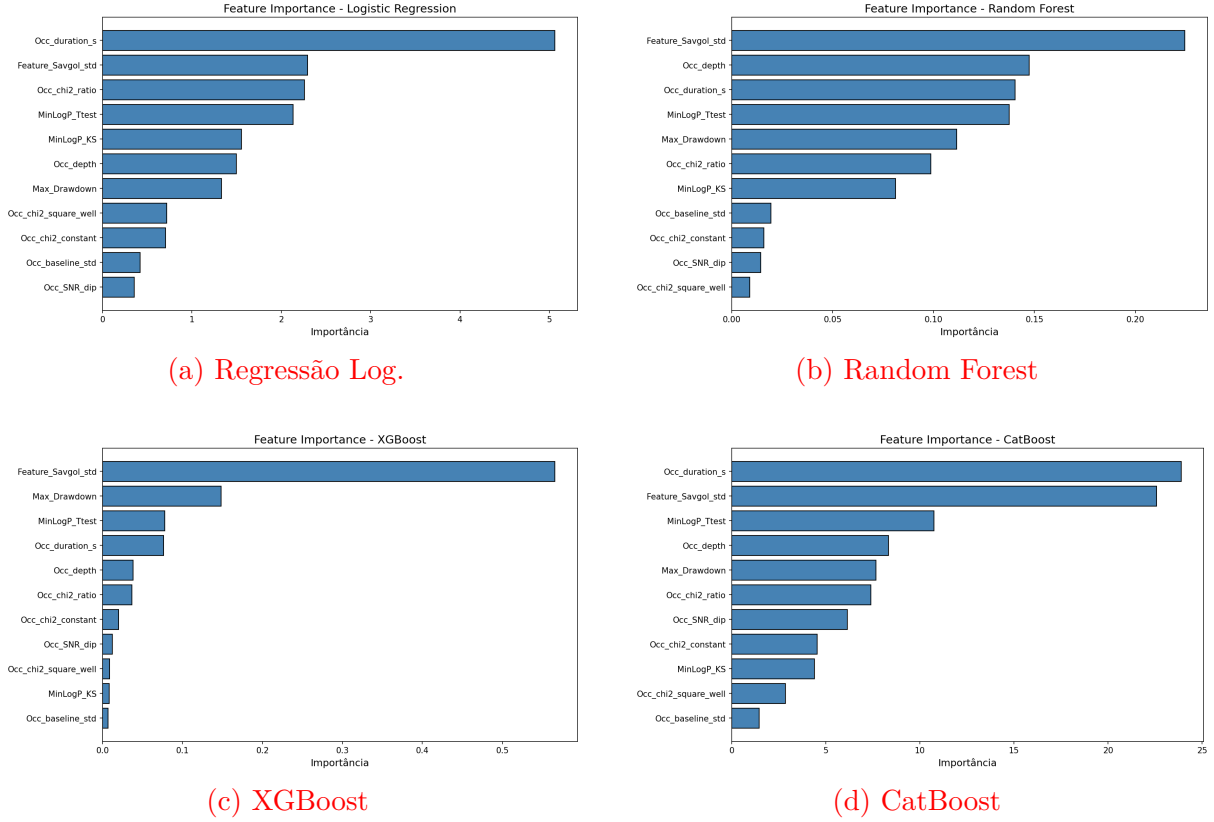
### 5.10.1 Análise de importância das *features*

A Figura 5.7 apresenta os gráficos de importância de variáveis para os quatro modelos no Experimento 2 (configuração enxuta, 11 *features* úteis após a remoção das redundantes); concentramos a análise nesta configuração porque são justamente essas variáveis que permanecem em uso na *pipeline* final. As figuras análogas das demais configurações estão no Apêndice A. A interpretação dos valores deve considerar as diferenças metodológicas discutidas na Seção 5.3: cada modelo emprega uma métrica própria — MDI normalizado para soma 1 (Random Forest), *weight* normalizado (XGBoost), *PredictionValuesChange* normalizado para soma 100 (CatBoost) e $|\beta_j|$–$|\theta_j|$ sem normalização (Regressão Logística) —, de modo que as escalas horizontais não são diretamente comparáveis entre os painéis.

Nos modelos de árvore (Random Forest e XGBoost), as variáveis mais relevantes são consistentemente:

- `Feature_Savgol_std`: a *feature* dominante em ambos os modelos — isolada, responde por ≈0,57 da importância no XGBoost. Mede a dispersão do fluxo suavizado, que cresce quando coexistem dois patamares (dentro e fora do *dip*). Com a remoção de `kmeans_centroid_dist` (Experimento 2), ela absorve o papel discriminativo antes atribuído àquela variável colinear, como antecipado pelo estudo de ablação.

**Experimento 2 (11 *features* úteis).**




(a) Regressão Log.

(b) Random Forest

(c) XGBoost

(d) CatBoost

Figura 5.7: Importância de variáveis por modelo — Experimento 2 (configuração enxuta, 11 *features*). (a) Regressão Logística, (b) Random Forest, (c) XGBoost, (d) CatBoost.

- `Max_Drawdown`: captura diretamente a magnitude da maior queda acumulada de fluxo, assinatura primária de uma ocultação.
- `Occ_depth` e `Occ_duration_s`: quantificam a profundidade e a duração do evento segundo critérios observacionais.
- `MinLogP_Ttest` (e, em menor grau, `Occ_chi2_ratio` e `MinLogP_KS`): os testes estatísticos entre quartis detectam a heterogeneidade no fluxo esperada em curvas com ocultação.

~~**Experimento 2 (11 *features* úteis).**(a) Regressão Log. (b) Random Forest (c) XGBoost (d) CatBoost Importância de variáveis por modelo — Experimento 2 (configuração enxuta, 11 *features*). (a) Regressão Logística, (b) Random Forest, (c) XGBoost, (d) CatBoost.~~

As figuras análogas das configurações com conjuntos maiores de *features* (Experimentos 4 e 5), que permitem acompanhar como a importância relativa das variáveis se redistribui à medida que o conjunto é reduzido, estão reunidas no Apêndice A.

Naquelas configurações, a *feature* `kmeans_centroid_dist` figura entre as mais importantes, e isso não é um artefato dos modelos: a análise exploratória dos dados confirma que ela é, por si só, discriminativa. A Figura 5.8 mostra que as curvas negativas se concentram em valores baixos de distância entre centróides — coerente com um fluxo de regime único, em que o ~~K-Means~~ K-means não encontra dois agrupamentos distintos —, enquanto as curvas positivas se espalham por valores altos, refletindo a coexistência dos dois patamares (dentro e fora do *dip*). É justamente por parecer tão promissora que o resultado do estudo de ablação (sua remoção não degrada o desempenho, pois `Feature_Savgol_std` é colinear e a substitui Seção A.3) se torna um achado relevante, e não uma simplificação trivial.

## 5.11 Validação cruzada estratificada

Os resultados das seções anteriores baseiam-se em uma única divisão treino/teste por experimento. Isso é útil para comparar modelos sob as mesmas condições, mas não diz o quanto o desempenho *varia* se mudarmos a divisão dos dados. Para medir essa variabilidade, repetimos a avaliação cinco vezes, com partições diferentes — procedimento conhecido como **validação cruzada com ~~$k = 5$~~ $q = 5$ dobras** (*k-fold*).

O procedimento é simples. Primeiro, embaralha-se o conjunto e divide-se em 5 partes (as "dobras") de tamanho parecido. Em seguida, faz-se cinco rodadas: em cada uma, quatro dobras servem de treino e a quinta de teste, alternando qual dobra é a de teste, de modo que toda curva é avaliada exatamente uma vez. As partições são **estratificadas** (cada dobra mantém a mesma proporção entre as duas classes, positiva e negativa —

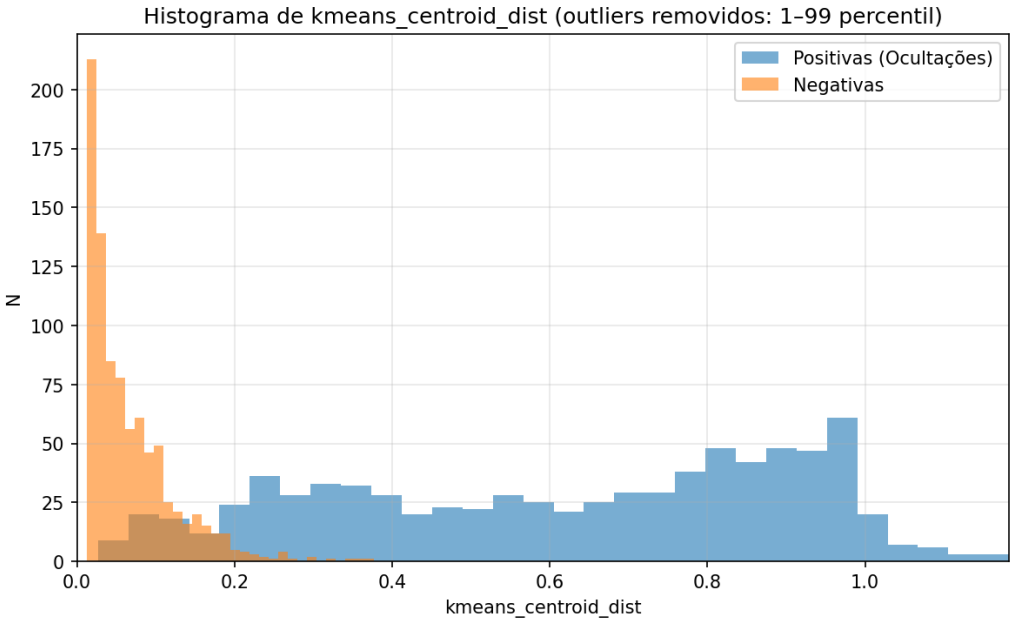



Figura 5.8: Distribuição da *feature* `kmeans_centroid_dist` por classe (1–99 percentil). As curvas negativas concentram-se em valores baixos (regime único) e as positivas em valores altos (dois patamares), evidenciando o poder discriminativo da variável.

continua sendo um problema binário) e respeitam o **agrupamento por curva** (todos os segmentos recortados de uma mesma curva ficam na mesma dobra, para que o modelo nunca seja testado em um pedaço de uma curva que viu no treino, o que inflaria artificialmente o desempenho). Em cada rodada, os quatro modelos são treinados e avaliados de forma independente, registrando-se F1-score, precisão, sensibilidade e AUC-ROC. O resultado final é a **média ± desvio padrão** dessas métricas sobre as cinco dobras: a média estima o desempenho típico e o desvio padrão, a sua variabilidade.

A Tabela 5.11 apresenta a média e o desvio padrão das métricas na validação cruzada estratificada ~~5-fold~~ *5-fold* (Experimento 5, 13 *features*; detalhado no Apêndice A).

Tabela 5.11: Validação cruzada estratificada (~~$k = 5$~~$q = 5$): média ± desvio padrão das métricas por modelo (Experimento 5).

| Modelo | Acurácia | Precisão | Sensibilidade | F1 | AUC-ROC |
|---|---|---|---|---|---|
| CatBoost | 0,9876 ± 0,0067 | 0,9912 ± 0,0034 | 0,9825 ± 0,0148 | 0,9868 ± 0,0072 | 0,9993 ± 0,0005 |
| XGBoost | 0,9864 ± 0,0058 | 0,9888 ± 0,0090 | 0,9825 ± 0,0167 | 0,9856 ± 0,0062 | 0,9992 ± 0,0008 |
| Regressão Log. | 0,9846 ± 0,0097 | 0,9911 ± 0,0057 | 0,9763 ± 0,0172 | 0,9836 ± 0,0104 | 0,9986 ± 0,0016 |
| Random Forest | 0,9840 ± 0,0068 | 0,9826 ± 0,0051 | 0,9838 ± 0,0129 | 0,9831 ± 0,0072 | 0,9992 ± 0,0007 |

A validação cruzada cumpre dois objetivos: (1) confirmar que o desempenho elevado observado nos experimentos anteriores não é artefato de uma partição favorável dos dados; e (2) fornecer desvios padrão que permitam avaliar se as diferenças entre modelos são significativas ou estão dentro da flutuação estatística. ~~Se o~~ O desvio padrão do F1 ~~for pequeno (e.g., < 0,01)~~ entre as dobras vai de 0,~~isso reforça~~ 0062 (XGBoost) a 0,0104 (Re-

gressão Logística) (Tabela 5.11). As diferenças entre as médias dos quatro modelos (no máximo 0,0037) são menores que um desvio padrão; a ~~conclusão de que o desempenho va~~lidação cruzada, portanto, não distingue os classificadores, o que é ~~robusto; se for grande, pode indicar sensibilidade à composição do conjunto de teste~~ coerente com o teste de McNemar (Seção 5.11.1).

### 5.11.1 Teste de McNemar entre modelos

Para determinar se as diferenças de desempenho entre pares de modelos são **estatisticamente significativas** — e não meras flutuações amostrais — aplicou-se o **teste de McNemar** (McNemar , 1947), cujo uso adequado em comparação de classificadores é discutido por Smith e Ruxton (2020). Esse teste é apropriado para comparar dois classificadores avaliados no mesmo conjunto de teste: constrói-se uma tabela $2 \times 2$ de concordância/discordância entre as predições dos dois modelos, e testa-se se os padrões de discordância são simétricos.

Sejam $b$ o número de exemplos que o modelo A erra e o modelo B acerta, e $c$ o número de exemplos que o modelo A acerta e o modelo B erra. A estatística de McNemar (com correção de continuidade) é:

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c}, \tag{5.1}$$

que segue uma distribuição $\chi^2$ com 1 grau de liberdade sob a hipótese nula $H_0 : b = c$ (os modelos têm a mesma taxa de erro). Se $p < 0{,}05$, rejeita-se $H_0$ e conclui-se que a diferença é significativa.

A Tabela 5.12 apresenta os resultados do teste de McNemar entre pares de modelos (Experimento 4, 14 *features*; detalhado no Apêndice A).

Tabela 5.12: Teste de McNemar entre pares de modelos (Experimento 4). $b$: apenas A acerta; $c$: apenas B acerta. $p < 0{,}05$ indicaria diferença significativa; nenhum par é significativo ($\alpha = 0{,}05$).

| **Modelo A** | **Modelo B** | $b$ | $c$ | $\chi^2$ | $p$ |
|---|---|---|---|---|---|
| RF | XGBoost | 1 | 1 | 0,50 | 0,4795 |
| RF | CatBoost | 1 | 2 | 0,00 | 1,0000 |
| RF | Reg. Log. | 2 | 2 | 0,25 | 0,6171 |
| XGBoost | CatBoost | 0 | 1 | 0,00 | 1,0000 |
| XGBoost | Reg. Log. | 3 | 3 | 0,17 | 0,6831 |
| CatBoost | Reg. Log. | 3 | 2 | 0,00 | 1,0000 |

**Conclusão:** Nenhum par de modelos apresenta diferença estatisticamente significativa; os quatro classificadores podem ser considerados equivalentes no conjunto de teste deste experimento.

Em cenários de alto desempenho (F1 $> 0{,}97$), é comum que os valores de $b$ e $c$ sejam pequenos, o que pode tornar o teste pouco potente. Se o McNemar indicar que as di-

ferenças **não** são significativas para a maioria dos pares, isso sugere que os modelos são equivalentes no contexto deste *dataset* e que a escolha entre eles pode ser guiada por critérios secundários (interpretabilidade, velocidade de inferência, facilidade de implantação, etc.).

## 5.12 Curvas de aprendizado dos modelos de *boosting*

A Seção 3.6.4 (Capítulo 3) apresentou as curvas de aprendizado como ferramenta gráfica para diagnosticar sobreajuste: acompanha-se o erro em treino e em validação em função do tamanho do conjunto de treino ou do número de épocas. Aplicamos aqui a segunda variante aos dois modelos de *boosting* da *pipeline*, registrando ~~o~~ a perda logarítmica (*logloss*, isto é, a entropia cruzada da Seção 3.3) de treino e de teste a cada iteração, na configuração enxuta do Experimento 2 (11 *features*, divisão 80%/20% por curva). A Figura 5.9 apresenta os resultados.

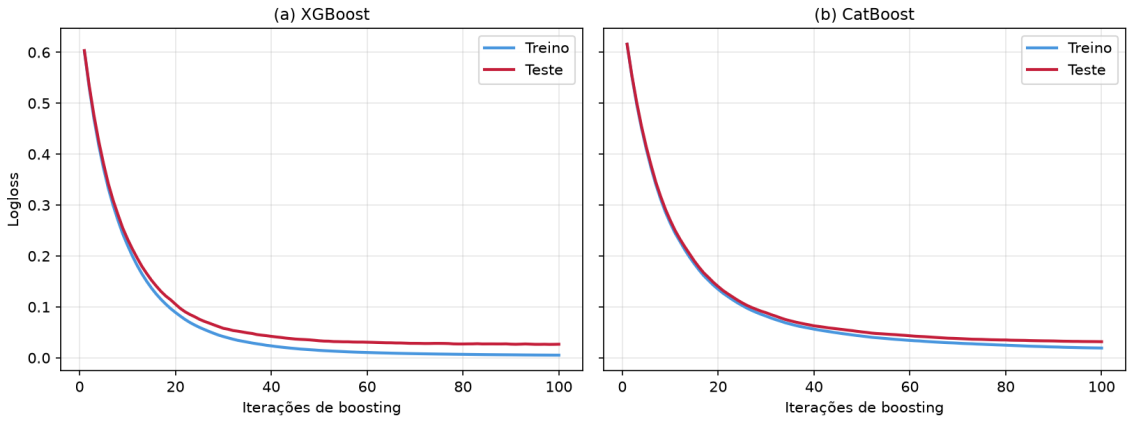



Figura 5.9: Curvas de aprendizado de (a) XGBoost e (b) CatBoost na configuração do Experimento 2: *logloss* de treino (azul) e de teste (vermelho) em função das iterações de *boosting*.

Dois padrões merecem registro. Primeiro, a perda de teste acompanha a de treino do início ao fim e estabiliza por volta da iteração 50–60, sem em nenhum momento inverter a tendência e voltar a crescer. Essa inversão seria a assinatura clássica de sobreajuste em *boosting*; sua ausência indica que as 100 iterações adotadas param antes da região de risco. Segundo, os *gaps* finais entre teste e treino são pequenos: 0,0215 no XGBoost (treino 0,0052; teste 0,0267) e 0,0125 no CatBoost (treino 0,0192; teste 0,0317). O CatBoost encerra com o menor *gap*, comportamento coerente com sua regularização estrutural (*ordered boosting* e árvores *oblivious*, Capítulo 3), enquanto o XGBoost desce a perda de treino mais fundo, o que ajuda a explicar suas probabilidades mais polarizadas no estudo de caso de Quaoar (Seção 5.9). Em conjunto com a ablação do tamanho do treino (Apên-

dice A), que cobre a outra dimensão da curva de aprendizado, esses resultados completam o diagnóstico gráfico de generalização introduzido no Capítulo 3.

## 5.13 Discussão dos resultados

### 5.13.1 Desempenho e generalização

Os resultados dos seis experimentos indicam que a *pipeline* de detecção automatizada atinge **desempenho elevado e estável** na classificação binária de curvas de luz: F1-score entre 0,974 e 0,994 no teste misto (todos os experimentos, exceto o Experimento 3) e entre 0,975 e 0,982 no teste exclusivamente em curvas reais (Experimento 3); AUC-ROC superior a 0,995 em todos os cenários. Esse patamar é compatível com o objetivo de desenvolver uma ferramenta de triagem que permita priorizar curvas candidatas a análise detalhada.

**Sobre a possibilidade de *overfitting*.** Todas as métricas referem-se ao **conjunto de teste**; portanto, o desempenho descrito é uma estimativa da capacidade de generalização. Vários indícios sugerem que o *overfitting* não domina os resultados: (i) as métricas são estáveis entre os seis experimentos e quatro modelos; (ii) a redução de 28 para 14 ou 11 *features* preserva ou melhora ligeiramente o F1; (iii) foram empregados mecanismos de regularização (Ridge na Regressão Logística, `class_weight='balanced'`, limites de profundidade nos *ensembles*); (iv) a validação cruzada ~~5-fold~~ *5-fold* (Seção 5.11) apresenta desvios padrão pequenos (≈0,006–0,010 em F1); (v) as curvas de aprendizado (Seção 5.12) mostram o *logloss* de teste convergindo junto ao de treino, sem divergência ao longo das iterações. A confirmação definitiva de boa generalização exigiria validação em dados independentes de outros catálogos.

### 5.13.2 Análise de redundância

A análise de redundância (Seção 5.2) identificou 15 variáveis redundantes ou com baixo poder discriminativo. A comparação entre os Experimentos 1 (28 *features*) e 4 (14 *features*) demonstra empiricamente que a remoção dessas variáveis não prejudica — e pode até melhorar — o desempenho dos classificadores. Isso é consistente com a teoria de *machine learning*: variáveis redundantes aumentam a dimensionalidade do espaço de entrada sem acrescentar informação, podendo elevar a variância dos modelos.

A predominância da *feature* `Feature_Savgol_std` no conjunto enxuto (Figura 5.7) é coerente com a física do problema: em uma ocultação, o fluxo normalizado cai durante o intervalo em que a estrela está atrás do corpo, fazendo coexistirem dois patamares (dentro e fora do *dip*); a dispersão da série suavizada cresce nessa situação, ao passo que em curvas ruidosas sem evento o fluxo é aproximadamente homogêneo e a dispersão é baixa. Variáveis correlatas que mediam o mesmo efeito por outra via — como o mínimo da

série suavizada (`Feature_Savgol_Min`) — mostraram-se redundantes e foram descartadas sem prejuízo de desempenho.

### 5.13.3 Interpretação da importância das *features*

A concordância entre os quatro modelos quanto às *features* mais importantes do conjunto enxuto — `Feature_Savgol_std`, `Max_Drawdown` e as métricas observacionais de profundidade e duração (`Occ_depth`, `Occ_duration_s`) — oferece confiança de que essas variáveis capturam as assinaturas físicas relevantes de uma ocultação: a magnitude da queda de fluxo, sua localização temporal e a separação entre os níveis de fluxo dentro e fora do evento. A coerência entre modelos de naturezas distintas (linear vs. árvores, *bagging* vs. *boosting*) reforça a robustez dessa interpretação.

O estudo de ablação (Seção 5.6) acrescenta uma nuance importante: embora `kmeans_centroid_dist` seja consistentemente apontada como a *feature* mais importante pelos modelos de árvore — chegando a 63% de importância no XGBoost —, sua remoção não degrada o desempenho. Isso demonstra que a **importância atribuída por algoritmos baseados em árvore não é sinônimo de insubstituibilidade**: uma *feature* pode concentrar alta importância simplesmente por oferecer um ponto de corte mais eficiente, enquanto outras variáveis colineares podem compensar sua ausência redistribuindo a capacidade preditiva. Esse resultado tem implicação prática direta: ~~o~~ a *pipeline* pode ser ~~simplificado~~ simplificada pela remoção do passo de ~~K-Means~~ K-means sem perda de qualidade.

### 5.13.4 Custo assimétrico e ajuste de limiar

Em um cenário real de triagem, o custo de perder uma ocultação genuína (falso negativo) supera amplamente o custo de revisar uma curva sem evento (falso positivo). A Seção 5.8 demonstra que essa assimetria pode ser incorporada à *pipeline* sem retreinamento, simplesmente reduzindo o limiar de decisão $\tau$. Essa abordagem explora o fato de que os modelos produzem probabilidades, não apenas rótulos binários, e que a separabilidade elevada entre as classes (AUC-ROC $\geqslant 0{,}998$) garante que a precisão se mantenha aceitável mesmo em patamares de sensibilidade próximos de 100%. A possibilidade de ajuste de limiar em pós-processamento é uma vantagem direta da formulação probabilística adotada, e confere à *pipeline* flexibilidade para operar em diferentes cenários de custo sem modificação dos modelos.

As limitações deste trabalho e as direções de pesquisa que delas decorrem são sintetizadas no Capítulo 6, à luz do conjunto completo dos resultados.

# Capítulo 6

# Conclusões

## 6.1 Resumo dos achados

A *pipeline* desenvolvida neste trabalho demonstrou que a detecção automatizada de ocultações estelares em curvas de luz pode atingir desempenho elevado (F1-score entre 0,978 e 0,994; AUC-ROC superior a 0,995), utilizando *features* clássicas extraídas das séries temporais e *ensembles* de árvores de decisão (Random Forest, XGBoost, CatBoost), além de Regressão Logística como *baseline*.

Os seis experimentos realizados permitiram identificar os seguintes padrões:

1. **O *feature engineering* baseado em domínio é o fator determinante da qualidade da classificação.** A redução de 28 ~~para 13–14~~ (Tabela 5.1) para 11–13 *features* (Tabela 5.2), com a remoção de variáveis redundantes~~),~~, não degradou o desempenho (em alguns casos, até o melhorou), indicando que um conjunto compacto de variáveis fisicamente motivadas é suficiente.

2. **A *feature* `kmeans_centroid_dist` é dispensável apesar de sua alta importância atribuída pelos modelos de árvore.** O estudo de ablação (Experimento 6) mostrou que a remoção dessa variável causa degradação desprezível ($\leqslant$0,3 pp em F1), demonstrando que importância de *feature* não é sinônimo de insubstituibilidade: outras variáveis colineares (sobretudo `Feature_Savgol_std`) compensam sua ausência.

3. **O ajuste do limiar de decisão permite operar a *pipeline* em regime de alta sensibilidade** (minimização de falsos negativos) sem retreinamento, explorando a separabilidade elevada entre as classes. O estudo de caso de Quaoar (Seção 5.9) demonstrou, em dados reais, que essa alavanca recupera anéis finos que passariam despercebidos no limiar padrão.

Como verificação complementar, as 67 curvas do banco com Occ_SNR_dip $<$ 3 foram submetidas aos ~~modelos treinados~~quatro modelos treinados do Experimento 2 (11

*features*); todas foram corretamente classificadas como positivas pelos três *ensembles* de árvores (67/67), e 66 das 67 pela Regressão Logística. Embora essas curvas tenham participado do treinamento e o teste não inclua negativos de baixo SNR, o resultado indica que os modelos não descartam sistematicamente eventos ruidosos. Essa é uma condição necessária para a aplicabilidade da *pipeline* em cenários de baixa relação sinal-ruído.

## 6.2 Resposta à hipótese de pesquisa

A hipótese formulada na Introdução — de que é possível obter desempenho comparável a abordagens baseadas em redes convolucionais ou inspeção manual usando *features* clássicas e *ensembles* de árvores, com vantagem de interpretabilidade — é **corroborada** pelos resultados obtidos. Os quatro modelos testados atingiram desempenho elevado e consistente; a concordância entre modelos de naturezas distintas (linear vs. árvores, *bagging* vs. *boosting*) quanto à importância das variáveis reforça a interpretabilidade do sistema e a solidez das conclusões.

## 6.3 Contribuições originais

As principais contribuições deste trabalho são:

1. ~~**Pipeline**~~ ***Pipeline*** **completa de detecção:** desde a ingestão de curvas de luz já reduzidas até a classificação binária com reportagem de métricas, importância de variáveis e predições por curva, implementada em Python e disponível para reprodução.

2. **Conjunto de *features* baseado em domínio:** ~~13–14~~ 11–13 variáveis que capturam assinaturas físicas de ocultações (estatísticas de Savitzky-Golay, *Max Drawdown*, métricas observacionais de profundidade e SNR, testes estatísticos entre quartis), selecionadas por análise de redundância e validadas experimentalmente.

3. **Estudo de ablação da *feature*** ~~**K-Means**~~**K-means:** demonstração empírica de que a importância atribuída por algoritmos de árvore não implica insubstituibilidade, com implicação prática para simplificação ~~do~~ da *pipeline*.

4. **Análise de curvas de aprendizado:** evidência de saturação em suas duas dimensões, a do volume de treino (ablação 65/35, Apêndice A) e a da convergência do *logloss* por iteração de *boosting* (Seção 5.12), orientando futuras decisões sobre coleta de dados.

5. **Análise de ajuste de limiar:** demonstração de que a assimetria de custo entre falsos negativos e falsos positivos pode ser incorporada em pós-processamento, sem retreinamento.

6. **Validação cruzada estratificada e teste de McNemar:** estimativa robusta da variabilidade do desempenho (desvio padrão pequeno entre dobras) e avaliação da significância estatística das diferenças entre modelos, que se mostraram equivalentes no conjunto de teste.

7. **Demonstração da capacidade de (re)visitar curvas em busca de eventos sutis:** o estudo de caso de Quaoar (Seção 5.9) mostrou, em dados reais externos ao treino, que o modelo separa um anel fino verdadeiro de um trecho de ruído semelhante por quase duas ordens de grandeza de probabilidade. Esse contraste, via ajuste de limiar, permite recuperar eventos que escapariam à classificação binária padrão.

## 6.4 Limitações

As limitações deste trabalho devem ser explicitadas para contextualizar adequadamente os resultados:

1. ***Dataset* parcialmente ~~sintéticos~~simulado:** das 891 curvas negativas, 702 são ~~sintéticas~~ simuladas e 186 são negativos por recorte (segmentos reais extraídos de curvas positivas); apenas 3 são detecções negativas nativas. Embora as curvas ~~sintéticas~~ simuladas tenham sido geradas com base em propriedades estatísticas de curvas reais, a capacidade de generalização para um cenário puramente observacional não foi validada.

2. ***Split* único por experimento:** cada experimento utiliza uma única partição treino/teste. A implementação de validação cruzada estratificada (Seção 5.11) mitiga parcialmente essa limitação; uma validação cruzada repetida com múltiplas sementes fortaleceria as conclusões.

3. **Ausência de validação em catálogos externos:** as curvas reais utilizadas neste trabalho provêm de duas fontes distintas: o catálogo `B/occ/asteroid` do VizieR (agregador público que reúne campanhas independentes envolvendo múltiplos observadores, telescópios e corpos) e o banco do Grupo do Rio (campanhas específicas de Umbriel e Chiron). Embora a heterogeneidade interna do VizieR já garanta diversidade observacional considerável, a generalização da *pipeline* para curvas oriundas de outros agregadores ou de levantamentos específicos não foi avaliada sistematicamente. Apesar disso, o ~~script~~ *script* utilizado para popular o banco SQL com dados do Grupo do Rio serve para qualquer tipo de curvas de luz, contanto que estejam em formato `.dat` de texto, com colunas separadas por espaço, a primeira coluna com o tempo e a segunda com o fluxo (uma terceira coluna, com a incerteza fotométrica, é lida mas não utilizada).

4. **Hiperparâmetros fixos:** os hiperparâmetros dos modelos foram definidos manualmente com base em valores recomendados pela literatura; não foi realizada busca em grade ou otimização bayesiana.

5. **Predominância de curvas com ocultações bem definidas:** o elevado desempenho dos modelos reflete, em parte, a predominância de curvas com ocultações claras (alto ~~S/N~~SNR, queda pronunciada) no conjunto de dados. Para curvas com baixo ~~S/N~~ SNR ou eventos rasantes, a tarefa de classificação é substancialmente mais difícil e o desempenho real da *pipeline* nessas condições ainda não foi avaliado sistematicamente.

6. **Barras de erro fotométricas não utilizadas:** as incertezas fotométricas de cada ponto não são armazenadas no banco nem usadas nas *features*. As escalas de ruído (por exemplo, no $\chi^2$ e no SNR do *dip*) são estimadas da dispersão da própria curva. Incorporá-las permitiria um $\chi^2$ propriamente dito e uma comparação mais justa entre curvas de qualidades diferentes.

## 6.5 Trabalhos futuros

Com base nos resultados e limitações identificados, sugerem-se as seguintes direções para trabalhos futuros:

- **Validação cruzada repetida:** executar a validação cruzada com múltiplas sementes (e.g., 10 repetições × 5 dobras) para obter intervalos de confiança robustos das métricas.

- **Experimento *real holdout*:** reservar todas as curvas reais (positivas e negativas genuínas) para o conjunto de teste, treinando apenas com curvas ~~sintéticas~~ simuladas e negativos por recorte, para avaliar a transferibilidade do modelo.

- ***Features* baseadas em difração de Fresnel:** incorporar características derivadas de modelos físicos mais sofisticados (escala de Fresnel, opacidade parcial, diâmetro aparente da estrela, etc.), que poderiam melhorar a discriminação em eventos de baixo SNR.

- **Detecção em regime de baixo ~~S/N~~SNR:** construir um subconjunto de teste composto exclusivamente por curvas difíceis (baixo ~~S/N~~SNR, ocultações rasantes, eventos com poucos pontos no *dip*, etc.) para avaliar o desempenho da *pipeline* nas condições em que a triagem automática é de fato necessária, isto é, onde a inspeção visual é ambígua.

- **Janela deslizante e *features* locais:** o estudo de recortes de Quaoar (Seção 5.9) mostrou que o ajuste de limiar resolve o caso prático de detecção de anéis finos, mas que a causa estrutural da baixa probabilidade absoluta atribuída a esses eventos é o caráter **global** das *features* (estatísticas agregadas sobre toda a janela diluem um *dip* de poucos segundos). Duas direções complementares atacam essa raiz: (i) aplicar a *pipeline* por **janela deslizante** de largura compatível com a duração típica do evento alvo, tornando o conjunto atual de *features* sensível a eventos locais; e (ii) enriquecer o conjunto com ***features* locais** (profundidade máxima de qualquer subjanela de $N$ pontos abaixo do *baseline*, simetria do perfil de ingresso/egresso, ajuste a poço retangular curto).

- **Classificação multi-classe de eventos:** estender a formulação binária (ocultação presente/ausente) para uma classificação multi-classe que distinga o **tipo** de evento observado: ocultação por corpo sólido, assinatura atmosférica (refração e absorção), detecção de anéis e padrões de difração de Fresnel. Essa extensão exigiria *features* adicionais (por exemplo, simetria do perfil de ingresso/egresso, profundidade relativa de quedas secundárias e ajuste a modelos de difração), além de um *dataset* curado com rótulos para cada categoria, possivelmente com apoio de curvas ~~sintéticas~~ geradas por simuladores que incluam esses fenômenos.

# Referências Bibliográficas


GÉRON, Aurélin. *Mãos à Obra: Aprendizado de Máquina com Scikit-Learn e TensorFlow*. 2. ed. Rio de Janeiro: Alta Books, 2019.

LEVADA, Alexandre L. M. *Manual completo para aprendizado de máquina: algoritmos para classificação supervisionada, agrupamento de dados e redução de dimensionalidade*. Departamento de Computação, Universidade Federal de São Carlos (UFSCar), 16 jan. 2026.

CAZENEUVE, Dorian; MARCHIS, Franck; BLACLARD, Guillaume; DALBA, Paul A.; MARTIN, Victor; ASENCIO, Joe. "ODNet: A Convolutional Neural Network for Asteroid Occultation Detection". *The Astronomical Journal*, 165(11), 2023.

WALL, J. V.; JENKINS, C. R. *Practical Statistics for Astronomers*. Cambridge University Press, 2003.

MORETTIN, Pedro A.; BUSSAB, Wilton de O. *Estatística Básica*. 6. ed. Revista e atualizada. São Paulo: Saraiva, 2010.

DEISENROTH, Marc Peter; FAISAL, A. Aldo; ONG, Cheng Soon. *Mathematics For Machine Learning*. Cambridge University Press, 2019. (Draft, October 15, 2019).

KNIELING, Bastian; SCHINDLER, Karsten; SICKAFOOSE, Amanda A.; PERSON, Michael J.; LEVINE, Stephen E.; KRABBE, Alfred. "Stellar Occultations in the Era of Data Mining and Modern Regression Models: Using Gaussian Processes to Analyze Light Curves and Improve Predictions". *The Planetary Science Journal*, 5(104), 2024. `https://doi.org/10.3847/PSJ/ad3819` [1]

FRASER, Wesley C.; PORTER, Simon B.; PELTIER, Lowell; KAVELAARS, J. J.; VERBISCER, Anne J.; et al. "Candidate Distant Trans-Neptunian Objects Detected by the New Horizons Subaru TNO Survey". *arXiv e-prints*, arXiv:2407.21142v1, 2024.

GOMES-JÚNIOR, A. R.; MORGADO, B. E.; BENEDETTI-ROSSI, G.; BOUFLEUR, R. C.; ROMMEL, F. L.; BANDA-HUARCA, M. V.; KILIC, Y.; BRAGA-RIBAS, F.; SICARDY, B. "SORA: Stellar occultation reduction and analysis".

*Monthly Notices of the Royal Astronomical Society*, 511(1), 1167–1181, 2022. https://doi.org/10.1093/mnras/stac032.

ROQUES, F.; MONCUQUET, M.; SICARDY, B. "Stellar occultations by small bodies - Diffraction effects". *The Astronomical Journal*, 93, 1549–1558, 1987. https://doi.org/10.1086/114438.

SICARDY, B.; ORTIZ, J. L.; ASSAFIN, M.; et al. "A Pluto-like radius and a high albedo for the dwarf planet Eris from an occultation". *Nature*, 478(7370), 493–496, 2011. https://doi.org/10.1038/nature10550.

SICARDY, B.; TALBOT, J.; MEZA, E.; CAMARGO, J. I. B.; et al. "Pluto's atmosphere from the 2015 June 29 ground-based stellar occultation at the time of the New Horizons flyby". *The Astrophysical Journal Letters*, 819(2), L38, 2016. https://doi.org/10.3847/2041-8205/819/2/L38.

ASSAFIN, M. *Differential aperture photometry and digital coronagraphy with PRAIA*. Planetary and Space Science, v. 239, p. 105816, dez. 2023. DOI: 10.1016/j.pss.2023.105816.

BRAGA-RIBAS, F.; SICARDY, B.; ORTIZ, J. L.; SNODGRASS, C.; ROQUES, F.; et al. "A ring system detected around the Centaur (10199) Chariklo". *Nature*, 508(7494), 72–75, 2014. https://doi.org/10.1038/nature13155.

ASSAFIN, M.; MARTINS, R.; CAMARGO, J.; ANDREI, A.; NETO, D.; BRAGA-RIBAS, F. "PRAIA - Platform for Reduction of Astronomical Images Automatically". 85–88, 2011.

ASSAFIN, M.; SANTOS-FILHO, S.; MORGADO, B. E.; GOMES-JÚNIOR, A. R.; SICARDY, B.; MARGOTI, G.; BENEDETTI-ROSSI, G.; BRAGA-RIBAS, F.; LAIDLER, T.; CAMARGO, J. I. B.; VIEIRA-MARTINS, R.; et al. "Kilometer-precise (UII) Umbriel physical properties from the multichord stellar occultation on 2020 September 21". *Monthly Notices of the Royal Astronomical Society*, 526(4), 61936204, 2023. https://doi.org/10.1093/mnras/stad3093.

ORTIZ, J.; PEREIRA, Chrystian; SICARDY, Bruno; BRAGA-RIBAS, Felipe; TAKEY, Ali; FOUAD, A.; SHAKER, A.; KASPI, S. "Changing material around (2060) Chiron revealed by an occultation on December 15, 2022". *Astronomy & Astrophysics*, 676, 2023. https://doi.org/10.1051/0004-6361/202347025.

PEREIRA, C. L.; SICARDY, B.; MORGADO, B. E.; BRAGA-RIBAS, F.; FERNÁNDEZ-VALENZUELA, E.; SOUAMI, D.; HOLLER, B. J.; BOUFLEUR, R. C.; MARGOTI, G.; ASSAFIN, M.; ORTIZ, J. L.; et al. "The

two rings of (50000) Quaoar". *Astronomy & Astrophysics*, 673, L4, 2023. https://doi.org/10.1051/0004-6361/202346365.

MORGADO, B. E.; SICARDY, B.; BRAGA-RIBAS, F.; ORTIZ, J. L.; SALO, H.; VACHIER, F.; DESMARS, J.; PEREIRA, C. L.; SANTOS-SANZ, P.; SFAIR, R.; DE SANTANA, T.; et al. "A dense ring of the trans-Neptunian object Quaoar outside its Roche limit". *Nature*, 614(7947), 239–243, 2023. https://doi.org/10.1038/s41586-022-05629-6.

GOMES-FERRANTE, W.; BRAGA-RIBAS, F. "Stellar Occultation Simulator: application to Planet 9". *The European Physical Journal Special Topics*, v. 232, n. 18-19, p. 3113–3118, 2023. DOI: 10.1140/epjs/s11734-023-01031-z.

FORNASIER, Sonia; LELLOUCH, Emmanuel; MÜLLER, Thomas G.; SANTOS-SANZ, Pablo; PANUZZO, Pasquale; KISS, Csaba; LIM, Taeho; MOMMERT, Michael; BOCKELÉE-MORVAN, Dominique; VILENIUS, Essi; STANSBERRY, John; TOZZI, Gian Paolo; MOTTOLA, Stefano; DELSANTI, Audrey; CROVISIER, Jacques; DUFFARD, René; HENRY, Frédéric; LACERDA, Pedro; BARUCCI, Antonella; GICQUEL, Arnaud. *TNOs are Cool: A survey of the trans-Neptunian region VIII. Combined Herschel PACS and SPIRE observations of nine bright targets at 70–500 $\mu$m.* Astronomy & Astrophysics, v. 555, p. A15, jul. 2013. DOI: 10.1051/0004-6361/201321329.

GOMES, Rodney da Silva. *On the origin of the Kuiper belt*, 2009.

ZHU, Qinglin; SUN, Mingchen; DONG, Xiang; ZHU, Pengfei. "Design and Simulation of Stellar Occultation Infrared Band Constellation". *Remote Sens.*, 14(3327), 2022. https://doi.org/10.3390/rs14143327

MORBIDELLI, Alessandro; MARROCCHI, Yves; ALI AHMAD, Adnan; BHANDARE, Asmita; CHARNOZ, Sébastien; COMMERÇON, Benoît; DULLEMOND, Cornelis P.; GUILLOT, Tristan; HENNEBELLE, Patrick; LEE, Yueh-Ning; LOVASCIO, Francesco; MARSCHALL, Raphael; MARTY, Bernard; MAURY, Anaëlle; TAMAMI, Okamoto. *Formation and evolution of a protoplanetary disk: Combining observations, simulations, and cosmochemical constraints.* Astronomy & Astrophysics, v. 691, p. A147, nov. 2024. DOI: 10.1051/0004-6361/202451388.

LÓPEZ DE PRADO, Marcos M. *Machine Learning for Asset Managers.* Cambridge: Cambridge University Press, 2020. (Elements in Quantitative Finance).

CARLEO, Giuseppe; CIRAC, Ignacio; CRANMER, Kyle; DAUDET, Laurent; SCHULD, Maria; TISHBY, Naftali; VOGT-MARANTO, Leslie; ZDEBO-

ROVÁ, Lenka. *Machine learning and the physical sciences*, 2019. (Review paper excerpts)

SUDJIANTO, Agus; LIU, Alice J. "Decomposing Global AUC into Cluster-Level Contributions for Localized Model Diagnostics". *arXiv preprint*, arXiv:2508.07495, 2025. https://arxiv.org/abs/2508.07495.

GRYAK, J.; HARALICK, R. M.; KAHROBAEI, D. *Solving the Conjugacy Decision Problem via Machine Learning*. Experimental Mathematics, v. 29, n. 1, p. 6678, 2018. Disponível em: https://doi.org/10.1080/10586458.2018.1434704.

LIU, Ying. A Comparative Study on Feature Selection Methods for Drug Discovery. *Journal of Chemical Information and Computer Sciences*, v. 44, n. 5, p. 18231828, 2004. DOI: 10.1021/ci049875d.

SHEWCHUK, Jonathan. *Concise Machine Learning*. UC Berkeley, Class CS 189/289A Lecture Notes, 2024.

MacQUEEN, James B. "Some Methods for Classification and Analysis of Multivariate Observations". In: *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, v. 1, p. 281–297. Berkeley: University of California Press, 1967.

PEDREGOSA, F.; VAROQUAUX, G.; GRAMFORT, A.; MICHEL, V.; THIRION, B.; GRISEL, O.; BLONDEL, M.; PRETTENHOFER, P.; WEISS, R.; DUBOURG, V.; VANDERPLAS, J.; PASSOS, A.; COURNAPEAU, D.; BRUCHER, M.; PERROT, M.; DUCHESNAY, É. "Scikit-learn: Machine Learning in Python". *Journal of Machine Learning Research*, 12, p. 2825–2830, 2011. https://jmlr.org/papers/v12/pedregosa11a.html. API e documentação disponíveis em https://scikit-learn.org/stable/user_guide.html#user-guide.

McNEMAR, Quinn. "Note on the sampling error of the difference between correlated proportions or percentages". *Psychometrika*, 12, n. 2, p. 153–157, 1947. https://doi.org/10.1007/BF02295996.

SMITH, Matilda; RUXTON, Graeme. "Effective use of the McNemar test". *Behavioral Ecology and Sociobiology*, 74, 133, 2020. https://doi.org/10.1007/s00265-020-02916-y.

SAITO, Takaya; REHMSMEIER, Marc. "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets". *PLOS ONE*, 10(3), e0118432, 2015. DOI: 10.1371/journal.pone.0118432.

ORTIZ, J. L.; SICARDY, B.; CAMARGO, J. I. B.; SANTOS-SANZ, P.; BRAGA-RIBAS, F. "Stellar occultations by Trans-Neptunian objects: from predictions to observations and prospects for the future". In: PRIALNIK, D.; BARUCCI, M. A.; YOUNG, L. A. (Ed.). *The Trans-Neptunian Solar System*. Amsterdam: Elsevier, 2020. p. 413–437. DOI: 10.1016/B978-0-12-816490-7.00019-9.

GAIA COLLABORATION; PRUSTI, T.; DE BRUIJNE, J. H. J.; et al. "The Gaia mission". *Astronomy & Astrophysics*, 595, A1, 2016. DOI: 10.1051/0004-6361/201629272.

GAIA COLLABORATION; VALLENARI, A.; BROWN, A. G. A.; PRUSTI, T.; et al. "Gaia Data Release 3: Summary of the content and survey properties". *Astronomy & Astrophysics*, 674, A1, 2023. DOI: 10.1051/0004-6361/202243940.

HERALD, D.; BREIT, D.; DUNHAM, D.; FRAPPA, E.; et al. *Occultation lights curves*. VizieR On-line Data Catalog: B/occ. Strasbourg: CDS, 2016. DOI: 10.26093/cds/vizier.102033. Disponível em: `https://cdsarc.cds.unistra.fr/viz-bin/cat/B/occ`.

BREIMAN, Leo. "Random Forests". *Machine Learning*, 45(1), 5–32, 2001. `https://doi.org/10.1023/A:1010933404324`.

CHEN, Tianqi; GUESTRIN, Carlos. "XGBoost: A Scalable Tree Boosting System". In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. San Francisco: ACM, 2016. p. 785–794. `https://doi.org/10.1145/2939672.2939785`.

PROKHORENKOVA, Liudmila; GUSEV, Gleb; VOROBEV, Aleksandr; DOROGUSH, Anna Veronika; GULIN, Andrey. "CatBoost: unbiased boosting with categorical features". In: *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*, p. 6639–6649, 2018. arXiv:1706.09516.

STROBL, Carolin; BOULESTEIX, Anne-Laure; ZEILEIS, Achim; HOTHORN, Torsten. "Bias in random forest variable importance measures: Illustrations, sources and a solution". *BMC Bioinformatics*, 8, 25, 2007. `https://doi.org/10.1186/1471-2105-8-25`.

ZHONG, Xugang; LIN, Yanze; ZHANG, Wei; BI, Qing. "Predicting diagnosis and survival of bone metastasis in breast cancer using machine learning". *Scientific Reports*, 13, 18301, 2023. `https://doi.org/10.1038/s41598-023-45438-z`.

CONDORCET, Marie Jean Antoine Nicolas de Caritat, marquis de. *Essai sur l'application de l'analyse à la probabilité des décisions rendues à la pluralité des voix*. Paris: Imprimerie Royale, 1785.

CALIŃSKI, T.; HARABASZ, J. “A dendrite method for cluster analysis”. *Communications in Statistics*, 3(1), 1–27, 1974. https://doi.org/10.1080/03610927408827101.

NOCEDAL, Jorge; WRIGHT, Stephen J. *Numerical Optimization*. 2. ed. New York: Springer, 2006. (Springer Series in Operations Research and Financial Engineering). https://doi.org/10.1007/978-0-387-40065-5.

SAVITZKY, A.; GOLAY, M. J. E. “Smoothing and differentiation of data by simplified least squares procedures”. *Analytical Chemistry*, v. 36, n. 8, p. 1627–1639, 1964. DOI: 10.1021/ac60214a047.

WELCH, B. L. “The generalization of ‘Student’s’ problem when several different population variances are involved”. *Biometrika*, v. 34, n. 1-2, p. 28–35, 1947. DOI: 10.1093/biomet/34.1-2.28.

KOLMOGOROV, A. N. “Sulla determinazione empirica di una legge di distribuzione”. *Giornale dell’Istituto Italiano degli Attuari*, v. 4, p. 83–91, 1933.

SMIRNOV, N. “Table for estimating the goodness of fit of empirical distributions”. *Annals of Mathematical Statistics*, v. 19, n. 2, p. 279–281, 1948. DOI: 10.1214/aoms/1177730256.

VIRTANEN, P.; GOMMERS, R.; OLIPHANT, T. E.; et al. “SciPy 1.0: fundamental algorithms for scientific computing in Python”. *Nature Methods*, v. 17, p. 261–272, 2020. DOI: 10.1038/s41592-019-0686-2.

ASTROPY COLLABORATION; PRICE-WHELAN, A. M.; LIM, P. L.; et al. “The Astropy Project: Sustaining and Growing a Community-oriented Open-source Project and the Latest Major Release (v5.0) of the Core Package”. *The Astrophysical Journal*, v. 935, n. 2, 167, 2022. DOI: 10.3847/1538-4357/ac7c74.

GINSBURG, A.; SIPŐCZ, B. M.; BRASSEUR, C. E.; et al. “astroquery: An Astronomical Web-querying Package in Python”. *The Astronomical Journal*, v. 157, n. 3, 98, 2019. DOI: 10.3847/1538-3881/aafc33.

McKINNEY, W. “Data Structures for Statistical Computing in Python”. In: *Proceedings of the 9th Python in Science Conference*, p. 56–61, 2010. DOI: 10.25080/Majora-92bf1922-00a.

# Apêndice A

# Resultados experimentais complementares

O Capítulo 5 apresenta em detalhe três experimentos — a linha de base com o conjunto completo de *features* (Experimento 1), a configuração enxuta de 11 *features* usada como referência operacional (Experimento 2) e o teste de generalização exclusivamente em curvas reais (Experimento 3) —, além de sintetizar todos os resultados na Tabela 5.10. Este apêndice reúne os **experimentos intermediários** cujas conclusões já foram incorporadas àquela síntese, mas que se documentam aqui por completude: a redução progressiva do conjunto de *features* para 14 (Experimento 4) e 13 variáveis (Experimento 5), a ablação isolada da *feature* `kmeans_centroid_dist` (Experimento 6) e uma variante do Experimento 3 com divisão 65%/35%, utilizada como ablação do tamanho do conjunto de treino. Todas as métricas referem-se ao conjunto de teste e seguem o protocolo descrito na Seção 5.4.

## A.1 Experimento 4: subconjunto de 14 *features*

No Experimento 4 foi utilizado um subconjunto reduzido de **14 *features***, removendo amplitude, desvio padrão do fluxo bruto, máximo do Savitzky-Golay, todas as derivadas e duas métricas observacionais (`Occ_flux_min` e `Occ_flux_min_over_baseline`). O objetivo é diminuir o risco de *overfitting* e melhorar a razão entre número de variáveis e tamanho do conjunto de treino. A Tabela A.1 apresenta os resultados.

O melhor F1-score foi alcançado pelo **CatBoost** (F1 = 0,9906). As métricas permanecem muito altas, indicando que a redução de 28 para 14 *features* preserva a informação discriminativa. A Figura A.1 apresenta as matrizes de confusão.

Tabela A.1: Métricas de desempenho no conjunto de teste — Experimento 4. Subconjunto de 14 *features*, divisão 80%/20%.

| Modelo | Acurácia | Precisão | Sensibilidade | F1-score | AUC-ROC |
|---|---|---|---|---|---|
| CatBoost | **0,9912** | 0,9937 | 0,9875 | **0,9906** | 0,9996 |
| Random Forest | 0,9882 | 0,9937 | 0,9813 | 0,9874 | 0,9997 |
| XGBoost | 0,9882 | 0,9875 | 0,9875 | 0,9875 | 0,9996 |
| Regressão Log. | 0,9882 | 1,0000 | 0,9750 | 0,9873 | 1,0000 |

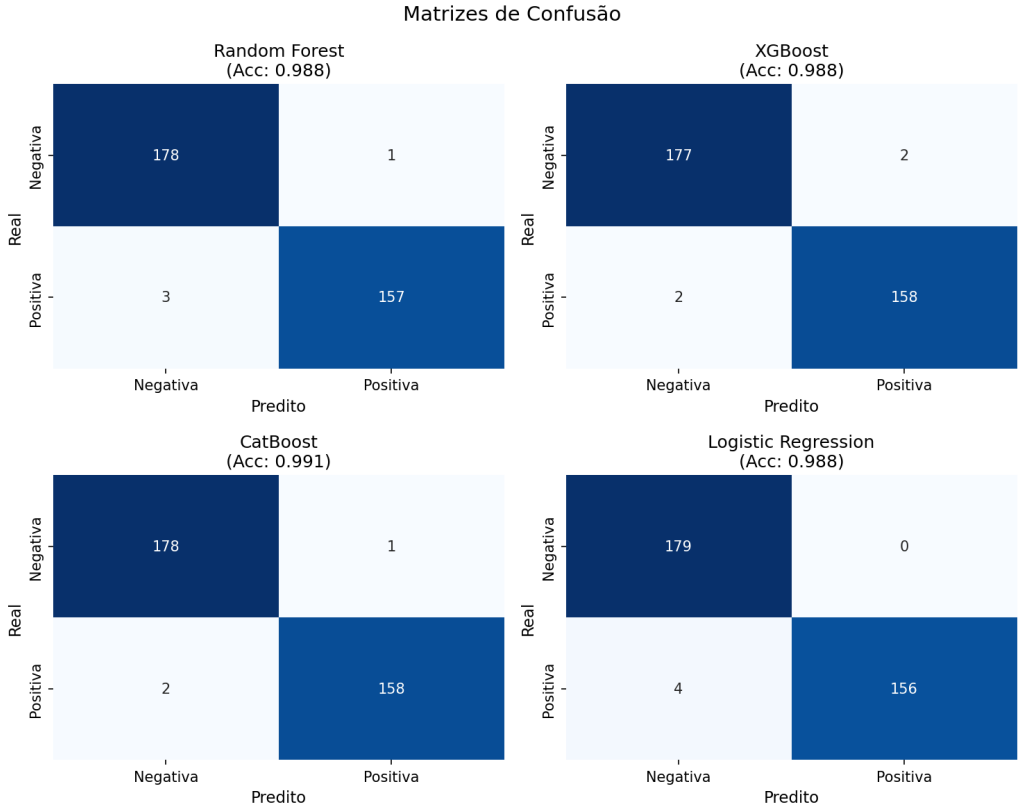



Figura A.1: Matrizes de confusão dos quatro modelos no Experimento 4 (14 *features*).

## A.2 Experimento 5: 13 *features* (sem `Feature_Savgol_Min`)

No Experimento 5 foi removida a *feature* `Feature_Savgol_Min`, mantendo 13 *features*. O objetivo é avaliar a contribuição dessa variável, que costuma figurar entre as mais relevantes nos gráficos de importância. A Tabela A.2 resume as métricas.

Tabela A.2: Métricas de desempenho no conjunto de teste — Experimento 5. 13 *features* (sem `Feature_Savgol_Min`), divisão 80%/20%.

| Modelo | Acurácia | Precisão | Sensibilidade | F1-score | AUC-ROC |
|---|---|---|---|---|---|
| Random Forest | **0,9912** | 1,0000 | 0,9813 | **0,9905** | 0,9998 |
| CatBoost | 0,9882 | 1,0000 | 0,9750 | 0,9873 | 0,9998 |
| Regressão Log. | 0,9882 | 1,0000 | 0,9750 | 0,9873 | 1,0000 |
| XGBoost | 0,9853 | 0,9874 | 0,9813 | 0,9843 | 0,9997 |

O **Random Forest** lidera com F1 = 0,9905. Em relação ao Experimento 4, a remoção de `Feature_Savgol_Min` causa queda moderada em alguns modelos, mas o desempenho global permanece elevado, indicando que as demais *features* absorvem parte da informação. As matrizes de confusão estão na Figura A.2. Vale notar que a análise de ajuste de limiar e a validação cruzada apresentadas no Capítulo 5 (Seções 5.8 e 5.11) foram conduzidas sobre esta configuração de 13 *features*.

## A.3 Experimento 6: ablação isolada do ~~K-Means~~K-means

No Experimento 6 foi removida `kmeans_centroid_dist`, mantendo 12 *features* (sem `Feature_Savgol_Min`), com divisão 80%/20%. A pergunta investigada é se a alta importância atribuída a essa variável pelos modelos de árvore reflete uma contribuição insubstituível ou se as demais *features* absorvem sua capacidade preditiva. A Tabela A.3 apresenta as métricas.

Tabela A.3: Métricas de desempenho no conjunto de teste — Experimento 6. 12 *features* (sem `kmeans_centroid_dist`), divisão 80%/20%.

| Modelo | Acurácia | Precisão | Sensibilidade | F1-score | AUC-ROC |
|---|---|---|---|---|---|
| Random Forest | **0,9912** | 0,9937 | 0,9875 | **0,9906** | 0,9998 |
| XGBoost | **0,9912** | 0,9937 | 0,9875 | **0,9906** | 0,9997 |
| CatBoost | 0,9882 | 0,9937 | 0,9813 | 0,9874 | 0,9997 |
| Regressão Log. | 0,9882 | 1,0000 | 0,9750 | 0,9873 | 1,0000 |

Random Forest e XGBoost empatam com F1 = 0,9906 — desempenho igual ou superior ao da configuração que mantém o ~~K-Means~~K-means. A Figura A.3 apresenta as matrizes de confusão.

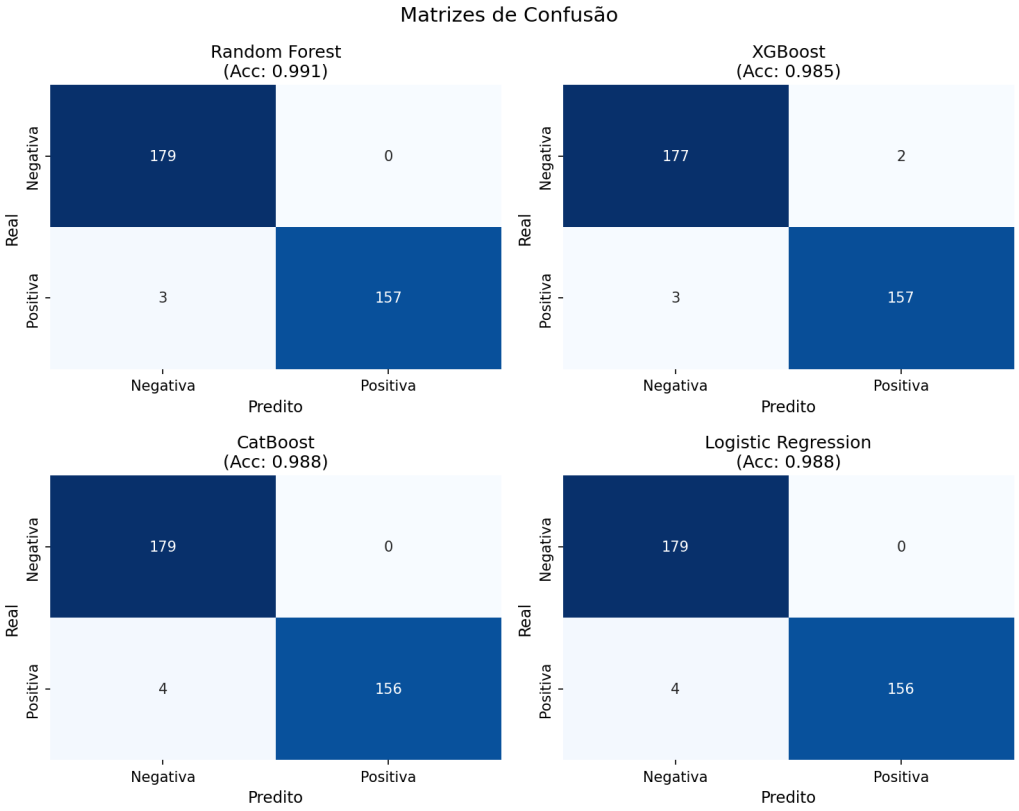



Figura A.2: Matrizes de confusão dos quatro modelos no Experimento 5 (13 *features*, sem `Feature_Savgol_Min`).

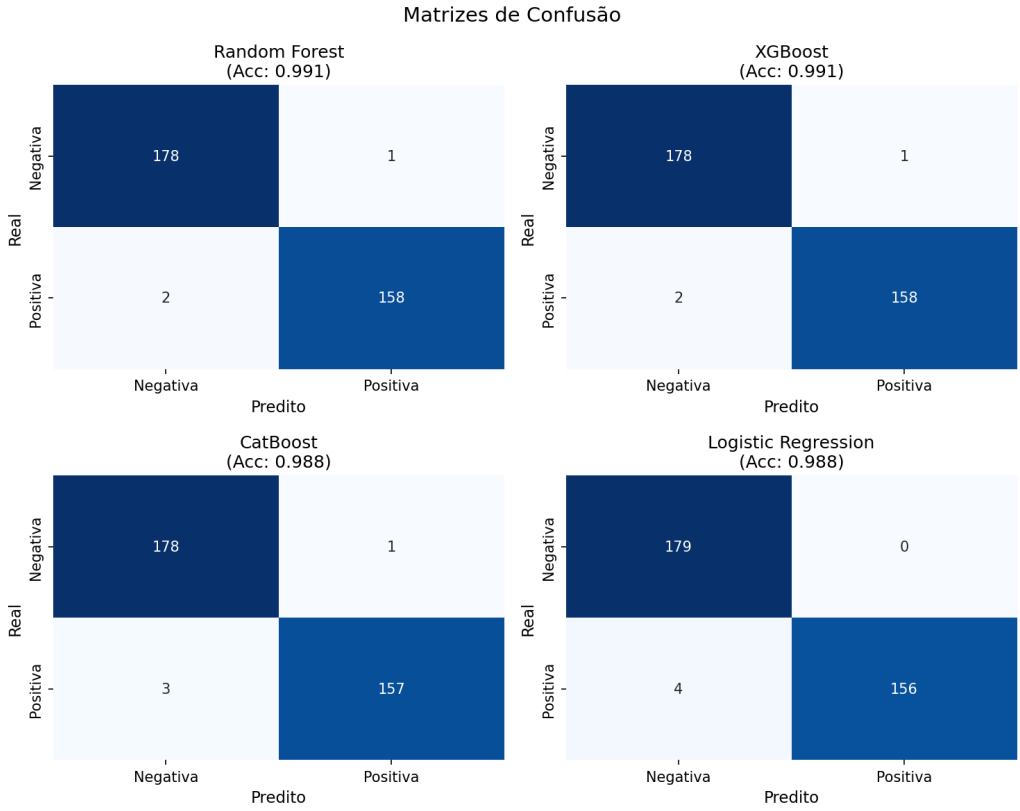



Figura A.3: Matrizes de confusão dos quatro modelos no Experimento 6 (12 *features*, sem `kmeans_centroid_dist`).

A Tabela A.4 confronta os F1-scores do Experimento 5 (13 *features*, com `kmeans`), do Experimento 6 (12 *features*, sem `kmeans`) e do Experimento 2 (11 *features*, sem `Feature_Savgol_Min` e sem `kmeans`), evidenciando que a remoção da *feature* de ~~K-Means~~ K-means não degrada o desempenho global.

Tabela A.4: Comparação do F1-score: com `kmeans` (Exp. 5), sem `kmeans` (Exp. 6) e ablação combinada (Exp. 2). Divisão 80%/20%.

| **Modelo** | **Exp. 5 (13 feat.)** | **Exp. 6 (12 feat., s/ kmeans)** | **Exp. 2 (11 feat.)** |
|---|---|---|---|
| Random Forest | **0,9905** | **0,9906** | 0,9874 |
| XGBoost | 0,9843 | **0,9906** | **0,9906** |
| CatBoost | 0,9873 | 0,9874 | **0,9905** |
| Regressão Log. | 0,9873 | 0,9873 | 0,9841 |

A importância antes concentrada em `kmeans_centroid_dist` é redistribuída entre variáveis como `Feature_Savgol_std`, `Occ_depth`, `Occ_duration_s` e `MinLogP_Ttest`. Esse resultado sustenta a conclusão discutida no Capítulo 5: `kmeans_centroid_dist` e `Feature_Savgol_std` capturam o mesmo fenômeno físico (a dispersão do fluxo causada pelos dois patamares dentro e fora do *dip*), de modo que a alta importância atribuída ao ~~K-Means~~ K-means não reflete informação insubstituível. Na prática, isso permite simplificar a *pipeline* eliminando o passo de ~~K-Means~~ K-means sem perda de desempenho.

Por fim, a Figura A.4 apresenta as matrizes de confusão da configuração enxuta de 11 *features* (Experimento 2), discutida em detalhe na Seção 5.6.1 do Capítulo 5.

## A.4 Variante do Experimento 3 com divisão 65/35: ablação do tamanho do treino

O Experimento 3 do Capítulo 5 avalia a generalização para curvas exclusivamente reais sob a mesma divisão 80%/20% dos demais experimentos. Esta seção documenta uma variante com divisão 65%/35%, que cumpre dois papéis: (i) ampliar o conjunto de teste real avaliado (347 curvas: 281 positivas e 66 negativas, contra 198 na configuração principal), reduzindo a incerteza amostral das métricas; e (ii) servir de **ablação do tamanho do conjunto de treino**. Ao remover cerca de 15% dos dados de treinamento, mede-se a sensibilidade do desempenho à quantidade de exemplos disponíveis.

A comparação com a configuração principal (Tabela 5.7, Capítulo 5) é direta: com 15% menos dados de treino, o melhor F1 recua de 0,9906 para 0,9838 (queda de apenas $\approx$0,7 ponto percentual) e a AUC-ROC permanece $\geqslant$ 0,9956 em todos os modelos. Essa estabilidade indica que a curva de aprendizado dos modelos está **saturada** na região de operação: o desempenho depende fracamente do volume de treino, comportamento incompatível com sobreajuste dominante (um modelo sobreajustado degradaria de forma acentuada ao perder dados de ajuste). O resultado converge com a validação cruzada

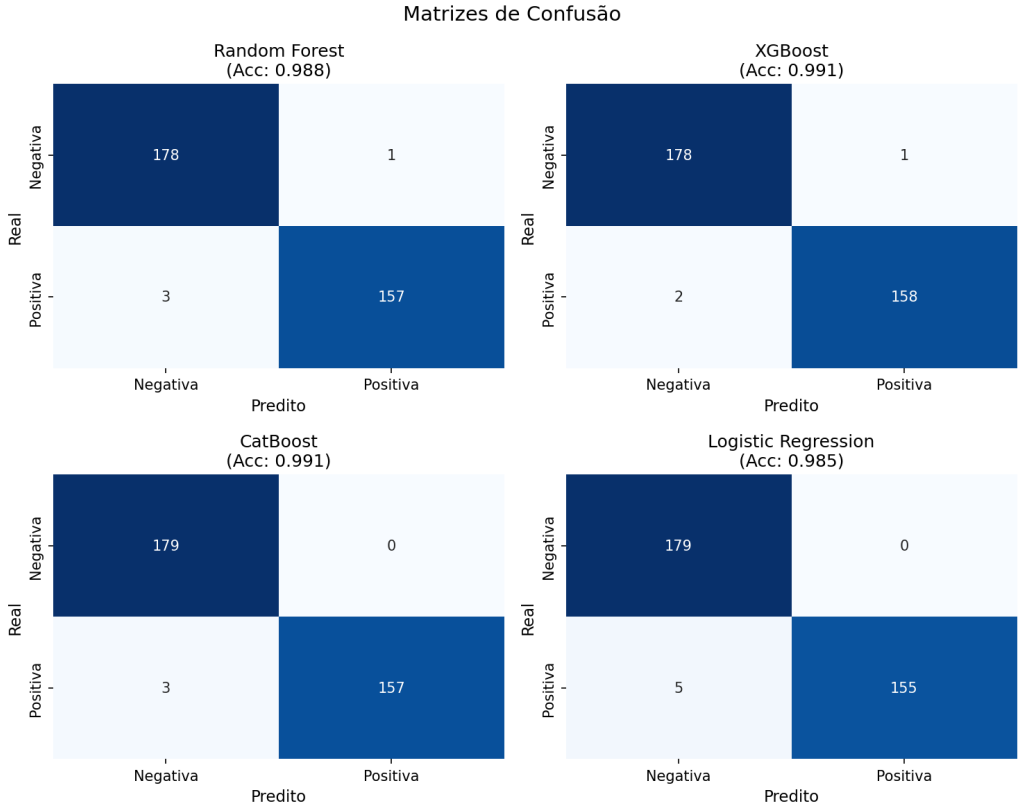



Figura A.4: Matrizes de confusão dos quatro modelos no Experimento 2 (11 *features*).

Tabela A.5: Métricas de desempenho no conjunto de teste — variante do Experimento 3 com divisão 65%/35% (teste somente em curvas reais: 347 curvas, 281 positivas e 66 negativas).

| **Modelo** | **Acurácia** | **Precisão** | **Sensibilidade** | **F1-score** | **AUC-ROC** |
|---|---|---|---|---|---|
| XGBoost | **0,9741** | 0,9928 | 0,9751 | **0,9838** | 0,9968 |
| Random Forest | 0,9683 | 0,9856 | 0,9751 | 0,9803 | 0,9956 |
| CatBoost | 0,9654 | 0,9891 | 0,9680 | 0,9784 | 0,9967 |
| Regressão Log. | 0,9597 | 0,9926 | 0,9573 | 0,9746 | 0,9959 |

estratificada (Seção 5.11), que apresenta desvios padrão pequenos entre dobras. As Figuras A.5 e A.6 apresentam as matrizes de confusão e as curvas ROC desta variante.

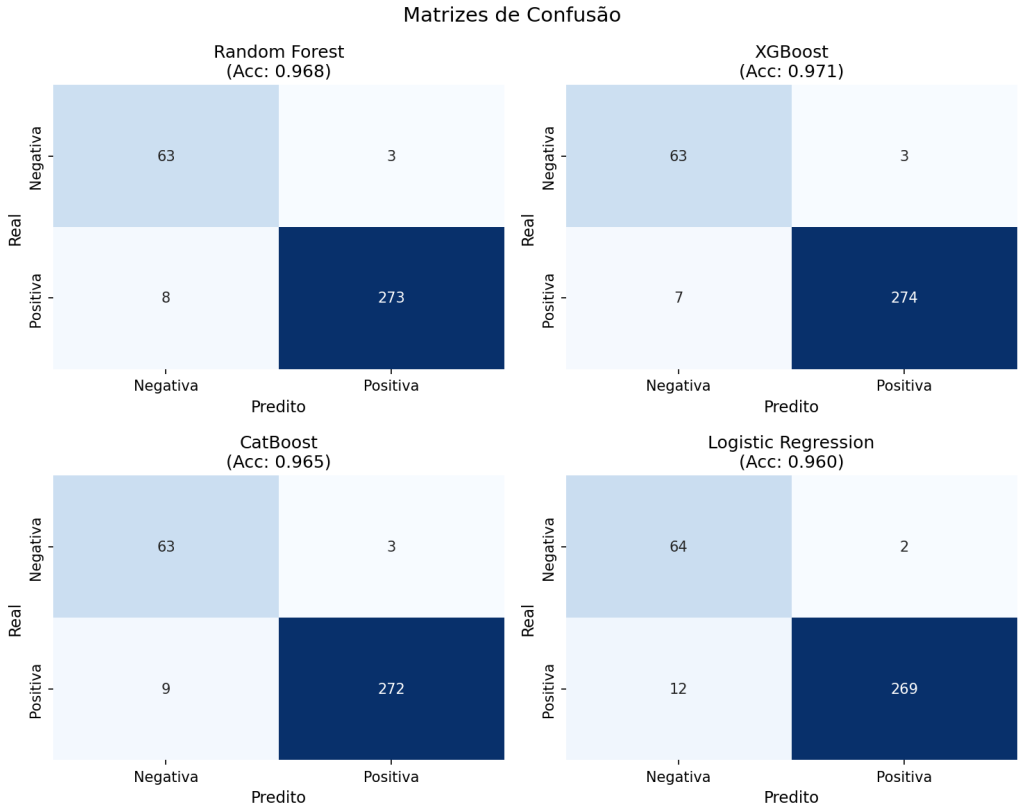



Figura A.5: Matrizes de confusão dos quatro modelos na variante 65/35 do Experimento 3 (teste somente em curvas reais).

## A.5 Curvas ROC dos experimentos complementares

As curvas ROC do Experimento 1 são apresentadas no Capítulo 5 (Figura 5.6); as da variante 65/35 do Experimento 3 estão na Seção A.4 (Figura A.6). Para completude, as Figuras A.7–A.10 reúnem as curvas ROC dos demais experimentos (2, 4, 5 e 6). Em todos os casos a AUC permanece acima de 0,999, confirmando a elevada separabilidade das classes no conjunto de teste misto.

## A.6 Importância das *features* nos Experimentos 4 e 5

O Capítulo 5 discute a importância das variáveis com base na figura do Experimento 1 (conjunto completo). As Figuras A.11 e A.12 apresentam os gráficos análogos para os

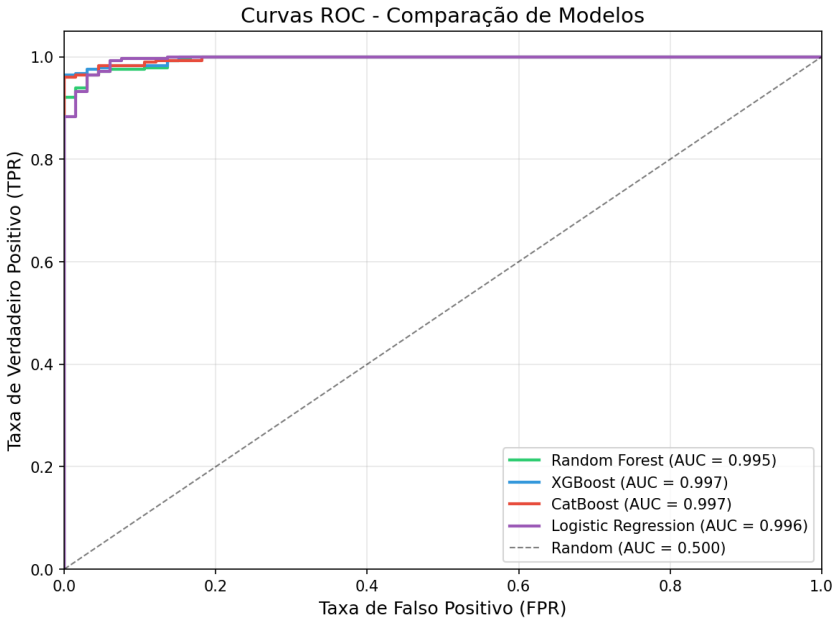



Figura A.6: Curvas ROC — variante 65/35 do Experimento 3 (teste somente em curvas reais).

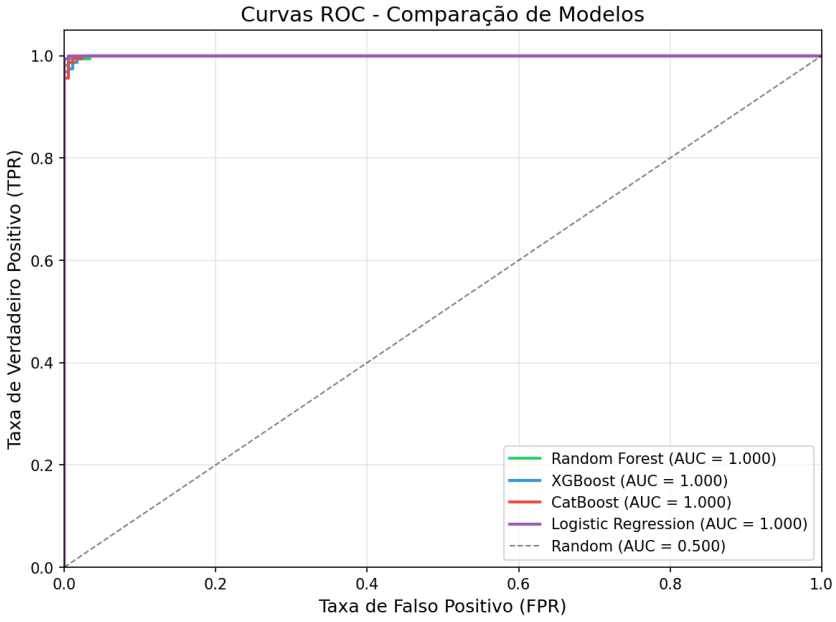



Figura A.7: Curvas ROC — Experimento 4 (14 *features*).

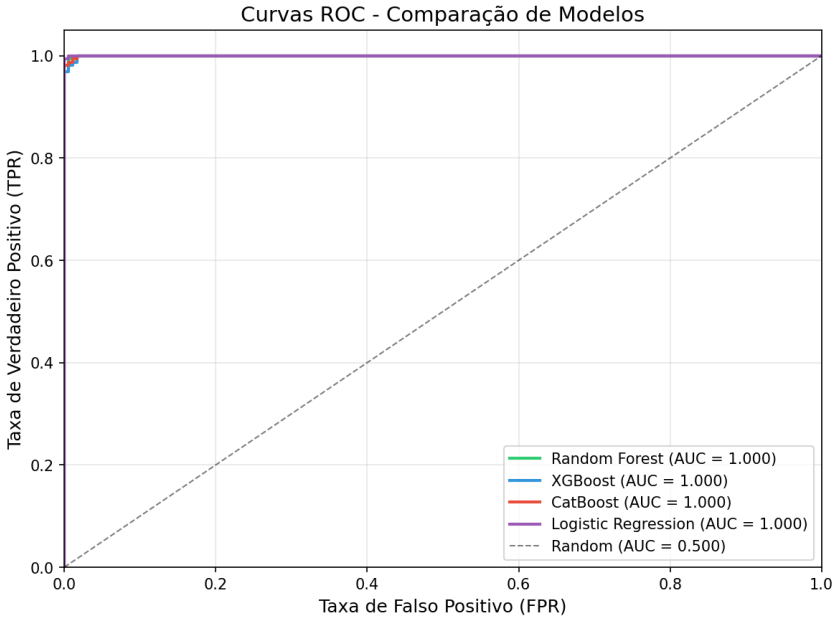



Figura A.8: Curvas ROC — Experimento 5 (13 *features*).

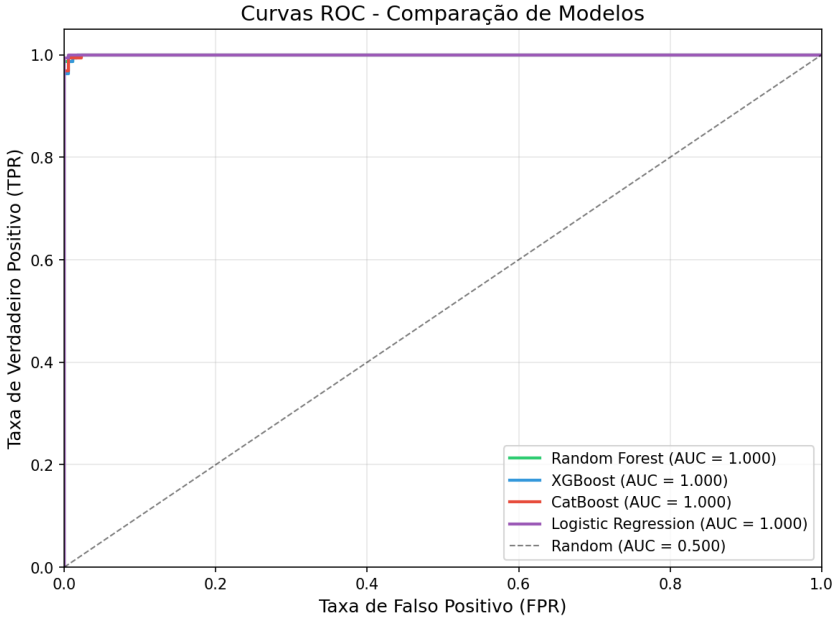



Figura A.9: Curvas ROC — Experimento 6 (12 *features*, sem `kmeans`).

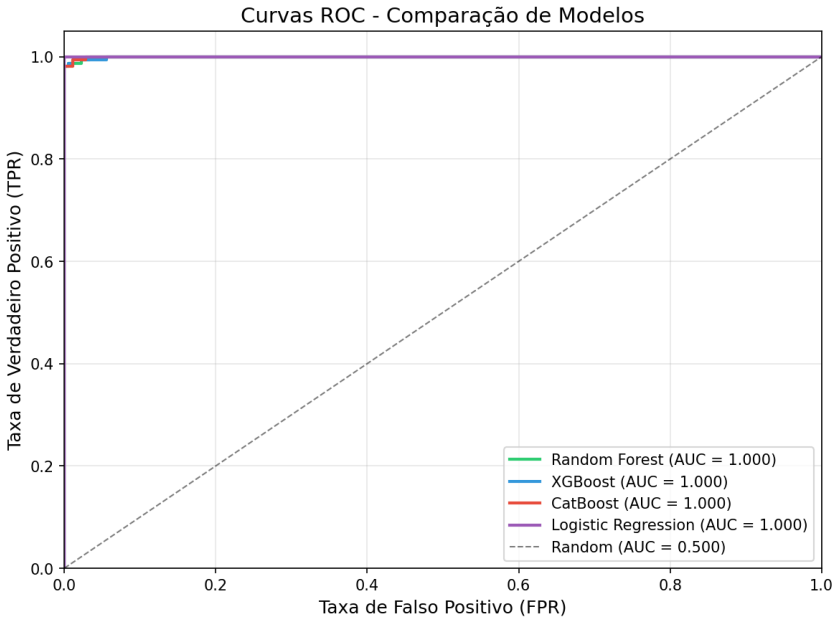
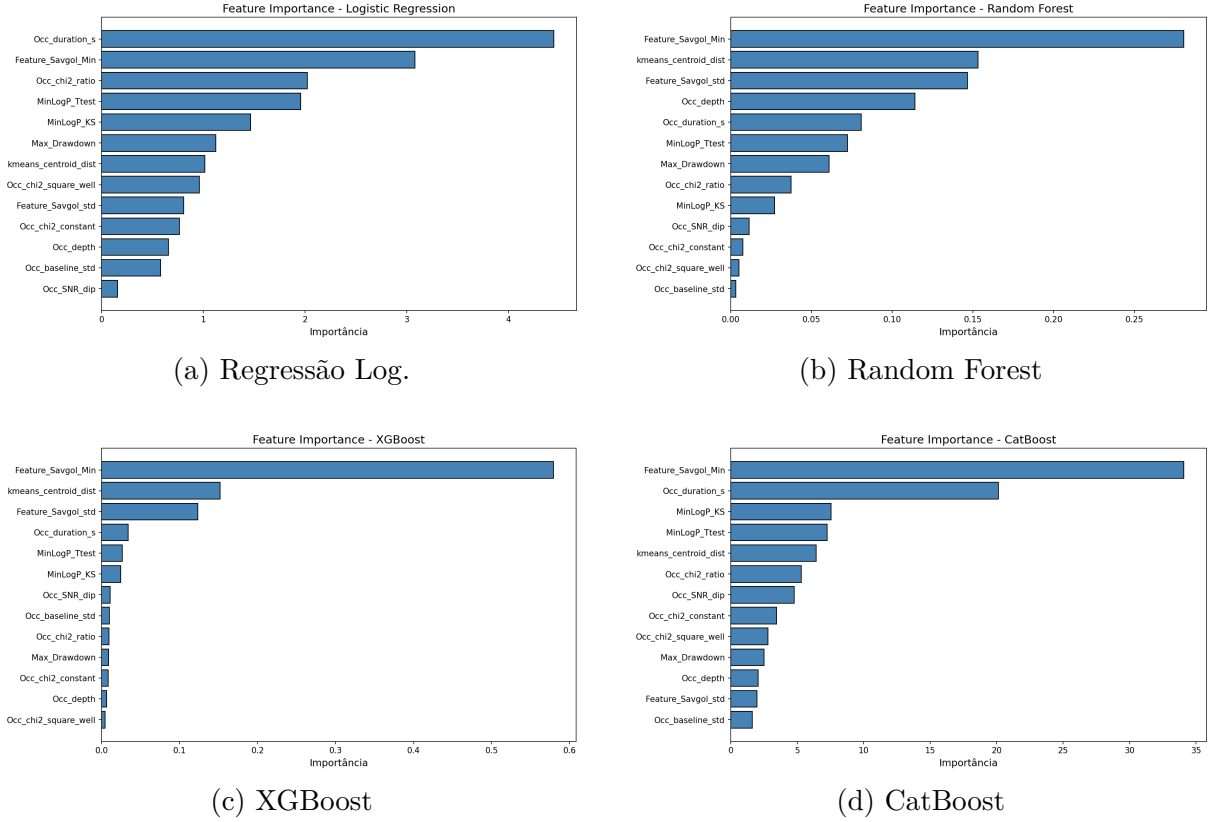



Figura A.10: Curvas ROC — Experimento 2 (11 *features*).

Experimentos 4 e 5, úteis para acompanhar como a relevância relativa das variáveis se redistribui à medida que o conjunto é reduzido. A interpretação dos valores deve considerar as diferenças metodológicas entre os métodos de cada modelo, discutidas na Seção 5.3.

**Experimento 4 (14 *features*).**




(a) Regressão Log.

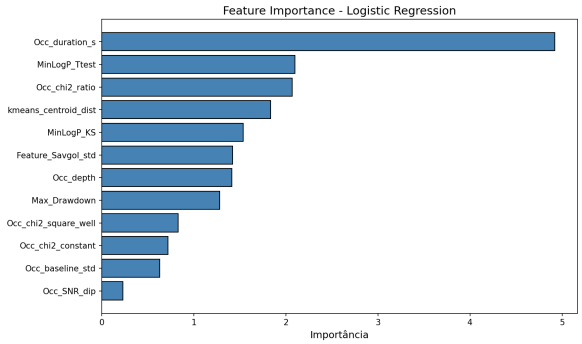
(b) Random Forest

(c) XGBoost

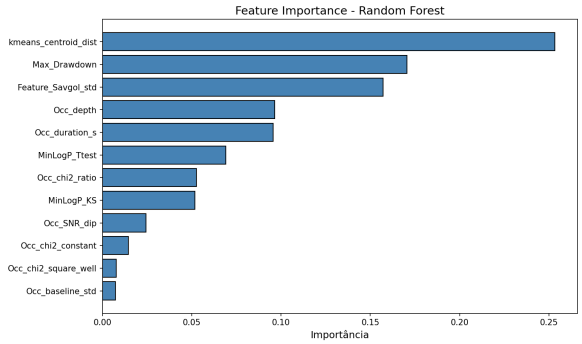
(d) CatBoost

Figura A.11: Importância de variáveis por modelo — Experimento 4 (14 *features*). (a) Regressão Logística, (b) Random Forest, (c) XGBoost, (d) CatBoost.

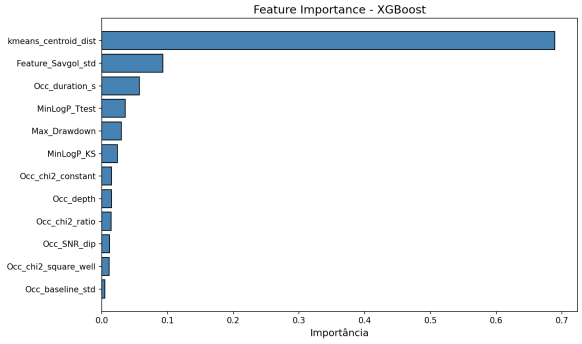
**Experimento 5 (13 *features*, sem `Feature_Savgol_Min`).**



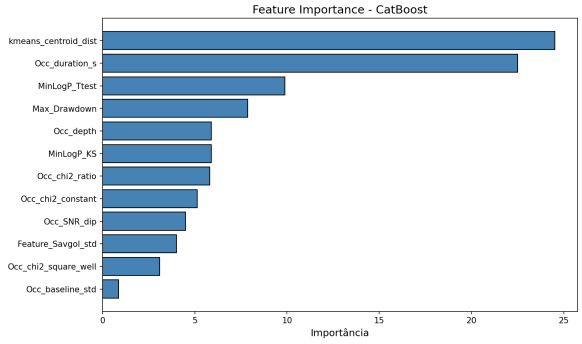

(a) Regressão Log.




(b) Random Forest




(c) XGBoost




(d) CatBoost

Figura A.12: Importância de variáveis por modelo — Experimento 5 (13 *features*, sem `Feature_Savgol_Min`). (a) Regressão Logística, (b) Random Forest, (c) XGBoost, (d) CatBoost.

# Apêndice B

# Hiperparâmetros dos classificadores

Este apêndice documenta os hiperparâmetros utilizados no treinamento dos classificadores (Regressão Logística, Random Forest, XGBoost, CatBoost) descritos no Capítulo 4. Os valores foram fixados no ~~script~~ *script* `train_model.py` e utilizados em todos os experimentos, sem busca em grade nem validação cruzada para seleção de hiperparâmetros; a semente aleatória (~~*random_state*~~`random_state`=42) foi mantida para reprodutibilidade.

**Regressão Logística:**

- Regularização: Ridge ($\ell_2$), valor padrão do scikit-learn (`penalty='l2'`).
- Força da regularização: $C = 1{,}0$ (valor padrão).
- Solver: `lbfgs` (padrão para regularização $\ell_2$).
- ~~*max_iter*~~`max_iter`: 1000.
- ~~*class_weight*~~`class_weight`: `balanced`.
- ~~*random_state*~~`random_state`: 42.

**Random Forest:**

- ~~*n_estimators*~~`n_estimators`: 100.
- ~~*max_depth*~~`max_depth`: 10.
- ~~*min_samples_split*~~`min_samples_split`: 5.
- ~~*min_samples_leaf*~~`min_samples_leaf`: 2.
- ~~*class_weight*~~`class_weight`: `balanced`.
- ~~*random_state*~~`random_state`: 42.

**XGBoost:**

- ~~*n_estimators*~~`n_estimators`: 100.
- ~~*max_depth*~~`max_depth`: 6.
- ~~*learning_rate*~~`learning_rate`: 0,1.
- ~~*subsample*~~`subsample`: 0,8.
- ~~*colsample_bytree*~~`colsample_bytree`: 0,8.
- Ponderação de classes: utilizada a opção padrão do XGBoost (sem ~~*scale_pos_weight*~~ `scale_pos_weight` explícito no ~~script~~*script*; o desbalanceamento é tratado indiretamente pelo algoritmo).
- ~~*random_state*~~`random_state`: 42.
- ~~*use_label_encoder: False; eval_metric*~~`use_label_encoder: False; eval_metric`: `logloss`.

**CatBoost:**

- ~~*iterations*~~`iterations`: 100.
- ~~*depth*~~`depth`: 6.
- ~~*learning_rate*~~`learning_rate`: 0,1.
- ~~*auto_class_weights*~~`auto_class_weights`: `Balanced`.
- ~~*random_state*~~`random_state`: 42.

Não foi utilizada busca em grade (~~Grid Search~~*grid search*) nem validação cruzada para escolha desses hiperparâmetros; os valores acima correspondem a uma configuração *baseline* fixa, mantida em todos os experimentos para permitir uma comparação justa entre eles, variando-se apenas o conjunto de *features* e a composição do conjunto de teste. Para trabalhos futuros, pode-se explorar validação cruzada estratificada com otimização de F1-score ou AUC-ROC para refinar os hiperparâmetros.

# Apêndice C

# Ambiente de software e versões

Para reprodutibilidade dos experimentos, segue a lista das principais ferramentas e bibliotecas utilizadas no projeto. As versões abaixo referem-se ao ambiente em que ~~o~~ a *pipeline* de treinamento (`train_model.py`) e a construção do ~~dataset foram executados~~*dataset* foram executadas, conforme arquivo `requirements.txt` (ou `requirements_frozen.txt`) do repositório do projeto.

**Pacotes principais:**

- Python: 3.x (recomenda-se 3.10 ou superior para compatibilidade com as bibliotecas listadas).
- numpy 1.26.4
- pandas 2.2.2
- scikit-learn 1.4.2
- xgboost 2.0.3
- catboost 1.2.5
- matplotlib 3.8.4
- seaborn 0.13.2
- joblib 1.4.2
- astropy 5.3.1
- scipy 1.13.1

**Outras dependências** (aquisição de dados, persistência, testes): SQLAlchemy 2.0.31, requests 2.32.3, python-dotenv 1.0.1, pyyaml 6.0.1, pytest 7.4.3. A aquisição de curvas de luz a partir do VizieR e do Grupo do Rio pode utilizar ainda `astroquery` e `aiohttp`;

nesse caso, as versões devem ser registradas com `pip freeze` no ambiente em que os ~~scripts~~ *scripts* de ingestão forem executados.

Para replicar o ambiente de treinamento, recomenda-se criar um ambiente virtual (venv ou conda) e instalar as dependências com `pip install -r requirements.txt` (ou `pip install -r requirements_frozen.txt` para versões fixas). A data de geração do ambiente utilizado nos experimentos do Capítulo 5 deve ser registrada pelo autor ao finalizar a dissertação.